

编 号：_____
审定成绩：_____

重庆邮电大学 毕业设计（论文）

中文题目	基于生成对抗网络的道路建筑图像 合成模型的设计与实现
英文题目	Design and Implementation of Road Building Image Synthesis Models Based on Generate Adversarial Networks
学院名称	计算机科学与技术学院/人工智能学院
学生姓名	张晓珊
专 业	计算机科学与技术
班 级	04931901
学 号	2019211450
指导教师	王豪 副教授
答 辩 组 负 责 人	于洪 教授

二〇二三年 六月

重庆邮电大学教务处制

计算机学院/人工智能学院本科毕业设计（论文）诚信承诺书

本人郑重承诺：

我向学院呈交的论文《基于生成对抗网络的道路建筑图像合成模型的设计与实现》，是本人在指导教师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明并致谢。本人完全意识到本声明的法律结果由本人承担。

年级 2019

专业 计算机科学与技术

班级 04931901

承诺人签名

年 月 日

学位论文版权使用授权书

本人完全了解重庆邮电大学有权保留、使用学位论文纸质版和电子版的规定，即学校有权向国家有关部门或机构送交论文，允许论文被查阅和借阅等。本人授权重庆邮电大学可以公布本学位论文的全部或部分内容，可编入有关数据库或信息系统进行检索、分析或评价，可以采用影印、缩印、扫描或拷贝等复制手段保存、汇编本学位论文。

（注：保密的学位论文在解密后适用本授权书。）

学生签名：

指导老师签名：

日期： 年 月 日

日期： 年 月 日

摘要

随着计算机视觉领域的飞速革新，自动驾驶技术得到快速发展。基于深度学习的计算机视觉在自动驾驶感知、规划等多个应用中显现出举足轻重的高效性，这些应用依赖大量的车载摄像头数据，而这些数据包含了用户的个人信息、位置隐私，存在严重的隐私泄露问题。特别是，可以从摄像头数据中挖掘道路建筑信息，以识别车辆位置信息，甚至泄露特殊地标位置。

本文针对以上问题，设计基于生成对抗网络的道路建筑图像合成模型，通过合成与真实图像结构相似但细节不同的图像，可以防止合成图像中的敏感区域被识别出，从而达到隐私保护的目的。本文的主要工作如下：

1. 创新性地引入 SSIM 结构损失函数作为约束条件，利用 FCN8s 语义分割模型提取真实道路建筑物图像敏感信息，并构建基于生成对抗网络的道路建筑图像合成模型，其生成器基于 U-net 结构，判别器为卷积神经网络；

2. 使用 pytorch 深度学习框架搭建并训练本文设计的合成模型，针对构建的道路建筑合成模型进行了消融对比实验，从图像质量、FCN-scores、AMT 分数三个方面评估本文构建的生成模型与现有最优的两个模型合成图像的适用性和隐私保护效力，结果表明本文构建的模型能够较好地折衷合成图像的适用性和隐私保护效力；

3. 构建了保护车载摄像头记录的道路建筑图像位置隐私的图像处理系统，并基于 Flask 完成了模型的部署与可视化。该系统的处理流程是：首先将真实图像输入进语义分割模型，得到掩盖敏感信息的标签图，然后将标签图和真实图像同时输入图像合成模型，最后得到一张结构与原图相似但是敏感部位细节不同的合成图像。

综上，本文通过模型设计、实验对比和模型部署实现三个工作的展开，构建了能够保护自动驾驶汽车位置隐私的道路建筑图像合成模型与处理系统。

关键词：生成对抗网络，图像生成，图像隐私保护，图像语义分割，自动驾驶

Abstract

With the rapid innovation in computer vision and deep learning, the technology of autonomous driving has developed quickly. Deep learning-based computer vision has demonstrated significant efficiency in various applications such as autonomous driving perception and planning. These applications rely heavily on vehicle-mounted camera data, which contains users' personal information and location privacy, leading to serious privacy concerns. Particularly, road and building information can be extracted from camera data, which can be used to identify vehicle location or even reveal the location of specific landmarks.

This thesis proposes a design and implementation of a road and building image synthesis model based on generative adversarial networks to address the aforementioned issue. By synthesizing images that have similar structures but different details from real images, it is possible to prevent the recognition of sensitive areas in the synthesized images, thereby achieving the goal of privacy protection. The main research work of this thesis is as follows:

1. Innovatively, the Structural Similarity Index (SSIM) loss function is introduced as a constraint. The FCN8s semantic segmentation model is utilized to extract sensitive information from real road and building images. Subsequently, a road and building image synthesis model based on a generative adversarial network (GAN) is constructed, where the generator adopts the U-net architecture and the discriminator employs a convolutional neural network (CNN).

2. The synthesis model proposed in this thesis was built and trained using the PyTorch deep learning framework. In order to evaluate the applicability and privacy protection effectiveness of the generated images by our proposed model, ablation comparative experiments were conducted. These experiments were carried out based on three aspects: image quality, FCN-scores, and AMT scores. The results demonstrated that the model constructed in this thesis achieved a good balance between synthesis applicability and privacy protection effectiveness when compared to the two existing state-of-the-art models.

3. An image processing system was developed to protect the location privacy of road and building images recorded by onboard cameras. The system was deployed and

visualized using Flask. The processing workflow of the system is as follows: First, real images are fed into a semantic segmentation model, which generates a masked label map that conceals sensitive information. Next, both the masked label map and the real image are simultaneously input into the image synthesis model. Finally, a synthesized image is obtained that preserves the overall structure similar to the original image but with different details in sensitive areas.

In summary, this thesis constructs a road building image synthesis model and processing framework that can protect the location privacy of autonomous vehicles through three tasks: model design, experimental comparison, and model deployment.

Keywords: Generative Adversarial Networks, Image generation, image privacy protection, image segmentation, Autonomous driving

目录

第 1 章 引言	1
1.1 研究背景和意义	1
1.2 国内外研究现状	2
1.2.1 自动驾驶汽车中的隐私保护	2
1.2.2 图像隐私保护	4
1.2.3 生成对抗网络在隐私保护中的应用	6
1.3 主要内容和工作安排	7
1.4 论文组织结构	8
第 2 章 相关理论与技术	9
2.1 图像语义分割	9
2.1.1 图像语义分割简介	9
2.1.2 全卷积神经网络	11
2.2 生成对抗网络	16
2.2.1 生成对抗网络原理	16
2.2.2 生成对抗网络的变体	19
2.3 Pytorch 深度学习框架	22
2.4 Flask 轻量级 PythonWeb 框架	24
2.4.1 Flask 简介	24
2.4.2 使用 Flask 部署 pytorch 模型	26
2.5 本章小结	26
第 3 章 模型设计与实现	27
3.1 模型架构	27
3.2 基于 FCN8s 的语义分割模型	28
3.2.1 模型的结构	28
3.2.2 模型的实现	29
3.3 基于生成对抗网络的道路建筑合成模型	31

3.3.1 生成器	31
3.3.2 判别器	33
3.3.3 损失函数	33
3.3.4 模型的结构	35
3.3.5 模型的实现	36
3.4 本章小结	38
第 4 章 实验分析与系统部署	39
4.1 实验过程	39
4.1.1 数据集准备	39
4.1.2 环境配置	40
4.1.3 FCN8s 模型训练	41
4.1.4 道路建筑图像合成模型训练	42
4.2 结果分析	46
4.2.1 评估指标与对比模型	46
4.2.2 图像适用性和隐私保护效力分析	47
4.2.3 图像质量分析	48
4.3 系统部署	51
4.4 本章小结	54
第 5 章 总结与展望	55
5.1 主要工作与总结	55
5.1.1 主要工作	55
5.1.2 总结	56
5.2 后续研究工作展望	57
参考文献	58
作者介绍	62
致谢	63
附录 A 合成图像效果	65
附录 B 英文翻译	66

第 1 章 引言

1.1 研究背景和意义

随着卷积神经网络（Convolutional Network, CNN）^[1]、生成对抗式网络（Generative Adversarial Networks, GANs）^[2]以及 Transformer^[3]等深度学习算法的提出，计算机视觉技术变得更加先进、可靠和高效，从而促进了计算机视觉技术在市场应用领域的蓬勃发展，据 Future Market Insights^[4]研究报告指出计算机视觉市场规模预计到 2023 年将达 129.1 亿美元。图像视频识别、物体检测、实例分割、面部识别、手势识别和增强现实等计算机视觉应用在医疗、制造、安全和自动驾驶等领域中得到广泛的应用。特别地，在自动驾驶发展早期，以激光雷达传感器解决方案为主，近年来得益于计算机视觉技术的准确高效性不断提升，以摄像头为主的计算机视觉传感解决方案已成为自动驾驶面向量产的关键^[5]。

自动驾驶技术取得的瞩目成就离不开感知（数据）和决策（算法）这两个关键要素。一辆自动驾驶汽车的构建一般由感知层、决策层和执行层三个方面组成，几乎每一层的实现都离不开深度学习、机器学习模型的支撑，特别是感知层，依赖于计算机视觉模型完成场景分割、物体检测等任务。这些技术为自动驾驶带来了便利的同时也涉及到了大量的隐私问题。训练自动驾驶模型通常需要收集大量环境数据，这些数据来自车载传感器。例如，可以从车载 GPS 获取车辆位置轨迹数据；从车内摄像头获取用户行为数据；从车前后摄像头获取周围道路建筑的信息；从车前后保险杠上的雷达识别交通情况等。这些数据不仅仅用于指导自动驾驶，还可以应用于智慧交通、智慧城市等领域。

当前全世界大部分地区已经发布一些数据保护法保护个人数据的使用。其中包括欧洲 GDPR、加拿大的 PIPEDA、加利福尼亚的 CCPA 和 APPI（日本）^[6]。当立法与现有的道路交通法相结合时，可以帮助减轻自动驾驶汽车用户的隐私忧虑。但难免会有不法分子利用法律漏洞，攻击携带大量数据的自动驾驶汽车，攻击者可以通过数据挖掘获取车辆、驾驶员、乘客甚至道路环境的信息。比如，攻击者可以从用户行为数据中推断用户性别、年龄和爱好等信息，从 GPS 数据中获取用户的地点位置、家庭住址、移动模式等信息。

本文主要关心车载摄像头带来的隐私问题。摄像头是自动驾驶汽车感知层的核心部件，车载摄像头直接记录了车辆所处物理世界的环境信息，恶意攻击者很容易从图像中的建筑物、街道直接推断或聚合分析车辆位置，从而导致位置隐私泄露。并且若某些机密建筑物或者地点被车载摄像头记录并泄露，这会极大的危害社会治安以及国家安全。因此，研究自动驾驶隐私保护对行业而言有利于确保自动驾驶技术的可持续发展，并促进其在未来的广泛应用，对个人而言有利于帮助建立人们对自动驾驶技术的信任和接受度，对国家而言有利于保护国家领土安全，从源头（车载摄像头记录的道路建筑图像）遏制图像中的位置隐私泄露。本文针对以上问题，设计基于生成对抗网络的道路建筑图像合成模型，通过合成与真实道路建筑图像结构相似但细节不同从而无法推断出位置的图像，以此减轻自动驾驶带来的图像隐私泄露，保护位置隐私。

1.2 国内外研究现状

与本文相关的研究可以总结为以下三个方面：自动驾驶汽车中的隐私保护、图像隐私保护以及生成对抗网络在隐私保护中的应用。

1.2.1 自动驾驶汽车中的隐私保护

自动驾驶汽车在运行过程中会采集和处理大量的数据来识别和理解驾驶场景，这些数据在这个数据驱动的智能时代被赋予大量价值的同时也携带着严重的隐私问题。文献^[7]表明已有大量研究以位置和轨迹隐私为关注点，提出了许多方法来保护车辆和驾驶员的隐私，但将它们应用在自动驾驶汽车时仍不能完全覆盖自动驾驶场景的隐私保护。因为自动驾驶不仅携带 GPS 等位置信息，还有大量摄像头图像和视频、传感器数据，攻击者还可以根据摄像头数据或者传感器数据进行推理攻击。针对自动驾驶中独特的隐私属性，文献^[8]将自动驾驶隐私保护分为两类：个人信息隐私保护、社会信息隐私保护。本文也将从这两个方面综述自动驾驶汽车中的隐私保护研究现状。

1) 个人信息隐私保护

个人隐私主要侧重于保护个人数据（车内摄像头记录的人脸、语音、个人行为等）的隐私。针对个人隐私的保护策略可以分为以下三类： k 匿名、差分隐私和隐私数据合成。

k 匿名已被广泛用于匿名化位置和轨迹数据^[9]。Krontiris 等人^[10]指出，匿名化可能会对检测性能产生不同的影响，这取决于被匿名化的区域对下游应用特征学习的重要性。 k 匿名通常用于匿名化位置和轨迹数据，其要求每个用户的位置在一组 k 个用户中不可区分，实现这一点的一种方法是使用虚拟位置^[11]，但是 k 匿名化已被证明在高维数据集的匿名上是低效的^[12]。

然而，由于随着时间的推移可能会对一个人进行多次观察，因此一般的数据匿名化对于自动驾驶中的数据隐私来说是不够的。比如，模糊面部并不能保护一个人的隐私，因为可以通过持续观察他们的着装和行为来跟踪他们。因此，需要先进的数据编辑技术，例如通过图像分割进行轮廓编辑^[13]。

在训练机器学习模型时，差分隐私^[14]可以被广泛应用于保护不同类型的自动驾驶隐私数据。为了实现差分隐私保护，大量研究通过向轨迹、音频和图像等原始数据添加噪声。吴等人^[15]，设计了一种基于时空相关性的差分隐私位置轨迹保护机制。文献^[16]向音频数据中添加噪声词以保护说话者的声音以及文本内容。

隐私数据合成是指为了在非结构化数据（如图像或视频）中识别物体，同时保留数据效用性，提出的基于 GANs 的方法来替换视频图像中的隐私目标。例如，在文献^[16]中，提出了生成全身和面部去识别方法，以避免识别人脸或其他生物特征标识符，如头发颜色、衣服、发型和个人物品。F. Pittaluga 等人^[17]提出基于 GANs 的图像隐私保护方法，使用生成器作为混淆器，以降低检测隐私像素的成功率。上述基于 GANs 的方法主要关注对象较小且单一，例如人脸和数字。

2) 社会信息隐私保护

自动驾驶汽车收集的数据不仅包含有关个人的隐私敏感信息，车载摄像头中包含大量道路建筑信息、比如路牌号、门牌号、行人、特殊建筑物、重要基础设施等。这些数据泄露的途径有两个方面，一是当第三方服务请求数据时容易导致泄露和隐私攻击，二是在自动驾驶模型训练或推理时容易发生数据泄露。

针对第一类隐私泄露的保护措施已有不少研究，比如，Chatzikokolakis 等人^[19]提出基于查询的自动驾驶服务，通过向被查询的隐私内容添加噪声来保护隐私。

针对第二类隐私泄露的保护措施研究主要集中在联邦学习上，联邦学习（Federated Learning, FL）^[20]是一种在多个设备或数据中心间实现模型共享的新型机器学习技术。这种分布式的机器学习方法可以避免数据隐私泄露的问题，并且可以减少数据传输的带宽和延迟。训练自动驾驶系统的机器学习模型通常使用来自多个源的数据，如果这些数据被恶意攻击，将造成社会信息泄露，严重危害公共安全。已有研究通过联邦学习解决社会信息隐私泄露问题，比如，Zhang^[21]等人搭建了一个用于自动驾驶目标检测的联邦学习系统，在该系统中目标检测模型使用车辆数据在本地训练，更新模型时使用对称加密技术在全局服务器上安全地聚合参数，并生成新的权重。

在社会信息隐私方面，人们希望保密的敏感数据可能与自动驾驶服务所需的数据直接相关联。比如，车载摄像头记录的街道建筑图像对目标检测等任务有用，但它可能会揭示关键基础设施的位置，如何权衡实用性和隐私是需要进一步研究的问题。

1.2.2 图像隐私保护

自动驾驶车载摄像头包含大量图像和视频数据，这些数据直接或间接地携带个人敏感信息、环境敏感信息。比如从车内摄像头数据中可以获取乘客人脸、姿态以及行为信息，从车外摄像头可以获取道路两旁的基础设施、建筑、行人、门牌号等信息。本文主要研究如何保护车外摄像头直接泄露的道路建筑隐私，于此密切相关的一个研究方向为图像隐私保护。

目前有关图像隐私保护的研究从加密算法、图像加噪、图像隐写术、深度学习四个方面展开。

1) 基于传统加密算法的图像隐私保护技术

这类技术一般采用的都是传统的对称性和不对称性的加密算法，以保证图像的隐私性。韩等人^[22]提出了一种基于图像哈希、六维超混沌和动态 DNA 编码相结合的彩色图像加密算法，该算法具有足够大的密钥空间，密钥敏感性强，具有更好的统计和差分特性，且能对篡改图像进行篡改定位分析，具有很好的安全性和强鲁棒性。但是经过加密算法的图像并不适用于下游任务（如图像分割、目标检测），加密后的图像大小和传输的时间都会增加，在实用性上表现差。

2) 基于图像加噪的隐私保护技术

这种技术是在图像中添加噪音,从而扰乱图像中的关键信息,保护图像的隐私。最简单的方法是为图像打马赛克,但这种方法需要结合目标检测,效率低下。另外一种加噪方法源于差分隐私,该方法的思想是向图像中添加噪声(如拉普拉斯噪声),获得满足差分隐私的发布图像。例如,魏^[23]提出基于小波变化的 WIP 方法,使用小波变换对图像进行变换压缩,再对变换后的主要特征添加噪声,获得满足差分隐私的发布图像。张等人^[24]提出结合矩阵分解和差分隐私的人脸图像发布算法,改算法将人脸图像视 2 维矩阵,利用矩阵低秩分解与奇异值分解技术压缩图像,然后利用拉普拉斯机制对挑选的矩阵添加相应的噪声,进而使整个处理过程满足差分隐私。这类方法的优点是简单易实现,但是加噪程度过大会影响图像的质量。

3) 基于图像隐写术的隐私保护技术

这种技术是在图像中嵌入一些隐藏信息,从而保护图像的隐私。李等人^[25]提出一种可以提高隐写灵活性、降低载体图像嵌入修改量的随机非满嵌算法,该算法主要思想是使图像的满嵌容量超过秘密信息长度的阈值,让载体图像达到非满嵌。自生成对抗网络提出后,为深度学习于信息隐藏的结合提供了契机^[26],Vplkonskiy 等人^[27]利用生成对抗网络对于复杂数据的建模能力,建模出图像不同像素之间的复杂依赖关系,从而生成更适合隐写且逼真的载体图像。

4) 基于深度学习的图像隐私保护技术

目前基于深度学习的隐私保护研究在权衡隐私图像的保护力度和实用性上有了很大的进步。葛^[27]提出一种差分隐私辅助分类 GANs 方法,该方法基于新的阈值裁剪方式和扰动策略,在保护训练图像的隐私性同时也可以提升模型的性能,从而达到隐私性和可用性之间的平衡。Yu 等人^[29]设计了一种基于深度多任务学习算法的图像隐私保护工具,该工具利用海量社交图像及其隐私设置有效地学习对象与隐私的相关性,并自动识别一组隐私敏感对象类,从而实现对大量隐私敏感对象类的快速准确检测。这种基于深度学习的算法具有能够提取图像特征,进行更加精准的隐私保护等优势,但对数据与计算资源的要求较高。

1.2.3 生成对抗网络在隐私保护中的应用

生成对抗网络（GANs）错误!未找到引用源。是迄今为止使用最广泛的生成模型，它在生成真实合成样本上的能力非常强大。在图像隐私保护中，GANs 可以用来脱敏图像，以便在保留原图像的内容的同时保护隐私。在这种情况下，生成器被训练为生成脱敏版本的图像，而判别器被训练为区分脱敏版本和原始版本的图像。GANs 在图像隐私保护中的另一种常见应用是生成扰动图像，即与原始图像内容相似但不完全相同的图像。这些扰动图像可以用来替换原始图像，从而提供对原始图像的隐私保护。

例如，Chen 等人^[30]，设计了一种基于 VGANs 的图像表示学习模型，用于保护面部表情识别的隐私，该方案可以保护人类 ID 并且保持表情识别的准确度。Xie 等人^[31]将 GANs 与差分隐私结合，让判别器直接使用原始隐私数据，在每一轮训练中用高斯噪声去扰动判别器的梯度，以此达到隐私保护的效果。谢等人^[32]提出了基于循环对抗网络的图像隐私保护，该方法用图像分割和 CycleGANs 组合，选择出不同的分割系数来辅助生成不同程度的隐私保护图像。

综上所述，已有的研究从数据、算法、模型等多方面角度开展保护自动驾驶隐私保护工作。其中针对摄像头数据的位置隐私保护图的研究较少，而位置隐私对用户人生安全、社会治安以及国家安全都极其重要。现有的基于 GANs 的非结构性数据隐私保护方法主要关注图像中的局部对象，例如人脸和数字。由于这些局部对象具有固定的特征，容易被检测和修改，可以达到隐私保护效果^[33]。但是，对于街道图像，其包含了各种对象，具有更复杂的结构，对某个对象的修改会影响到其他对象的结构，因此不能直接应用上述基于 GANs 的方法来保护车辆摄像头记录的道路建筑图像。综上，本文的主要挑战在于，如何平衡道路建筑图像中的位置隐私保护力度与图像的效用性。本文旨在设计一个生成对抗网络，能够根据真实道路建筑图像输入来合成无法被推断出位置信息的图像，并使合成的图像具有和原图像相似的结构特征，该合成图像会尽最大程度保持数据效用性，不妨碍下游应用的同时避免图像位置隐私的泄露。

1.3 主要内容和工作安排

为了保护自动驾驶车载摄像头记录的道路建筑图像中的敏感信息，缓解自动驾驶系统中存在的图像隐私泄露问题，同时减轻摄像头图像应用在下游任务时受到的精度降低的影响，本文面向自动驾驶图像隐私保护领域，尝试平衡道路建筑图像中的位置隐私保护力度与图像的效用性，旨在设计一个生成对抗网络，能够根据真实道路建筑图像输入来合成无法被推断出位置信息的图像，使合成的图像具有和原图像相似的结构特征，该合成图像会尽最大程度保持数据效用性，不妨碍下游应用的同时避免图像位置隐私的泄露。本文研究的主要内容包括：如何提取图像中敏感部位的结构特征、如何合成与原图像具有相似结构但无法推断出位置信息的图像、如何评估合成图像的隐私保护程度和效用性、如何搭建一个可视化网页部署本文构建的模型。

为了提取图像中敏感部位的结构特征，本文拟建立一个图像语义分割模型，如全卷积网络（Fully convolutional networks, FCN）^[34]。像素级分类有利于我们获取敏感信息在图像中的位置，本文拟使用已标记好敏感部位并配对的道路建筑图像来训练 FCN 模型，使用该模型提取每一张输入图像的像素级标签，获得遮掩住敏感部位的标签图，该标签图将作为后续生成模型的输入。

为了合成与原图像具有相似结构但无法推断出位置信息的图像，本文拟采用图像到图像转换（Image-to-Image）^[36]的思想，即将一张遮掩住敏感部位的道路建筑标签图，转换成一张真实的道路建筑图像。目前学术界针对图像风格迁移的研究有很多，其中影响广泛的是 pix2pix^[42]和 cycleGANs^[36]。pix2pix 和 CycleGANs 都是用于图像风格迁移的生成对抗网络模型。它们的目标都是将一种类型的图像转换成另一种类型的图像。本文拟基于 pix2pix 建立道路建筑图像合成模型，将真实图像与经过 FCN 网络的标签图进行配对，作为合成模型的训练数据，训练模型输出得到与原图像具有相似结构但无法推断出位置信息的合成图像。

为了评估合成图像的隐私保护程度和效用性，本文拟使用 FCN-scores 作为评价指标。最后为了更好的呈现本文研究成果，计划利用 Pytorch 的深度学习框架来实现模型的构建、训练和测试，并利用 Flask 来实现模型的部署。本文工作安排及进度计划如表 1.1 所示。

表 1.1 本文工作安排及进度计划

工作安排	时间安排
查阅资料、研究背景与意义	2023.01.08-2023.01.28
收集资料和阅读文献，学习生成对抗网络原理与搭建方法、全卷积网络搭建方法、pytorch 深度学习框架、flask 等相关知识	2021.02.1-2021.03.10
完成道路建筑图像合成模型的搭建，并实验测试调优模型，记录分析比较实验结果	2021.03.11-2021.04.18
实现模型部署	2021.04.19-2021.05.01
撰写和修改论文	2021.05.01-2021.05.14
完成论文与制作答辩 PPT	2021.05.15-2021.05.30

1.4 论文组织结构

本文共分为 5 章，每一章的大致内如下：

第 1 章是引言，这一章从计算机视觉技术的发展入手，从自动驾驶领域出发，阐述了本论文的研究背景与意义，并对无人驾驶车辆的隐私保护、图像隐私保护以及生成对抗网络在隐私保护中的应用三个方面的国内外研究状况进行了系统的回顾。最后，对论文的主要工作进行了阐述，并对论文的工作进度进行了安排。

第 2 章是介绍了实现道路建筑图像合成模型的相关理论与技术，包括本文构建的模型涉及到的理论与技术包括图像语义分割、生成对抗网络、Pytorch 和 flask 框架。

第 3 章为模型的设计与实现，主要介绍了基于 FCN-8s 的图像语义分割模型以及基于生成对抗网络的道路建筑图像合成模型的结构与实现。

第 4 章为试验和结果分析，这一章对实验环境的设置、数据集的设置、模型训练的参数设置、模型训练的结果进行了详细的描述，并对实验结果进行了详细的说明，并给出了在 Web 页面上的实现方法。

第 5 章总结了本文工作并给出后续研究工作展望。

第2章 相关理论与技术

2.1 图像语义分割

图像语义分割（Image Semantic Segmentation, ISS）是一个涉及广泛的研究领域，本节将先对图像语义分割做简单介绍，再详细介绍本文拟构建的全卷积神经网络的原理。

2.1.1 图像语义分割简介

图像语义分割是计算机视觉、模式识别和人工智能等研究领域的交叉学科，也是数字图像处理和机器视觉的研究重点之一^[37]。语义分割是在像素级尺度对图像分割进行研究，它将属于同一类的像素都被分类为一个类别，例如街道、树木、行人在一个图像中必须被分为不同的类别。实例分割是一种类似于语义分割的方法，其区别在于对图像中的每一个像素点进行归类，而实例分割是对同一类的不同对象进行分类。在实例分割中，为了区别于其它实体，给每一个实体指定了一个唯一的识别号。图 2.1 说明了语义分割与实例分割的区别。

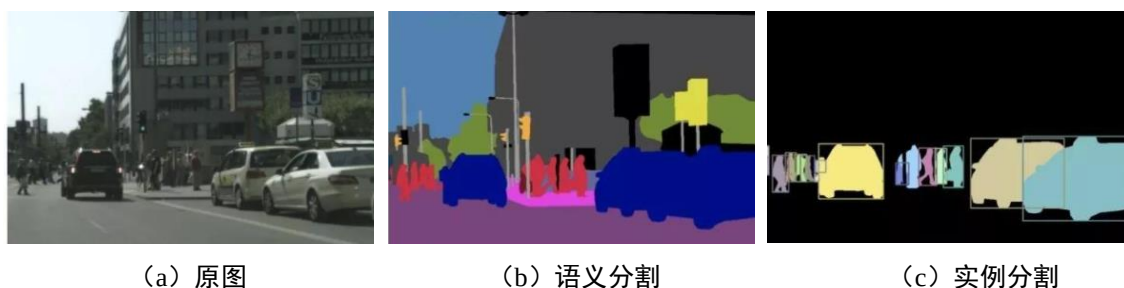
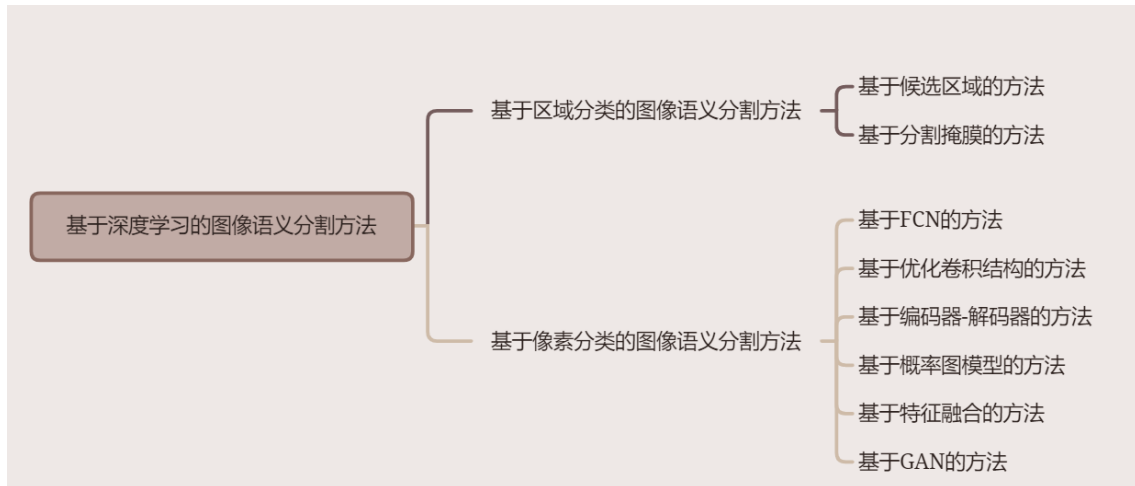


图 2.1 图像语义分割与实例分割示例

图像语义分割应用广泛，例如智能交通系统、自动驾驶、机器人视觉、医学图像分析等领域。图像语义分割的实现方式有很多种，目前基于深度学习的方法已经成为主流，并取得了令人瞩目的成果。综述^[37]总结并整理了相关文献后将基于深度学习的图像语义分割进行了分类，图 2.2 是对基于深度学习的语义分割方法的分类。

图 2.2 基于深度学习的图像语义分割方法分类^[37]

基于区域分类的 ISS 的主要思想是将图像划分为若干个区域。该算法利用卷积神经网络来提取一张图中各个区域的特征。在此基础上，选择适当的分类器将各区域内的图像块划分为若干个类别，每个图像块的像素语义标签一致。该方法需要手动选择分割算法、特征提取算法和分类算法等参数，对人员的经验和知识要求较高，也容易受到噪声和异常值的影响，需要进行更加精细的参数调整和模型优化，同时该方法已经有一段时间的历史，它在一些复杂场景下的表现可能不如一些新的深度学习方法。

基于像素级分类的 ISS 的主要思想是对图像中的每个像素进行分类，以实现像素级别的语义分割。与区域分类相比，采用像素级分类进行语义分割能够充分利用像素级信息，因此具有更高的精度和效率。但像素级分类方法计算量大，需要更多的计算资源和快速的计算方法。近年来，深度学习方法的兴起使得可以直接从数据中学习特征和分类器，大大提高了像素级语义分割的性能，常用的模型包括 FCN、U-net、SegNet 等。

针对自动驾驶环境下相机采集到的影像大小不确定的问题，本文拟利用 FCN 网络对图像进行语义分割，提取图像中的敏感部位。将在下一小节详细介绍 FCN 网络的原理。

2.1.2 全卷积神经网络

在上一节提到，全卷积网络（fully convolutional network, FCN）^[34]是基于像素级分类的图像语义分割模型，它使用卷积神经网络将图像像素转换为像素类，即将不同类物体用不同的像素值表示，将同一类物体用相同像素值表示，从而达到像素级别的语义分割效果。图 2.3 描述了全卷积网络模型最基本的设计。

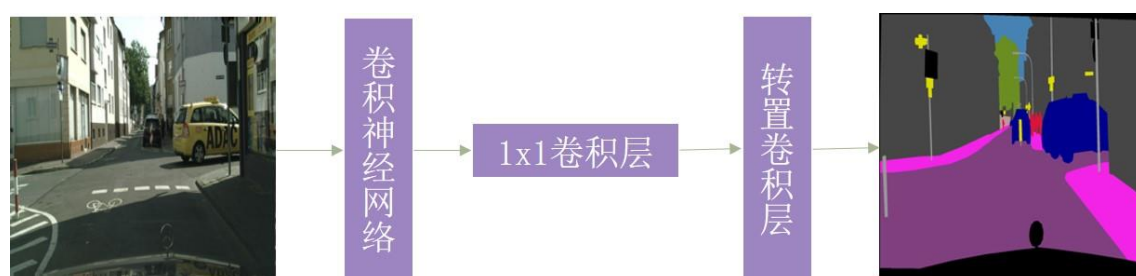


图 2.3 全卷积网络

全卷积网络是在卷积神经网络的基础上使用一些技巧，包括转置卷积、全卷积化、跳级结构。为了更好地理解 FCN 网络结构，本小节将分为 4 个模块介绍全连接网络。

1) 卷积神经网络的结构

CNN 的核心思想是利用卷积层提取输入数据的特征，其架构图如图 2.4 所示。

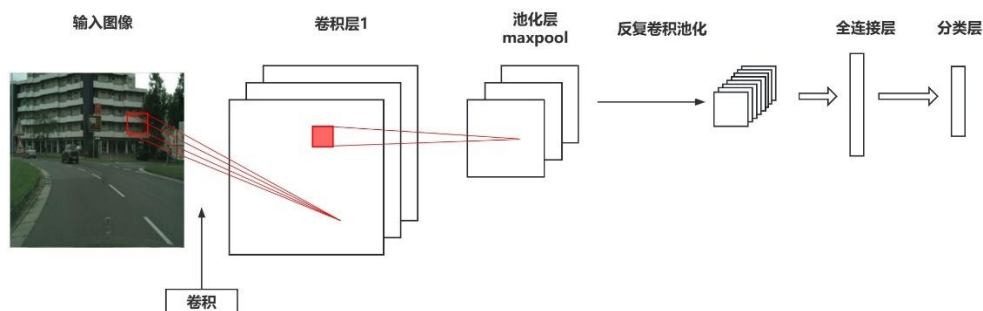


图 2.4 卷积神经网络架构图

由图 2.4 可知，典型的 CNN 包括卷积层、池化层和全连接层。CNN 的输入一般为像高、像宽、像通道数三个维度的三维张量。常规的灰度图是 1 个通道，彩色图是 3 个通道（即红，绿，蓝 3 个通道）。

在卷积神经网络中，卷积层起着对输入数据进行特征提取的作用。卷积层包含一系列具有学习能力的滤波函数，每一个滤波函数通过对输入信号的局部区域进行卷积，从而得到相应的特征值。经过卷积运算后，在输入信号的各个可能的位置上施加此滤波器，从而得到一幅能够反映出输入信号的局部特性的特征图。特征图的值可以通过公式 2.1 计算。

$$G[m,n] = (f * h)[m,n] = \sum_j \sum_k h[j,k] f[m-j, n-k] \quad (2.1)$$

公式 2.1 中 f 为输入数据， h 为滤波器，计算结果的行列索引分别记为 m 和 n 。图 2.5 详细展示了一个输入图像大小为 3×3 ，滤波器大小为 2×2 的卷积操作的过程。

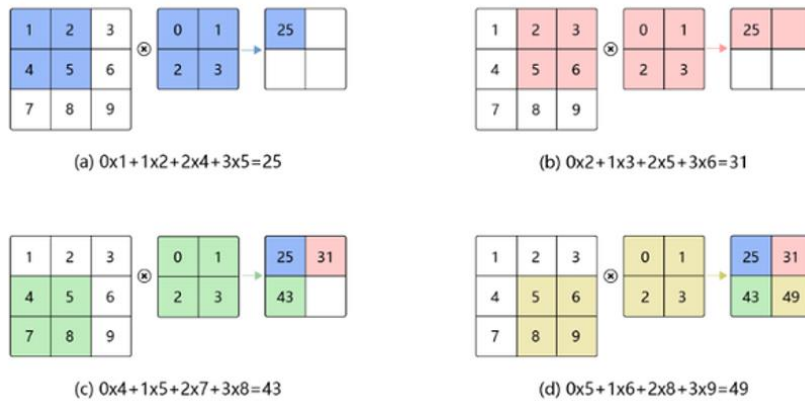


图 2.5 卷积操作图解

卷积层的卷积操作实际上是一个互相关运算，通过互相关运算对每个像素点赋予一个权值，从公式 1 和图 2.5 可知，该操作是线性的。然而卷积神经网络通常作用于图像类数据，此类数据特征复杂，具有明显的非线性，因此通常都会在卷积层之后叠加一个非线性映射层。特征图通过非线性映射层，实际就是在非线性激活函数的作用下增强卷积神经网络的非线性表达能力，从而得到更加复杂的深度特征。常见的激活函数图像如图 2.6 所示。对应的公式和特点如表 2.1 所示。

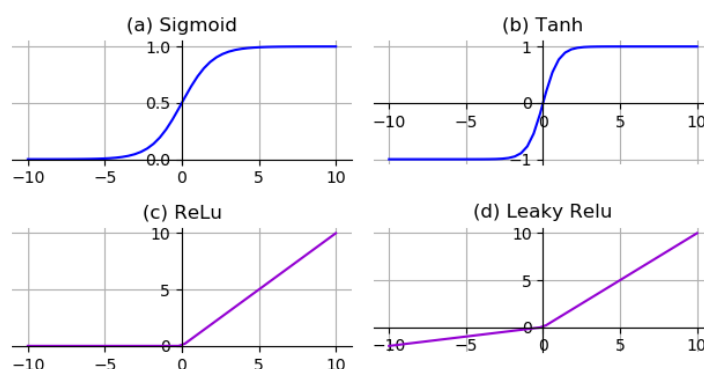


图 2.6 常见激活函数图像 (a)-(d)分别为 sigmoid、Tanh、ReLU、Leaky ReLU

图 2.6 展示了 4 个常用的激活函数图像，图中对应函数的公式及其特点如表 X 所示。

表 2.1 常见激活函数的公式及其特点

名称	公式	特点
Sigmoid	$f(x) = 1/(1 + e^{-x})$	- 输出范围 (0,1)，可以将一个实数映射到(0,1)的区间，适合做二分类，函数图像如图 2.6(a)所示 - 存在梯度消失问题，神经网络使用 Sigmoid 激活函数进行反向传播时，输出接近 0 或 1 的神经元其梯度趋近于 0
Tanh	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	- 输出范围 (-1,1)，与 sigmoid 函数曲线相似，为以 0 为中心的 S 型曲线（如图 2.6(b)所示） - 当输入较大或较小时，输出几乎是平滑的并且梯度较小，也存在梯度消失问题
ReLU	$f(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$	- 输出范围 (0, x)，是一种分段线性函数，函数图像如图 2.6(c)所示 - 当输入为正时，导数为 1，一定程度上改善了梯度消失问题，加速梯度下降的收敛速度 - 当输入为负时，ReLU 完全失效，在反向传播过程中，如果输入负数，则梯度将完全为零，存在神经元死亡问题
Leaky ReLU	$f(x) = \begin{cases} x, & x > 0 \\ \gamma x, & x \leq 0 \end{cases}$	- 输出范围 $(-\infty, +\infty)$，其函数图像如图 2.6(d)所示 - 通过把 x 的非常小的线性分量给予负输入来调整负值的零梯度问题，当 $x < 0$ 时，它得到 γ 的正梯度。该函数一定程度上缓解了神经元死亡问题

表 2.1 给出了包括 Sigmoid 函数、Tanh 函数、ReLU 函数以及 Leaky ReLU 在内的四个激活函数的公式以及特点。之所以要使用激活函数是因为神经网络的每一层实质上都是线性求和的过程，如果没有激活函数，不论网络多么复杂，最后的输出都是输入的线性组合，这是很难解决复杂问题的。通过图 2.6 和表 2.1 提供的信息可以知道常见的激活函数都是非线性的，这样可以让神经网络逼近其他非线性函数，从而解决更多非线性问题。

池化层，它是卷积神经网络的另一个关键部分。池化层的主要功能是降低卷积层输出特征图的维度，从而降低计算成本和模型参数的数量，同时也提高模型的鲁棒性和泛化能力。最大池化和平均池化是两种常见的池化技术。相对于平均池化，即取所有池化区域内特征值的平均值，最大池化是指取每个池化区域内特征值的最大值。池化的过程为一个滑动窗口在输入矩阵上滑动，滑动过程中取这个窗口中数据矩阵上最大值或均值作为输出。

全连接层通常位于卷积层和输出层之间，其主要作用是将卷积层输出的特征图展开成一个向量，即将这些特征图结合在一起来计算全局特征，然后用这些特征来计算每个类别的得分。在分类问题中，为了得到类别的概率分布情况，往往还会在全连接层后跟一个 *soft max* 函数，通过 *soft max* 函数就可以将多分类的输出值转换为范围在 [0,1]，和为 1 的概率分布。函数计算公式如公式 2.2 所示：

$$\text{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad (2.2)$$

其中 z 为向量， z_i 和 z_j 是其中的一个元素， j 是输出类别的个数。

图 2.7 说明了基于卷积神经网络的图像识别过程，输入图像在经历了多个卷积、池化层之后，再通过一个全连接层，会得到一个长度为 1000 的输出向量，该向量再经过一个 *soft max* 函数，得到代表输入图像属于每一类的概率（如图 2.7 最右侧所示）。这个概率信息是 1 维的，即只能标识整个图片的类别，不能标识每个像素点的类别，所以这种全连接方法不适用于像素级图像分割。

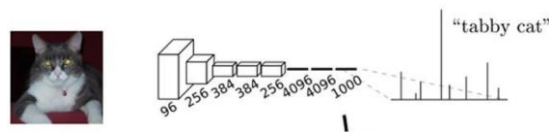


图 2.7 基于 CNN 的图片识别过程^[34]

2) 卷积化

从图 2.7 的例子可以知道，CNN 可以较好地分类整个图像属于哪一类。FCN 能够实现像素数级别的分类，这是由于 FCN 能够把所有的全连接层都转换为卷积层。如图 2.8 所示，与图 2.7 相比，图 2.8 将图 2.7 中的全连接层转化为一个个卷积层，最后得到的不再是 1 维的概率信息，而是二维的热力图。

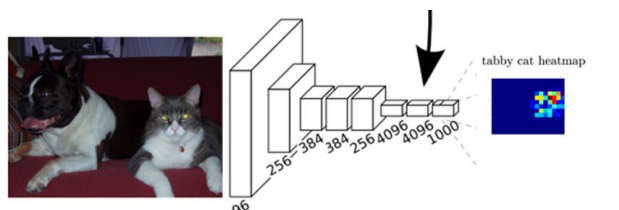


图 2.8 将全连接层替换为卷积层^[34]

3) 转置卷积

卷积神经网络层，通常会减少下采样输入图像的空间维度，如果输入和输出图像的空间维度相同，在以像素级分类的语义分割中将会很方便。例如，输出像素所处的通道维可以保有输入像素在同一位置上的分类结果。为了实现这一点，尤其是在空间维度被卷积神经网络层缩小后，可以使用另一种类型的卷积神经网络层——转置卷积，它可以增加上采样中间层特征图的空间维度。

转置卷积可以看作是正向卷积的逆操作。正向卷积将输入数据通过滤波器的卷积操作生成特征图，而转置卷积则将特征图映射回输入空间，生成更高分辨率的特征图。图 2.9 解释了如何为 2×2 的输入张量计算滤波器为 2×2 的转置卷积。

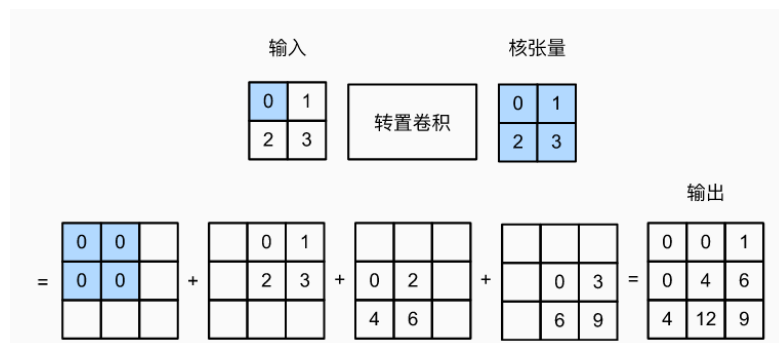


图 2.9 滤波器为 2×2 的转置卷积

在 FCN 网络中，转置卷积的作用在于将小尺寸的高维特征图恢复回去。

4) 跳级结构

在传统的卷积神经网络中，特征从下往上经过一系列卷积层和池化层逐渐减少，最终经过全连接层输出结果。但这样的结构容易出现梯度消失问题，导致网络无法学习到更深层次的特征。

跳级结构通过在低层和高层之间添加直接连接，将低层的特征与高层的特征结合在一起，从而传递更丰富的信息，提高特征的表达能力。在跳级结构的过程中，可以采取多种方法对特征图进行上采样，并对其进行融合。在这些方法中，最常用的方法是用转置卷积来实现上采样，用 `concat` 来实现特征图的合并。

总之，FCN 网络将取代了传统 CNN 中的全连接层替换为卷积层，同时，为保持图像的空间信息，引入转置卷积、跳级结构等功能，实现底层与高层特征的融合，以提升图像分割精度。FCN 网络的结构如图 2.10 所示。

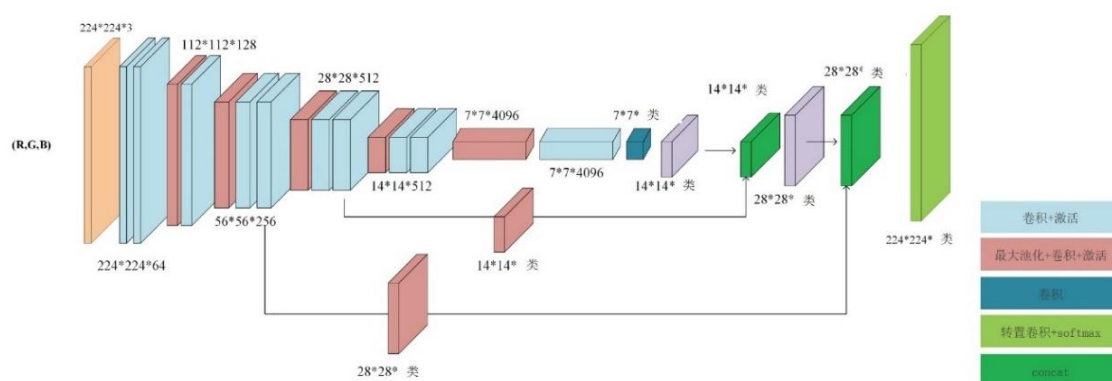


图 2.10 FCN 网络结构

本文将采用 FCN 网络结构, 提取道路建筑图像中的敏感部位。

2.2 生成对抗网络

2.2.1 生成对抗网络原理

生成对抗网络 (Generative Adversarial Nets, GANs)^[2]是由 Goodfellow 等人于 2014 年提出的一种非监督式学习方法，通过两个神经网络相互对抗的方式进行学习。GANs 由一个生成模型 G 和一个判别模型 D 组成， G 将随机噪声作为输入，

输出与真实数据分布尽可能相似的数据， D 是一个二分类模型，用于判断一个输入数据是来自真实数据分布还是来自 G 。 G 的训练目标是使 D 犯错误的概率最大化，而 D 的训练目标则是尽可能完全正确地判断出输入数据来自真实分布而不是 G ，整个训练过程相当于一个二元极小极大博弈（minmax two-player game）^[38]。文献[2]已证明，在任意函数 G 和 D 的空间中，存在一个唯一解，使得 G 模拟到真实数据分布，并且此时 D 恒等于 $\frac{1}{2}$ 。接下来，本文将详细介绍从 GANs 的数学原理，然后在 2.2.2 小节介绍今年来由 GANs 衍生出的几个代表性变体。

首先给出本节会用到的符号定义，如表 2.2 所示：

表 2.2 第 2.2.1 节所用符号及释义

符号	释义
$x \sim p_{data}$	真实数据服从的概率分布
p_g	生成器生成的数据服从的分布
z	随机噪声
y	条件输入
$p_z(z)$	随机噪声 z 服从的概率分布
$G(z, \theta_g)$	噪声空间到数据空间的映射，是由参数 θ_g 定义的可微函数
$D(x, \theta_d)$	神经网络，是由参数 θ_d 定义的可微函数
$D(x)$	表示 x 来自真实数据分布而不是生成器 G 的概率

可以用于 GANs 的目标函数有很多，本文主要介绍 Goodfellow 在文章 Generative Adversarial Nets 中介绍的极小极大目标函数。训练一个好的 GANs 的目标有两个，一个是针对判别器 D ，使其能够正确判断出输入是来自真实分布还是生成器，对此判别器的目标函数为：

$$\max_D E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log(1 - D(G(z)))] \quad (2.3)$$

对于生成器，其目标为生成判别器难以判断真伪的数据，因此，其目标函数为：

$$\min_G E_{z \sim p_z} [\log(1 - D(G(z)))] \quad (2.4)$$

结合式 2.3 和式 2.4 得到 GANs 的总目标函数为：

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (2.5)$$

其中， $\log D(x)$ 是 $[1 \ 0]^T$ 和 $[D(x) \ 1-D(x)]^T$ 之间的交叉熵。解式 2.5 需要先固定 G ，让 D 达到最优，由文献[2]可知，最优的 $D_G^*(x)$ 为：

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \quad (2.6)$$

将式 2.6 代入式 2.5，将 GANs 目标函数重建为：

$$\begin{aligned} C(G) &= \max_D V(D, G) \\ &= E_{x \sim p_{data}} [\log D_G^*(x)] + E_{z \sim p_z} [\log(1 - D_G^*(G(z)))] \\ &= E_{x \sim p_{data}} [\log D_G^*(x)] + E_{x \sim p_g} [\log(1 - D_G^*(x))] \\ &= E_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{\frac{1}{2}(p_{data}(x) + p_g(x))} \right] + E_{x \sim p_g} \left[\log \frac{p_g(x)}{\frac{1}{2}(p_{data}(x) + p_g(x))} \right] - 2 \log 2 \end{aligned} \quad (2.7)$$

由 KL 散度和 JS 散度的定义：

$$KL(p \parallel q) = \int p(x) \log \frac{p(x)}{q(x)} dx \quad (2.8)$$

$$JS(p \parallel q) = \frac{1}{2} KL(p \parallel \frac{p+q}{2}) + \frac{1}{2} KL(q \parallel \frac{p+q}{2}) \quad (2.9)$$

式 2.7 可以写为：

$$\begin{aligned} C(G) &= KL(p_{data} \parallel \frac{p_{data} + p_g}{2}) + KL(p_g \parallel \frac{p_{data} + p_g}{2}) - 2 \log 2 \\ &= 2JS(p_{data} \parallel p_g) - 2 \log 2 \end{aligned} \quad (2.10)$$

由于两个分布的 JS 散度总是非负的，当且仅当它们相等时，即 $p_{data} = p_g$ 时 JS 散度值等于 0，此时 $C(G)$ 的全局最小值 $C^* = -\log 4$ 。

总之，从理论上讲，理想情况下，训练 GAN 可以实现纳什均衡，即生成器和判别器处于均衡状态。若把纳什均衡看作是生成器的目标，在这个阶段，确定生成器产生的图片是否有效已经不可行了，因为鉴别器有 0.5 的几率认为它们是真实的数据。但是实际上，GANs 的训练很不稳定，容易发生模式崩塌现象，即生成器一直只生成同一张图片。针对原始 GANs 的种种问题，目前已有大量研究发明了 GANs 的各种变体，将在下一个小节介绍最具代表性地几个变体。

最后，来自 GANLab (<https://poloclub.github.io/ganlab/>) 可视化网站的图片详细且生动地表现出了 GANs 的训练过程，如图 2.11 所示：

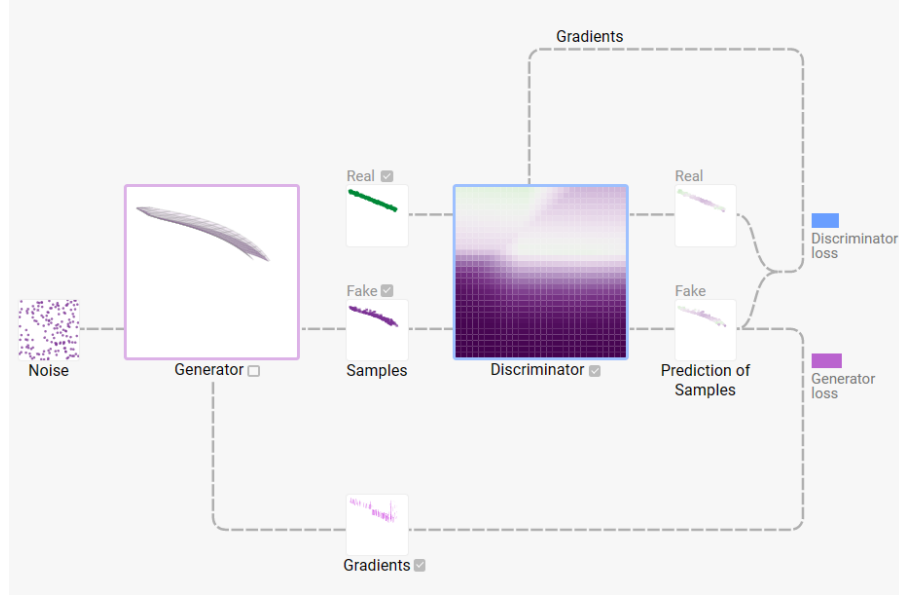


图 2.11 GANs 的训练过程

2.2.2 生成对抗网络的变体

1) 条件 GANs (conditional GANs, cGANs) [39]

顾名思义，条件 GANs 是拥有条件样本的 GANs 的变体。cGANs 可以控制 GANs 生成的图片，而不是单纯的随机生成图片，它使用额外的信息 y （比如标签图）来执行生成任务，其目标函数为：

$$\min_G \max_D f(D, G) = \mathbb{E}_x[\log(D(x|y))] + \mathbb{E}_z[\log(1 - D(G(z|y)))] \quad (2.11)$$

cGANs 的结构如图 2.12 所示：

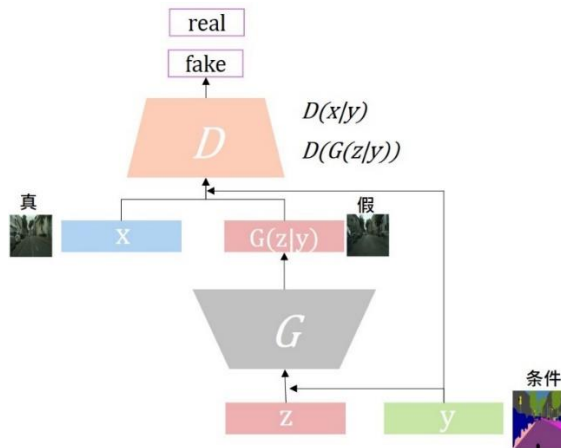


图 2.12 cGANs 的结构

2) Wasserstein GANs (WGANs) [40]

WGANs 是在原始 GANs 上的改进，为了更好地训练 GANs，WGANs 将原来使用的 JS 散度替换成了 Wasserstein 距离，通过添加梯度惩罚项，实现了 WGANs 算法，并且证实了 WGANs 训练稳定。WGANs 的目标函数如下：

$$\min_G \max_D \mathbb{E}_x[f_w(x)] - \mathbb{E}_z[f_w(G(z))] \quad (2.12)$$

除了优化目标函数外，在训练 WGAN 方面也有改进，如去掉判别器的最后一个 sigmoid 层，在每次更新后将判别器参数的绝对值截断为不超过一个固定常数，并使用优化算法 RMSProp 而不是 Adam。

3) 深度卷积 GANs (DCGANs) [41]

DCGANs 的生成器和判别器基于深度卷积神经网络。由于传统 GANs 仅使用多层感知机 (MLP)，但随着卷积神经网络在图像领域的快速发展，它们在性能上超越了 MLP，在图像生成任务上表现更好，因此 DCGANs 的作者团队在生成器和判别器中分别用了卷积神经网络的架构，同时做出相应改进，具体改进措施如表 2.3 所示。

表 2.3 DCGANs 的改进措施

改进	内容
1	使用指定步长的卷积层代替池化层，这样在下采样过程时不再是固定的抛弃某些位置的像素值，而是让网络自己去学习下采样的方式
2	移除全连接层，而是将生成器输入的噪声变成四维的张量，来实现用卷积层代替全连接层
3	在生成器和判别器中都使用批量标准化(BatchNormalization, BN)
4	生成器除去输出层采用 Tanh 激活函数外，全部使用 ReLU 作为激活函数
5	判别器所有层都使用 LeakyReLU 作为激活函数

虽然关于 DCGANs 的改进没有严格的数学证明，但其实验验证这些改进会带来更好的结果。DCGANs 通过以上的改进得到的生成器和判别器的结构如图 2.13 和图 2.14 所示：

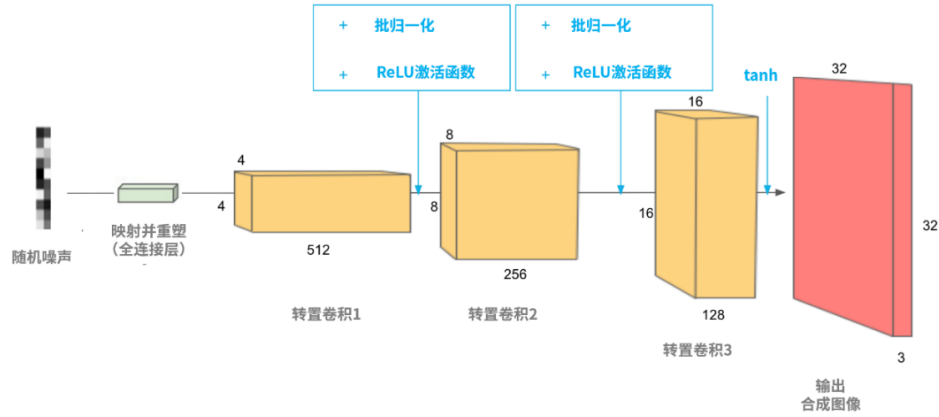


图 2.13 DCGAN 生成器的结构

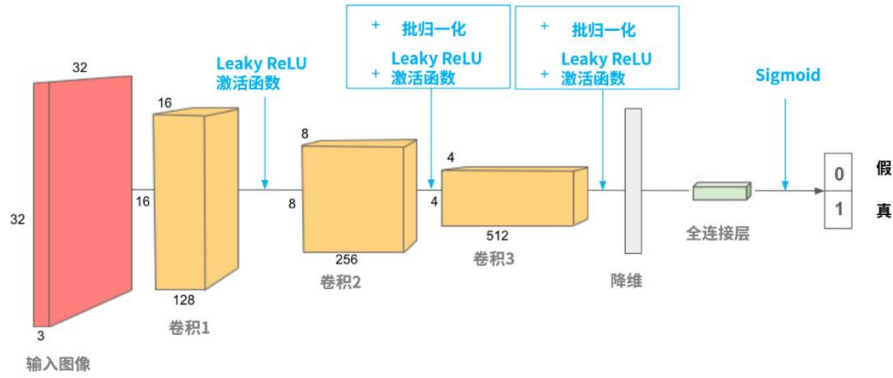


图 2.14 DCGAN 判别器的结构

4) Pix2pix^[42]

Pix2pix 文章标题为“Image-to-Image Translation with Conditional Adversarial Networks”，如标题所述，pix2pix 是一个可以实现图像到图像转换的条件生成对抗网络。Pix2pix 由一个具有 U-net 结构的生成器和 PathGANs 判别器组成。特别地，pix2pix 不但能从输入图像到输出图像之间的映射关系，而且能通过学习损失函数来构建该映射关系。此外 pix2pix 在 cGANs 的损失函数基础上添加了 $L1$ 损失，用来鼓励生成的图像尽可能与真实图像保持相似，这也有利于更快的收敛和更稳定的训练，pix2pix 的损失函数为：

$$\arg \min_G \max_D \mathcal{L}_{cGAN}(G, D) + \lambda \mathcal{L}_{L1}(G) \quad (2.13)$$

其中 $\mathcal{L}_{cGAN}$ 和 $\mathcal{L}_{L1}$ 分别为条件 GANs 的损失函数（见式 2.11）和 L_1 损失函数（见式 2.14）。

$$\mathcal{L}_{L1}(G) = \mathbb{E}_{x,y,z} [\|y - G(x, z)\|_1] \quad (2.14)$$

Pix2pix 可用于手稿图到彩色图、白天到黑夜、标签图到真实图等转换，这些转换是像素级别的转换，因此要求训练数据是配对好的数据。本文受 pix2pix 的启发，拟基于此实现从掩盖敏感信息的标签图到真实道路建筑图的转换。

2.3 Pytorch 深度学习框架

Pytorch 是 torch 的 Python 版本，由 Facebook 人工智能研究院（FAIR）开发和维护的开源深度学习框架^[43]。

Torch 是一个开源的科学计算框架，主要用于张量（tensor）计算，支持 GPU 加速，具有高效、灵活、易用的特点，被广泛应用于深度学习、计算机视觉、自然语言处理等领域。Pytorch 是基于 Python 语言，可以通过 Python 编写和调用 Torch 的 API。Pytorch 中的 torch 具有定义多维张量结构及基于张量的多种数学操作。本文将使用 Pytorch 框架完成模型的搭建、训练与实验，接下来将介绍一些在实验过程会用到的工具包。

1) Torch.Tensor

主要功能：定义 tensor 类型，包含 7 种 CPU tensor 类型和 8 种 GPU tensor 类型（整型、浮点型）（8/16/32/64 位），实现基于 Tensor 的各种数学操作、各种类型的转换。比如相加、相乘、求绝对值等

2) torch.Storage

用于管理张量数据存储的底层类。一般情况下数据以 CPU 类型存储，如果要用 cuda 加速的话必须把模型和数据同时以 GPU 类型存储。

3) torch.nn

该模块提供了大量用于神经网络构建的类和函数。使用该模块可以方便地定义和组合各种神经网络层和模块。在 torch.nn 中，神经网络层和模块可以通过继承 nn.Module 类来实现的。通过继承 nn.Module 类，我们可以自定义各种神经网络层和模块，并将它们组合成更复杂的网络结构。

4) torch.autograd

该模块用于自动求导，它可以自动计算张量的梯度，从而帮助使用反向传播算法来更新模型参数以最小化损失函数。使用 `torch.autograd` 计算梯度的基本步骤一般为：首先定义需要梯度的张量，在 PyTorch 中，可以通过设置 `requires_grad=True` 来指定一个张量需要梯度。例如，可以使用 `torch.Tensor()` 函数创建一个张量，并将其设置为需要梯度的状态：`x = torch.Tensor([1, 2, 3], requires_grad=True)`。然后定义计算图，在 PyTorch 中，可以通过在张量上执行一系列操作来定义计算图。例如，使用 `x.sum()` 函数计算张量的和：`y = x.sum()`。最后计算梯度，在计算完成后，通过调用 `y.backward()` 函数自动计算梯度。此时，PyTorch 会自动计算 `y` 相对于 `x` 的梯度，并将结果存储在 `x.grad` 属性中。

5) `torch.optim`

该模块用于实现各种优化算法，以帮助更快地训练和调整深度学习模型，其中常用的优化算法有 SGD、Adam 和 Adgrad 等。

6) `torch.cuda`

该模块提供了一些工具和函数，可以帮助利用 GPU 进行深度学习计算，例如，`torch.cuda.is_available()`：该函数可以检查当前系统是否支持 CUDA 和 GPU 计算。

7) `torch.utils`

提供一些常用的实用工具和功能，其中最常用的是 `torch.utils.data.Dataset` 和 `torch.utils.data.DataLoader`，`Dataset` 函数用来创建、保存数据集，`DataLoader` 函数中包含对数据集的一些操作。用自己的数据集训练模型时需要继承这两个类并覆写相关方法。

8) `torchvision`

包含了目前流行的数据集，模型结构和常用的图片转换工具。具体有如下四部分：`torchvision.models`：提供深度学习中各种经典的网络结构、预训练好的模型，如：Alex-Net、VGG、ResNet、Inception 等；`torchvision.datasets`：提供常用的数据集，设计上继承 `torch.utils.data.Dataset`，主要包括：MNIST、CIFAR10/100、ImageNet、COCO 等；`torchvision.transforms`：提供常用的数据预处理操作，主要包括对 Tensor 及 PIL Image 对象的操作；`torchvision.utils`：工具类，如保存张量作为图像到磁盘，给一个小批量创建一个图像网格。

最后由表 2.4 给出使用 Pytorch 的最佳实践。

表 2.4 pytorch 最佳实践

Pytorch 的最佳实践		
1	使用 GPU 进行模型训练	Eg: device=torch.device('cuda' if torch.cuda.is_available() else 'cpu') model = Model().to(device)
2	使用 DataLoader 加载数据	Eg: train_loader=DataLoader(train_dataset, batch_size=32, shuffle=True, num_workers=4, pin_memory=True)
3	使用预训练模型进行迁移学习	Eg: import torchvision.models as models model = models.resnet18(pretrained=True)
4	使用学习率调度器进行优化	Eg: optimizer=torch.optim.SGD(model.parameters(), lr=0.01) scheduler=torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=10)
5	使用 TensorBoard 可视化训练过程	Eg: from torch.utils.tensorboard import SummaryWriter

2.4 Flask 轻量级 PythonWeb 框架

2.4.1 Flask 简介

Flask 是一个轻量级的 Web 应用程序框架，基于 Python 语言开发，其 WSGI 工具箱采用 Werkzeug，模板引擎则使用 Jinja2^[44]。其中两个环境依赖是 Werkzeug 和 jinja2，这意味着它不需要依赖外部库 因此称其为轻量级框架。

Flask 的架构由三个主要组件组成：应用程序、路由和视图函数

1) 应用程序

Flask 应用程序是一个基于 WSGI（Web Server Gateway interface）的可调用对象，用于处理 HTTP 请求并返回 HTTP 响应。WSGI 是一种 Web 应用程序域 Web 服务器之间通信的标准接口，它定义了一个以 Python 为基础的 API，使得 Python Web 应用程序可以与不同的 Web 服务器之间进行互操作，从而达到了 Web 应用程序与 Web 服务器的解耦。WSGI 工作流程如图 2.15 所示：

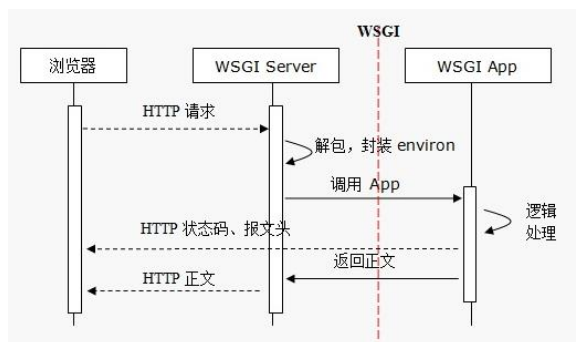


图 2.15 WSGI 工作流程

如图 2.15，首先用户发起 HTTP 请求，请求被服务器接收，服务器解包并将请求转发给 WSGI 应用程序。WSGI 应用程序接收请求，并从 environ 中获取请求相关的信息，并对其进行逻辑处理。然后应用程序将响应内容发送给服务器，服务器再将响应内容发送给浏览器（客户端），从而完成 HTTP 响应。

2) 路由

Flask 中的路由用于将 HTTP 请求映射到视图函数，并使用 `werkzeug` 来做路由分发，路由分发工作流程如图 2.16 所示：

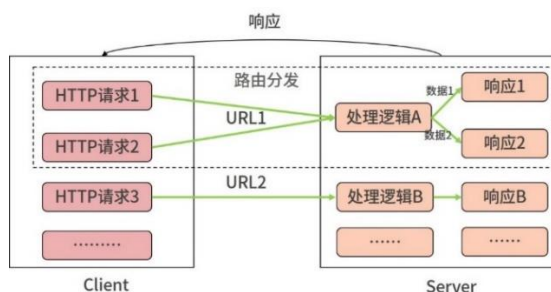


图 2.16 路由分发工作流程

Flask 路由分发是将 HTTP 请求映射到特定的处理逻辑的过程，在 flask 中，可以使用路由映射装饰器 `@app.route()` 来将函数注册为处理逻辑，并通过该装饰器将 URL 和视图函数进行绑定。

3) 视图函数

视图函数是对 Web 应用程序的请求进行处理，并将响应（一般为 HTML 文件）返回给网页。Flask 使用 Jinja2 完成 HTML 模板的渲染，Jinja2 的工作流程如图 2.17 所示：

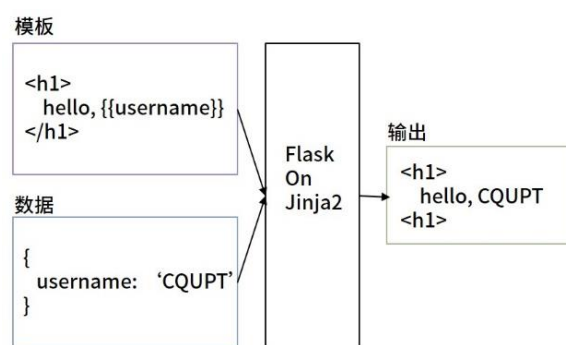


图 2.17 Jinja2 工作流程

2.4.2 使用 Flask 部署 pytorch 模型

从前文介绍可知，Flask 是一种轻量级的 Web 框架，易于学习和使用，同时也有着广泛的社区支持。使用 Flask 部署 PyTorch 模型需要进行以下几个步骤：

1) 创建 Flask 应用程序：通过‘Flask(__name__)’创建一个 Flask 应用程序实例，并通过 ‘@app.route’装饰器定义路由和视图函数。

2) 加载 PyTorch 模型：使用 PyTorch 的 torch.load 方法加载预训练的模型。

3) 定义预处理和后处理函数：预处理函数用于将输入图像转换为模型期望的张量格式，后处理函数用于将模型输出的张量转换为人类可读的结果格式。

4) 编写 Flask 视图函数：在 Flask 视图函数中，先读取上传的图像，然后使用预处理函数将其转换为张量，再将张量传入加载的 PyTorch 模型进行预测，最后使用后处理函数将预测结果返回给客户端。

本文将按照上述步骤部署训练好的道路建筑图像生成模型，并编写界面美观的 HTML 页面，展示模型的结果。

2.5 本章小结

本章主要介绍了与道路建筑合成模型有关的理论和技术。在这一章，首先对图像分割技术进行了简单介绍，又重点介绍了 FCN 模型，这是本文研究拟采用的图像分割模型。接着对生成对抗网络进行了介绍，这是本文构建道路建筑合成模型的理论基础。最后介绍了 Pytorch 框架和 Flask 架构，这是本文将使用到的重要工具，其中 pytorch 用于搭建模型，Flask 用于部署模型并展示模型结果。

第3章 模型设计与实现

3.1 模型架构

本文要解决的问题聚焦在自动驾驶场景，自动驾驶车载摄像头记录的道路建筑图像含有大量的个人隐私和公共位置信息，当图像投入到下游任务中时很容易受到攻击，泄露个人隐私和位置隐私。本文构建模型的主要目的是保护车载摄像头记录的道路建筑图像中的敏感部位不被机器识别出，也不被肉眼推测出，同时保证图像在下游任务的适用性。根据 1.2 节的国内外研究现状可知，已有文献针对保护图像特定部位隐私并保持图像实用性展开研究，受文献^[42]启发，该文献提出一种基于 GAN 的图像转换模型，可以由线稿图生成色块图，由语义分割标签图生成真实原图。由此，可以基于文献^[42]模型构建一个由掩盖图像敏感部位的标签图生成与真实图像相似但敏感部位细节不同的合成图像。如何快速有效地提取图像敏感部位，文献^[32]给出了可行的思路：使用图像语义分割方法分割图像中指定的敏感部位。同时语义分割是像素级别的分割方法，其结果自然地替换隐私部位每个像素的值，达到掩盖敏感部位细节的效果。

结合文献^[32]和文献^[42]的思想，并改进方法模型，本文设计了基于生成对抗网络的道路建筑图像合成模型，模型包含两个子模块：一是用于提取并掩盖图像中隐私部位的语义分割模块，可以在训练语义分割模型时设置不同的语义标签，得到能够分割出不同敏感部位的标签图；二是道路建筑图像合成模块，该模块基于生成对抗网络构建。本文设计的模型架构如图 3.1 所示。

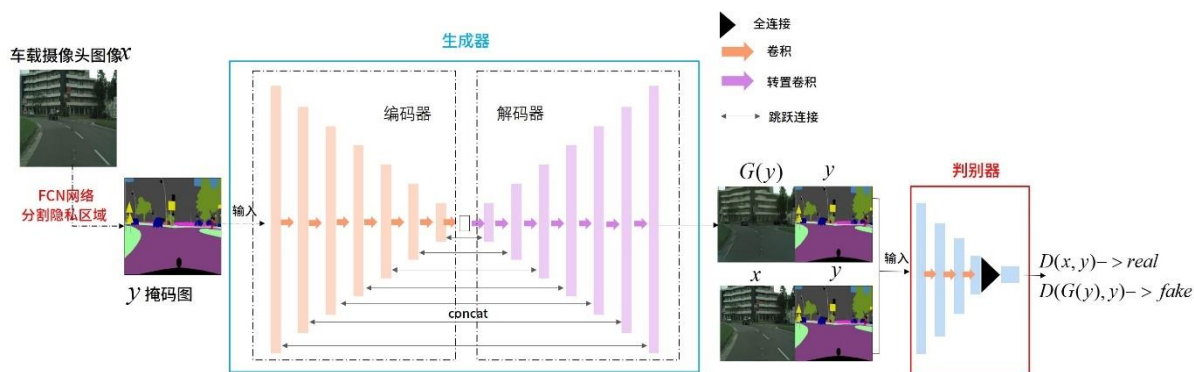


图 3.1 模型架构图

当真实道路建筑图像通过该框架时(如图 3.1 中的 x) 首先会输入进基于 FCN8s 的语义分割模型, 得到掩盖敏感信息的标签图, 然后标签图和真实图像会同时输入图像合成模型, 得到一张结构与原图相似但是敏感部位细节不同的合成图像。该合成图像保留了真实图像的结构, 即车仍是车、人仍是人, 房屋建筑和街道设施等物体的位置结构没有改变, 只是敏感部位的表面细节发生变化。这样的合成图像仍然能够在下游任务(比如目标检测)中使用, 同时也保护了个人隐私和位置隐私。

3.2 基于 FCN8s 的语义分割模型

3.2.1 模型的结构

本文基于 FCN8s 网络建立一个图像语义分割模型, 用于分割真实道路建筑图像中的敏感部位。该模型可以分类出图像中的每一个像素, 只要实际场景的图像与训练数据集属于同一类, 则可以将真实图像通过这个语义分割模型, 从而获得真实图像的每个像素的类标签, 实现对真实图像隐私部位的分割。

本文构建的 FCN8s 模型来源于文献[34], 文献对比了包括 ResNet, AlexNet, VGG 在内的 3 种性能较好的卷积神经网络进行实验, 其中表现效果最好的是 VGG16。文献[34]同样也比较了 FCN32s、FCN16s 和 FCN8s 的性能, 实验结果证明分割效果最好的是 FCN8s。关于 FCN 各个模型的详细介绍可参见本文第 2 章, 下面给出 FCN8s 网络结构, 如表 3.1 所示:

表 3.1 FCN8s 网络结构

预训练模型 (VGG16)	上采样部分
Conv1:3*3*64 conv x2 pool1:maxpool	3*3*512 deconv BatchNorm
Conv2:3*3*128 conv x2 pool2:maxpool	3*3*256 deconv BatchNorm
Conv3:3*3*256 conv x3 pool3:maxpool	3*3*128 deconv BatchNorm
Conv4:3*3*512 conv x3 pool4:maxpool	3*3*64 deconv BatchNorm
Conv5:3*3*512 conv x3 pool5:maxpool	3*3*32 deconv BatchNorm
Conv6:1*1*1024 conv7:1*1*1024	1*1*n_class

FCN8s 使用 VGG16 作为预训练模型，且只使用 VGG16 网络的前五个块，去掉了后面的全连接层，加入上采样进行特征融合，上采样通过跳级连接和转置卷积实现，图 3.2 是 FCN8s 训练结构图。

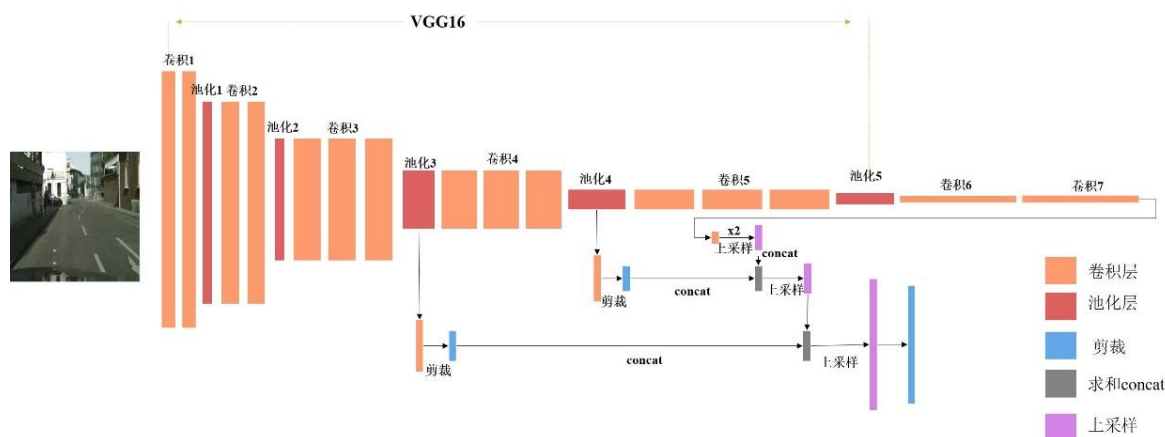


图 3.2 FCN8s 网络运算流程图

如图 3.2 所示，FCN8s 网络就是先将最后的卷积结果 conv7 通过转置卷积把图像大小扩大 2 倍，然后和 pool4 的结果进行 concat（pool4 的结果会先进行卷积、剪裁）。然后再利用转置卷积将 concat 的结果扩大 2 倍，再和 pool3 的结果进行 concat（pool3 的结果会先进行卷积、剪裁），最后通过转置卷积扩大 8 倍，而获得与输入图像大小相同的结果。

3.2.2 模型的实现

本文构建的 FCN8s 模型是在 Pytorch 深度学习框架上实现的，在 pytorch 中使用 torch.nn 模块实现模型构建，通过继承 nn.Module 类，可以自定义各种神经网络层和模块，并将它们组合成更复杂的网络结构。

首先需要构建一个名为 FCN8s 的类，该类继承 nn.Module，并重写 __init__ 函数和 forward 函数。__init__ 函数主要用来做参数初始化，在这里初始化模型结构，向其中添加需要的卷积、池化、归一化、转置卷积等操作。forward 函数表示前向传播，即构建网络的先后运算步骤。

本文直接使用 torchvision.models 工具包中自带的 VGG 网络，在 __init__ 函数中，传入预训练模型，即 VGG16，在此基础上添加激活函数层、转置卷积层、批

归一化层，并对每层设置合适的输入输出通道数、kernel_size、stride、padding 等参数。具体每层的设置见表 3.2，同时图 3.3 给出了在 pytorch 中实现 FCN8s 结构的代码截图：

表 3.2 FCN8s 网络层设置

层数	操作	输入输出通道数	Kernel_size 卷积核	Stride 步长	Padding 补丁	Dilation 填充	Output_padding 输出补丁
deconv1	ConvTranspose2d	512*512	3	2	1	1	1
bn1	BatchNorm2d	512	\	\	\	\	\
Deconv2	ConvTranspose2d	512*256	3	2	1	1	1
Bn2	BatchNorm2d	256	\	\	\	\	\
Deconv3	ConvTranspose2d	256*128	3	2	1	1	1
Bn3	BatchNorm2d	128	\	\	\	\	\
Deconv4	ConvTranspose2d	128*64	3	2	1	1	1
Bn4	BatchNorm2d	64	\	\	\	\	\
Deconv5	ConvTranspose2d	64*32	3	2	1	1	1
Bn5	BatchNorm2d	32	\	\	\	\	\
classifier	Conv2d	32*n_class	1	\	\	\	\

```

def __init__(self, pretrained_net, n_class):
    super().__init__()
    self.n_class = n_class
    self.pretrained_net = pretrained_net
    self.relu = nn.ReLU(inplace=True)
    self.deconv1 = nn.ConvTranspose2d(512, 512, kernel_size=3, stride=2, padding=1, dilation=1, output_padding=1)
    self.bn1 = nn.BatchNorm2d(512)
    self.deconv2 = nn.ConvTranspose2d(512, 256, kernel_size=3, stride=2, padding=1, dilation=1, output_padding=1)
    self.bn2 = nn.BatchNorm2d(256)
    self.deconv3 = nn.ConvTranspose2d(256, 128, kernel_size=3, stride=2, padding=1, dilation=1, output_padding=1)
    self.bn3 = nn.BatchNorm2d(128)
    self.deconv4 = nn.ConvTranspose2d(128, 64, kernel_size=3, stride=2, padding=1, dilation=1, output_padding=1)
    self.bn4 = nn.BatchNorm2d(64)
    self.deconv5 = nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2, padding=1, dilation=1, output_padding=1)
    self.bn5 = nn.BatchNorm2d(32)
    self.classifier = nn.Conv2d(32, n_class, kernel_size=1)

def forward(self, x):
    output = self.pretrained_net(x)
    x5 = output['x5'] # size=(N, 512, x.H/32, x.W/32)
    x4 = output['x4'] # size=(N, 512, x.H/16, x.W/16)
    x3 = output['x3'] # size=(N, 256, x.H/8, x.W/8)
    score = self.relu(self.deconv1(x5)) # size=(N, 512, x.H/16, x.W/16)
    score = self.bn1(score + x4) # element-wise add, size=(N, 512, x.H/16, x.W/16)
    score = self.relu(self.deconv2(score)) # size=(N, 256, x.H/8, x.W/8)
    score = self.bn2(score + x3) # element-wise add, size=(N, 256, x.H/8, x.W/8)
    score = self.bn3(self.relu(self.deconv3(score))) # size=(N, 128, x.H/4, x.W/4)
    score = self.bn4(self.relu(self.deconv4(score))) # size=(N, 64, x.H/2, x.W/2)
    score = self.bn5(self.relu(self.deconv5(score))) # size=(N, 32, x.H, x.W)
    score = self.classifier(score) # size=(N, n_class, x.H/1, x.W/1)

```

图 3.3 pytorch 中实现 FCN8s 结构的代码截图

FCN8s 网络的训练将在第 4 章详细讲述。至此，基于 FCN8s 的语义分割模型搭建完成，通过该模型可以实现提取道路建筑图像中的敏感部位，在训练 FCN8s 模型时，训练数据集为配对的标签图-真实图，这里标签图可以根据选定的敏感信息进行调整，标签图中应该只标注出包含敏感信息的部分。从真实道路建筑图像中提取出敏感部位后，将真实道路建筑图像与掩盖敏感信息的标签图作为一对输入进合成模型就可得到能够保护位置隐私且不妨碍下游应用使用的合成图像。接下来，介绍本文使用的合成模型的构建过程。

3.3 基于生成对抗网络的道路建筑合成模型

本节将从生成器、判别器、损失函数、模型的结构和模型的实现五个方面展开，为了方便后文的阅读，给出本节会用到的符号及其释义。

表 3.3 符号及释义

符号	释义
G	生成器
D	判别器
I	来自现实车载摄像头记录的真实图像集
I'	来自生成器生成的合成图像集
x'	$x \in I$
x	$x' \in I'$
y	经过语义分割的标签数据

3.3.1 生成器

首先回顾本文需要解决的问题：根据车载摄像头记录的真实道路建筑图像，合成一张肉眼无法识别出位置的隐私保护图像，同时该合成图像不妨碍下游应用的使用。针对该问题，一个很直观的想法就是将图像中的隐私部位打码，基于此想法，本文决定使用图像到图像转换的思想进行图像合成，即将真实道路建筑图像转换成为一张敏感部位轮廓与真实图像一致但是细节信息不同的合成图像。为了提取真实道路建筑图像的敏感部位，本文已构建语义分割模型（见 3.1 节），现在，需要构建生成器，以满足以下要求：(1).为了保持真实道路建筑图像与合成图像在结

构上相似，且合成图像不妨碍下游任务，需要生成器的输出尽可能接近原始输入；
(2).要求的敏感部位的信息不能被识别出，也不能通过肉眼推测出。

为此本文使用“U-net”结构构建生成器网络，因为“U-net”在重建图像核心结构上有很好的表现。生成器包含一个编码器用来提取图像特征，和一个解码器用来从中间特征图恢复出图像。编码器就是进行下采样操作的一组组卷积层，解码器就是网络中进行上采样的一组组转置卷积层。输入会经过编码器，进行特征提取，随着卷积层数的增多，输出的特征图分辨率会不断降低，当下采样进行到一定程度时（a bottlrneck layer），就开始反转，进行上采样操作。这样的结构需要所有信息流过网络的所有层，但在许多图像转换问题中，深层信息在输入输出之间共享是很难的，因为经过多次下采样后特征图中保留的输入信息很少，之后上采样难以从深层特征图中恢复出输入图像的信息。因此，为了能够直接共享输入输出信息，在编码器和解码器直接添加跳跃连接，如同“U-net”网络结构一样，在网络第 i 层和第 $n-i$ 层之间添加跳跃连接，这里 n 是网络总层数。每个跳跃连接就是将第 i 层和第 $n-i$ 层的特征图在通道维度上进行相加连接。

在图像合成过程中，生成器 G 需要达到三个目标：1.模拟输入图像集的真实分布，使生成图像与真实图像不可区分（指不可被判别器区分）；2.生成的图像尽可能真实（指欺骗过肉眼）；3.为了保护图像中的隐私，生成图像中的隐私部位的细节信息应该与输入图像中对应部位的信息不同，并且不能被机器识别出，也不能被肉眼推测出。为了达到以上三个目标，需要设计对应的损失函数，具体细节详见3.2.3小节。综上，生成器的结构如图3.4所示：

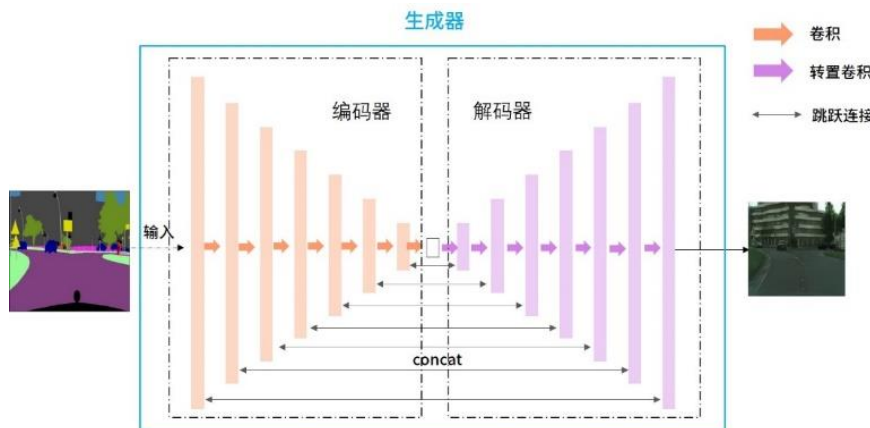


图 3.4 生成器结构图

3.3.2 判别器

生成对抗网络中另一个重要组成部分是判别器。本文构建的判别器需要执行一个分类任务，即区分输入的图像是来自现实车载摄像头记录的数据集 I 还是来自生成器的生成的数据集 I' 。原始 GANs 中的判别器为多层感知机，考虑到本文的应用场景，判别器需要区分复杂的高分辨率图像，多层感知机的分类能力显然不够。因此，本文构建含有四层卷积和一层全连接的卷积神经网络作为判别器，并设置合适的卷积核，以捕获图形细节和全局结构，判别器的输出为图像来自真实分布的概率。并且受 pix2pix 模型的启发，判别器的输入并非仅仅是生成器生成的图像，而是生成图像与输入图像在通道维度上拼接，判别器的结构图如图 3.5 所示：

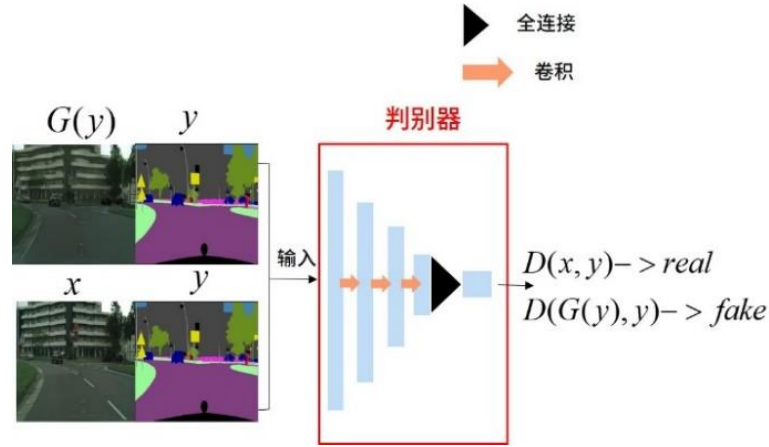


图 3.5 判别器结构图

3.3.3 损失函数

本文构建的生成器是以掩盖了敏感信息的标签图作为输入，即是有条件的输入。因此，对应的条件生成对抗网络的损失函数可以表达为：

$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x, x'}[\log D(x, x')] + \mathbb{E}_{x, y}[\log(1 - D(x, G(x, y)))] \quad (3.1)$$

其中 G 尝试取最小化损失函数，而 D 尝试去最大化损失函数，即 $G^* = \arg \min_G \max_D \mathcal{L}_{cGAN}(G, D)$ 。

生成器的任务不光是要‘欺骗’判别器，也要让输出尽可能接近原始输入，这需要使用额外的损失函数来衡量输入与输出之间的距离。衡量距离最常用的两种方

法是计算目标值与估计值的绝对值差的总和或差值的平方和。由此衍生出两种损失函数 $L1$ 损失和 $L2$ 损失， $L1$ 损失是最小化绝对值差的总和， $L2$ 损失是最小化差值的平方和。文献^[42]表明在条件生成对抗网络中，添加 $L1$ 损失的效果比添加 $L2$ 损失的效果更好，并且考虑到本模型的隐私保护要求，要尽可能使敏感部位与真实图像之间的距离越远越好，因此，本文将添加 $L1$ 损失，表达为：

$$\mathcal{L}_{L1}(G) = \mathbb{E}_{x,y,x'} [\|x' - G(x,y)\|] \quad (3.2)$$

虽然判别器可以确保生成图像服从真实图像的分布，但是不能保证生成图像和真实图像在结构上相似。当遇到复杂的生成场景，GANs 在训练时很容易发生模式崩塌现象，即生成器针对不同的输入永远只生成同一张图像。这是因为基于 GANs 生成的一些图像可能会忽略视觉感知。对此，本文引入结构衡量指标（Structure Similarity Index Measure, SSIM）损失^[45]，SSIM 可以衡量图片的失真程度，也可以衡量两张图片的相似程度。与均方误差和峰值信噪比衡量绝对误差不同，SSIM 是感知模型，即更符合人眼的直观感受。

SSIM 主要考量图片的三个关键特征：亮度 L ，对比度 C ，结构 S 。亮度以平均灰度衡量，通过平均所有像素的值得到，比如真实图像 x 的亮度等于 $\mu_x = \frac{1}{N} \sum_{i=1}^N x_i$ ， N 为图像像素总数，对比两张图像的亮度得到对比函数：

$$l(x, x') = \frac{2\mu_x\mu_{x'} + C_1}{\mu_x^2 + \mu_{x'}^2 + C_1} \quad (3.3)$$

对比度通过灰度标准差来衡量，即 $\sigma_x = (\frac{1}{N-1} \sum_{i=1}^N (x_i - \mu_x)^2)^{\frac{1}{2}}$ ，相应对比函数为：

$$c(x, x') = \frac{2\mu_x\mu_{x'} + C_2}{\mu_x^2 + \mu_{x'}^2 + C_2} \quad (3.4)$$

结构对比比较的是经过归一化后 $x - \mu_x / \sigma_x$ 与 $x' - \mu_{x'} / \sigma_{x'}$ 的比较，可以用相关性系数衡量：

$$s(x, x') = \frac{\sigma_{xx'} + C_3}{\sigma_x\sigma_{x'} + C_3}, \quad \sigma_{xx'} = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_x)(x'_i - \mu_{x'}) \quad (3.5)$$

上式中， C_1, C_2, C_3 均为常量，且要避免相应对比函数的值接近 0 时不稳定。结合式 3.3-3.5 得到 SSIM 为：

$$SSIM(x, x') = l(x, x')^\alpha \cdot c(x, x')^\beta \cdot s(x, x')^\gamma \quad (3.6)$$

其中 α, β, γ 分别表示不同特征在 SSIM 衡量中的占比，在本文中都取 1。SSIM 的值介于 [0,1]，当值等于 0 是说明 x 和 x' 完全不同，当等于 1 时说明 x 和 x' 基本相同，为了最小化损失函数， $\mathcal{L}_{ssim}(G)$ 定义为：

$$\mathcal{L}_{ssim}(G) = \mathbb{E}_{x \sim I, x' \sim I'}[1 - SSIM(x, x')] \quad (3.7)$$

综上，本文构建的生成对抗网络的总损失函数为：

$$\begin{aligned} G^* &= \arg \min_G \max_D \mathcal{L}_{cGAN}(G, D) + \lambda_1 \mathcal{L}_{L1}(G) + \lambda_2 \mathcal{L}_{ssim}(G) \\ \mathcal{L}_{cGAN}(G, D) &= \mathbb{E}_{x, x'}[\log D(x, x')] + \mathbb{E}_{x, y}[\log(1 - D(x, G(x, y)))] \\ \mathcal{L}_{L1}(G) &= \mathbb{E}_{x, y, x'}[\|x' - G(x, y)\|_1] \\ \mathcal{L}_{ssim}(G) &= \mathbb{E}_{x, x'}[1 - SSIM(x, x')], SSIM(x, x') = l(x, x')^\alpha \cdot c(x, x')^\beta \cdot s(x, x')^\gamma \end{aligned} \quad (3.8)$$

其中 λ_1 和 λ_2 是决定后 L1 损失和 SSIM 损失的超参数，同时也充当正则化项调整总损失函数的规模。

3.3.4 模型的结构

本文构建的生成器和判别器是基于文献[42]提出的模型结构构建的。生成器基于 U-net 结构，由编码器和解码器组成，编码器每一层都是使用卷积-批归一化-LeakyReLU 激活函数，解码器每一层都是使用转置卷积-批归一化-ReLU 激活函数。

批归一化是指对每层的输入数据做归一化处理，即将输入数据沿着批次的方向归一化为零均值和单位标准差。在本文构建的网络中，在生成器的编码器模块，有 8 个大小为 4*4 的卷积核，除第一层外，每一层都有 Leaky ReLU 激活函数批量归一化，同时激活函数都放在每个模块的第一步，这是为了方便跳跃连接进行 concat 操作。在解码器中，所有层采用 ReLU 作为激活函数，卷积层替换为转置卷积层。

选取合适的激活函数对神经网络的训练非常重要，通过大量文献阅读，已有实验证明[41]在生成对抗网络中，生成器为编码器解码器结构时，编码器使用 LeakyReLU 和解码器使用 ReLU 的组合的生成器效果较好。

判别器是具有 4 个卷积核大小为 4*4 的卷积层，并以全连接层为结尾的卷积神经网络，具体的网络体系结构信息如表 3.4 所示。

表 3.4 生成对抗网络体系结构

层数	生成器		判别器
	encoder	decoder	
1	4*4*64 Conv,	ReLU, 4*4*512 deconv, BatchNorm	4*4*64 Conv, LeakyReLU
2	Leaky ReLU, 4*4*128 Conv, BatchNorm	ReLU, 4*4*512 deconv, BatchNorm	4*4*128 Conv, BatchNorm, LeakyReLU
3	Leaky ReLU, 4*4*256 Conv, BatchNorm	ReLU, 4*4*512 deconv, BatchNorm	4*4*256 Conv, BatchNorm, LeakyReLU
4	Leaky ReLU, 4*4*512 Conv, BatchNorm	ReLU, 4*4*512 deconv, BatchNorm	4*4*512 Conv, BatchNorm, LeakyReLU
5	Leaky ReLU, 4*4*512 Conv, BatchNorm	ReLU, 4*4*256 deconv, BatchNorm	4*4*1 fullyconnect, Sigmoid
6	Leaky ReLU, 4*4*512 Conv, BatchNorm	ReLU, 4*4*128 deconv, BatchNorm	
7	Leaky ReLU, 4*4*512 Conv, BatchNorm	ReLU, 4*4*64 deconv, BatchNorm	
8	Leaky ReLU, 4*4*512 Conv, BatchNorm	ReLU, 4*4*3 deconv, BatchNorm	

3.3.5 模型的实现

与实现 FCN8s 模型一样，使用 Pytorch 深度学习框架实现道路建筑图像合成模型的建立。首先需要构建名为 Generator 和 Discriminator 的类，两个类都需要继承 nn.Module，并重写 __init__ 函数和 forward 函数。由表 3.4 的网络结构可知，生成器和判别器的每层都包含卷积/转置卷积、LeakyReLU/ReLU 激活函数、批量归一化，因此本文使用 nn.Sequential 按照模块化的方式构建网络。nn.Sequential 是 nn.module 的容器，用于按顺序包装一组网络层。图 3.6 展示了使用 nn.Sequential 构建表 3.4 第 2 层网络的代码，在 nn.Sequential 中依次 nn.LeakyReLU、nn.Conv2d、nn.BatchNorm2d。

```
# 128 * 128
self.en2 = nn.Sequential(
    nn.LeakyReLU(0.2, inplace=True),
    nn.Conv2d(ngf, ngf * 2, kernel_size=4, stride=2, padding=1),
    nn.BatchNorm2d(ngf * 2)
)
```

图 3.6 nn.sequential 构建网络代码示例

表 3.4 中的每层都可以按照上述方式进行构建，具体每层的设置见表 3.5-3.6。

表 3.5 生成器每层参数

层数	操作	输入输出 通道数	Kernel_size 卷积核	Stride 步长	Padding 补丁	层数	操作	输入输出 通道数	Kernel_size 卷积核	Stride 步长	Padding 补丁
Encoder1	Conv2d	3*64	4	2	1	Decoder1	ReLU	\	\	\	\
						Decoder1	ConvTranspose2d	512*512	4	2	1
						Decoder1	BatchNorm2d	512	\	\	\
						Decoder1	Dropout(p=0.5)	\	\	\	\
						Decoder1	ReLU	\	\	\	\
Encoder2	LeakyReLU(0.2)	\	\	\	\	Decoder2	ConvTranspose2d	1024*512	4	2	1
Encoder2	Conv2d	64*128	4	2	1	Decoder2	BatchNorm2d	512	\	\	\
Encoder2	BatchNorm2d	128	\	\	\	Decoder2	Dropout(p=0.5)	\	\	\	\
						Decoder2	ReLU	\	\	\	\
Encoder3	LeakyReLU(0.2)	\	\	\	\	Decoder3	ConvTranspose2d	1024*512	4	2	1
Encoder3	Conv2d	128*256	4	2	1	Decoder3	BatchNorm2d	512	\	\	\
Encoder3	BatchNorm2d	256	\	\	\	Decoder3	Dropout(p=0.5)	\	\	\	\
						Decoder3	ReLU	\	\	\	\
Encoder4	LeakyReLU(0.2)	\	\	\	\	Decoder4	ConvTranspose2d	1024*512	4	2	1
Encoder4	Conv2d	256*512	4	2	1	Decoder4	BatchNorm2d	512	\	\	\
Encoder4	BatchNorm2d	512	\	\	\	Decoder4	Dropout(p=0.5)	\	\	\	\
						Decoder4	ReLU	\	\	\	\
Encoder5	LeakyReLU(0.2)	\	\	\	\	Decoder5	ConvTranspose2d	1024*256	4	2	1
Encoder5	Conv2d	512*512	4	2	1	Decoder5	BatchNorm2d	256	\	\	\
Encoder5	BatchNorm2d	512	\	\	\	Decoder5	Dropout(p=0.5)	\	\	\	\
						Decoder5	ReLU	\	\	\	\
Encoder6	LeakyReLU(0.2)	\	\	\	\	Decoder6	ConvTranspose2d	512*128	4	2	1
Encoder6	Conv2d	512*512	4	2	1	Decoder6	BatchNorm2d	128	\	\	\
Encoder6	BatchNorm2d	512	\	\	\	Decoder6	Dropout(p=0.5)	\	\	\	\
						Decoder6	ReLU	\	\	\	\
Encoder7	LeakyReLU(0.2)	\	\	\	\	Decoder7	ConvTranspose2d	256*64	4	2	1
Encoder7	Conv2d	512*512	4	2	1	Decoder7	BatchNorm2d	64	\	\	\
Encoder7	BatchNorm2d	512	\	\	\	Decoder7	Dropout(p=0.5)	\	\	\	\
						Decoder7	ReLU	\	\	\	\
Encoder8	LeakyReLU(0.2)	\	\	\	\	Decoder8	ReLU	\	\	\	\
Encoder8	Conv2d	512*512	4	2	1	Decoder8	ConvTranspose2d	128*3	4	2	1
Encoder8	BatchNorm2d	512	\	\	\	Decoder8	nn.Tanh()	\	\	\	\

表 3.6 判别器每层参数

层数	操作	输入输出 通道数	Kernel_size 卷积核	Stride 步长	Padding 补丁
layer1	Conv2d	6*64	4	2	1
	LeakyReLU(0.2)	\	\	\	\
layer2	Conv2d	64*128	4	2	1
	BatchNorm2d	128	\	\	\
	LeakyReLU(0.2)	\	\	\	\
layer3	Conv2d	128*256	4	2	1
	BatchNorm2d	256	\	\	\
	LeakyReLU(0.2)	\	\	\	\
layer4	Conv2d	256*512	4	1	1
	BatchNorm2d	512	\	\	\
	LeakyReLU(0.2)	\	\	\	\
layer5	Conv2d	512*1	4	1	1
	Sigmoid	\	\	\	\

至此，基于生成对抗网络的道路建筑图像合成模型构建完成，模型以真实道路建筑图像作为输入，先经过一个基于 FCN8s 的语义分割模型，从真实图像中分割敏感部分，并输出标记出敏感部位的标签图，然后将标签图输入专门构建的生成对抗网络，生成与真实图像结构相似但是敏感部位细节信息不同的合成图像。

3.4 本章小结

本章给出了本文所构建的模型架构图，并分模块地介绍了基于 FCN8s 的语义分割模型和基于 GANs 的道路建筑图像合成模型。首先介绍了 FCN8s 模型的架构以及实现，然后介绍了道路建筑图像合成模型的生成器、判别器、损失函数以及模型总体架构和实现过程，并给出了实现过程中的部分代码截图和网络参数设置。

第 4 章 实验分析与系统部署

4.1 实验过程

4.1.1 数据集准备

本文构建的模型都将通过 `cityscapes` 数据集进行实验。`Cityscapes` 是一种基于语义分割的城市街道场景图像数据集。该数据集从 50 个城市采集了大量的街景图像，其中包含 5 千张经过高品质标注的图像，和两万张经过简单标注的图像。从 <https://www.cityscapes-dataset.com/> 即可下载需要的街道场景数据集，本文下载的数据集为：`leftImg8bit`（大小 11G，存放训练源图）、`gtFine`（大小 241M，存放精细标注文件）、`gtCoarse`（1.3G，存放非精细标注的文件）。

文件夹 `leftImg8bit`：先分 `train/val/test` 三个文件夹，每个文件夹下又按城市分子文件夹，每个城市文件夹下就是源图 `png` 文件。

文件夹 `gtFine`：总共标注了 35 个类，分 `train`（18 个城市）/`val`（3 个城市）/`test`（6 个城市）三个文件夹，每个文件夹下又按城市分子文件夹，每个城市文件夹下针对每张源图 `png` 文件对应了 6 张标注文件：

1. `xxx_gtFine_color.png` 代表可视化的分割图，一般不用；
2. `xxx_gtFine_instanceIds.png` 在实例分割时使用，代表实例分割标注文件；
3. `xxx_gtFine_labelIds.png` 在语义分割时使用，每个像素值就是标签值；
4. `xxx_gtFine_labelTrainIds.png` 训练时使用的标签
5. `xxx_gtFine_polygons.json` 代表边界多边形（35 小类及对应边界像素位置）

此外，借助 `cityscapes scripts` 工具，可以自定义语义分割标签。`Citiscapesscripts/helpers/label.py` 文件包含 `cityscapes` 数据集的所有类和相关 `label` 的对应关系。

如图 4.1 所示，其中“`id`”列代表类别编号，从 0-33 加上 -1 总计有 35 种类型；“`hasInstances`”列代表是否有实例标记，如果为 `True` 则会在 `InstanceIds.png` 标记出来，本文构建语义分割模型，不需要标记实例，所以该列均为 `False`；“`trainId`”表示训练时用到的标签 `id`，如果为 255 则表示不标注，本文将包括道路建筑、行人在内

的 20 类作为敏感信息，并按 0-19 的顺序设置这些类的 trainId，同时需要将每类的“ignoreInEval”值设为 False，然后调用 cityscapesscripts/preparation 里边的 createTrainIdLabelImgs.py 来生成 xxx_labelTrainIds.png，实现自动标记自定义的敏感信息。

#	name	id	trainId	category	catId	hasInstances	ignoreInEval	color
Label(	'unlabeled'	0	255	'void'	0	False	True	(0, 0, 0)
Label(	'ego vehicle'	1	255	'void'	0	False	True	(0, 0, 0)
Label(	'rectification border'	2	255	'void'	0	False	True	(0, 0, 0)
Label(	'out of roi'	3	255	'void'	0	False	True	(0, 0, 0)
Label(	'static'	4	255	'void'	0	False	True	(0, 0, 0)
Label(	'dynamic'	5	255	'void'	0	False	True	(111, 74, 0)
Label(	'ground'	6	255	'void'	0	False	True	(81, 0, 81)
Label(	'road'	7	0	'flat'	1	False	False	(128, 64, 128)
Label(	'sidewalk'	8	1	'flat'	1	False	False	(244, 35, 232)
Label(	'parking'	9	255	'flat'	1	False	True	(250, 170, 160)

图 4.1 cityscapes 数据集的所有类和相关 label 的对应关系

Cityscapes 数据集拥有场景丰富的街道图景，数据集中的图片都来自于车载摄像头，符合本文研究内容的要求。

4.1.2 环境配置

本文实验在 featurize 机器学习在线实验室完成 <https://featurize.cn/>。featurize 是提供搭载 GPU 的在线实验室，使用方法简单多样，提供多种实例租用服务，还提供四种方式连接实例，工作区，SSH，Visual Studio Code 远程连接 和 PyCharm 远程连接。本文租用的实例配置如表 4.1 所示：

表 4.1 实例配置

名称	型号
GPU	GPU RTX 3060， 共 12.6GB 显存
CPU	5 核 E5-2680 v4
内存	27.1GB
硬盘	483.2GB

工作区是经过 Featurize 定制后的 JupyterLab, 可以直接上传自己的代码文件和数据集至工作区。由于 featurize 是按时收费, 本文首先在本地的 pycharm 中搭建好模型, 调试好代码后再将代码打包上传至 featurize 工作区, 开始训练和实验。在本地的 pycharm 中搭建模型使用的软件要与 featurize 工作区的环境兼容, 表 4.2 展示了本文使用到的各个软件的名称及其版本。

表 4.2 主要软件名称及其版本

软件名称	版本
CUDA	11.0
Opencv	4.7.0.72
Python	3.8.16
imageio	2.26.0
numpy	1.23.5
tensorboard	2.12.0
torch	1.13.1
flask	2.2.3

4.1.3 FCN8s 模型训练

按照第三章构建 FCN8s 模型, 并使用 4.1.1 小节构建的数据集训练模型。首先需要构建一个 CityScapesDataset 类, 该类继承 Cityscapes 类。Cityscapes 类是 python 的 torchvision 工具包中自带的专门用于加载 citiscapes 数据集的类。需要重写该类 __init__ 方法、__getitem__ 方法和 __len__ 方法。在 __init__ 中初始化数据集的 root、split 方式(‘train’)、mode 模式(‘fine’, 使用精细标注的数据集)、分割类型(‘semantic’)和类别数(20 类)。在 __getitem__ 中读入图片并做相应的处理, 比如 json 文件到标签图的转换, 标签图与真实图像的对应还有一些数据标准化以及格式转换(numpy->tensor)。

然后构建训练函数, 设置相应的参数, 模型的训练参数设置如表 4.3 所示:

表 4.3 FCN8s 模型训练参数

参数	值	释义
n_class	20	语义分割类别数
batch_size	2	每次训练在训练集中取得样本个数
epochs	100	迭代次数
lr	1e-4	学习率
w_decay	1e-5	权重衰退系数
gamma	0.5	调整学习率的系数
step_size	50	每 50 个 epochs 调整一次学习率

按照以上参数和环境配置的模型，在 cityscapes 数据集上训练一个 epoch 大概需要 20 分钟时间，本文一共训练 100 个 epoch，其损失变化如图 4.2 所示。

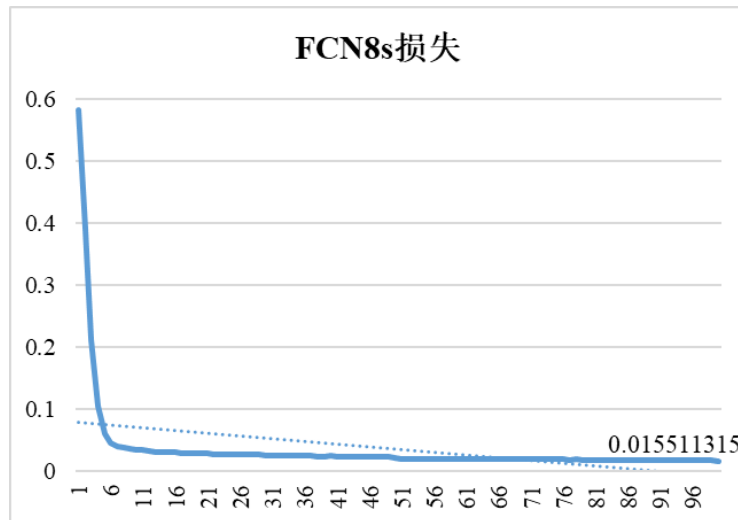


图 4.2 在 cityscapes 数据集上训练 FCN8s 网络的损失变化

由图 4.2 可知，经过 100 轮的训练，FCN8s 的损失值能够较好地收敛。

4.1.4 道路建筑图像合成模型训练

训练第 3.3 节构建的道路建筑合成模型。不同于原始 GANss 模型训练数据集由单张图像组成，本文构建的模型训练数据集为匹配好的真实道路建筑图像以及遮盖了敏感信息的标签图。其中标签图同样借助 cityscapes scripts 工具，生成标记了 20 类敏感信息的标签图，具体方法见 4.1.2 小节介绍。并构建用于匹配真实

图像与标签图的函数，命名为 `process_cityscapes`，该函数的功能就是将两类图片变换为 256×256 的大小然后按图像名进行匹配，最后将两张图像在第一个维度上相加得到一张大小为 512×256 的拼接图像。实现函数的代码截图如图 4.3 所示：

```
def process_cityscapes(gtFine_dir, leftImg8bit_dir, output_dir, phase):
    save_phase = 'test' if phase == 'val' else 'train'
    savedir = os.path.join(output_dir, save_phase)
    os.makedirs(savedir, exist_ok=True)
    os.makedirs(savedir + 'A', exist_ok=True)
    os.makedirs(savedir + 'B', exist_ok=True)
    print("Directory structure prepared at %s" % output_dir)
    segmap_expr = os.path.join(gtFine_dir, phase) + "/*/*.color.png"
    segmap_paths = glob.glob(segmap_expr)
    segmap_paths = sorted(segmap_paths)
    photo_expr = os.path.join(leftImg8bit_dir, phase) + "/*/*.leftImg8bit.png"
    photo_paths = glob.glob(photo_expr)
    photo_paths = sorted(photo_paths)
    assert len(segmap_paths) == len(photo_paths), \
        "%d images that match [%s], and %d images that match [%s]. Aborting." % (len(segmap_paths), segmap_expr, len(photo_paths), photo_expr)
    for i, (segmap_path, photo_path) in enumerate(zip(segmap_paths, photo_paths)):
        check_matching_pair(segmap_path, photo_path)
        segmap = load_resized_img(segmap_path)
        photo = load_resized_img(photo_path)
        # data for pix2pix where the two images are placed side-by-side
        sidebyside = Image.new('RGB', (512, 256))
        sidebyside.paste(segmap, (256, 0))
        sidebyside.paste(photo, (0, 0))
        savepath = os.path.join(savedir, "%d.jpg" % i)
        sidebyside.save(savepath, format='JPEG', subsampling=0, quality=100)
```

图 4.3 `process_cityscapes()`函数截图

然后编写训练生成器和判别器的函数，代码截图如图 4.4 和 4.5 所示：

```
# 训练判别器
def D_train(D: Discriminator, G: Generator, X, BCELoss, optimizer_D):
    # 标签转实物
    image_size = X.size(3) // 2
    # (C,H,W)
    x = X[:, :, :, image_size:].to(device) # 标签图
    y = X[:, :, :, :image_size].to(device) # 实物图
    xy = torch.cat([x, y], dim=1) # 在channel维重叠 xy!=X
    # 梯度初始化=0
    D.zero_grad()
    # 在真实数据上，真实图像为xy
    D_output_real = D(xy).squeeze() # squeeze去除tensor中维度为1的维度
    # logD(xy)
    D_real_loss = BCELoss(D_output_real, torch.ones(D_output_real.size()).to(device))
    # 在假数据上，即由标签图生成的图
    G_output = G(x)
    X_fake = torch.cat([x, G_output], dim=1) # 假数据是指把真实数据和生成数据cat起的数据
    D_output_fake = D(X_fake).squeeze()
    # log(1-D(x,G(x,z)))
    D_fake_loss = BCELoss(D_output_fake, torch.zeros(D_output_fake.size()).to(device))
    # 反向传播并优化
    D_loss = (D_real_loss + D_fake_loss) * 0.5
    D_loss.backward()
    optimizer_D.step()

    return D_loss.data.item()
```

图 4.4 `D_train()`训练判别器的代码截图

```
def G_train(D:Discriminator, G:Generator, X, BCELoss, L1, ssLoss, optimizer_G, lamb, beta):
    # 标签转实物
    image_size = X.size(3) // 2
    x = X[:, :, :, image_size:].to(device)
    y = X[:, :, :, :image_size].to(device)
    G.zero_grad()
    G_output = G(x) # 标签图作为G的输入, x'
    X_fake = torch.cat([x, G_output], dim=1) # 标签图与生成假原图的拼接, xx'
    D_output_fake = D(X_fake).squeeze()
    G_BCELoss = BCELoss(D_output_fake, torch.ones(D_output_fake.size()).to(device))
    G_L1Loss = L1(G_output, y) # x'与y
    ssim_value = 1-ssLoss(G_output, y)
    # 反向传播并优化
    G_loss = G_BCELoss + lamb * G_L1Loss + beta*ssim_value
    G_loss.backward()
    optimizer_G.step()

    return G_loss.data.item(), G_BCELoss.data.item(), G_L1Loss.data.item()
```

图 4.5 G_train()训练生成器的代码截图

D_train()如图 4.4 所示，函数参数为判别器 D、生成器 G、训练数据 X、二分交叉熵损失函数 BCELoss、判别器优化器 optimizer_D。G_train()如图 4.5 所示，函数参数包括判别器 D、生成器 G、训练数据 X、二分交叉熵损失函数 BCELoss、L1 正则化函数、ssim 损失函数、判别器优化器 optimizer_G、L1 正则化的权重、ssim 的权重。

在训练生成对抗网络时，按照来自文献[2]提供的标准方法，交替迭代训练生成器和判别器。但与文献[2]不同的是，并非训练 G 来最小化 $\log(1 - D(x, G(x, y)))$ ，而是最大化 $\log D(x, G(x, y))$ 。本文使用 mini batch SGD 并应用 Adam 优化器，其他训练参数如表 4.4 所示：

表 4.4 道路建筑合成模型训练参数

参数	值	释义
lr_G	0.0002	生成器优化器的学习率
lr_D	0.0002	判别器优化器的学习率
beta1	0.5	优化器的动量系数 1
beta2	0.999	优化器的动量系数 2
lamb	10	L1 损失的权重
beta	200	SSIM 损失的权重
epochs	200	训练迭代次数
batch_size	14	每次训练在训练集中取得样本个数

按照以上参数和环境配置的模型，训练一个 epoch 大概需要 1-2 分钟时间，此外，还配置了 tensorboard 工具实时监控训练过程，tensorboard 是一个可视化工具，它可以可视化将训练过程产生的损失函数值、图像、计算图等。图 4.6 和 4.7 分别展示了判别器和生成器在训练过程时的损失值变化。

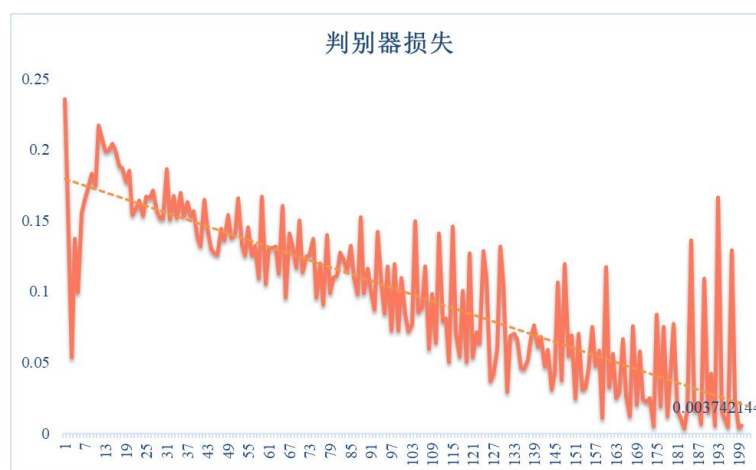


图 4.6 判别器损失值变化图

总共训练 200 个 epoch，在第 200epoch 时判别器的损失值为 0.003742，由图 4.6 可知，判别器的损失逐渐收敛至 0，整个收敛过程波动较大，锯齿现象较为严重，经历了损失迅速下降后又恢复，之后再逐步下降的过程。

生成器的损失值变化如图 4.7 所示，在第 200epoch 时生成器的损失值为 0.74097846，整个训练过程生成器的损失逐渐收敛，收敛速度较为缓慢生成器的对抗能力较判别器稍有逊色。

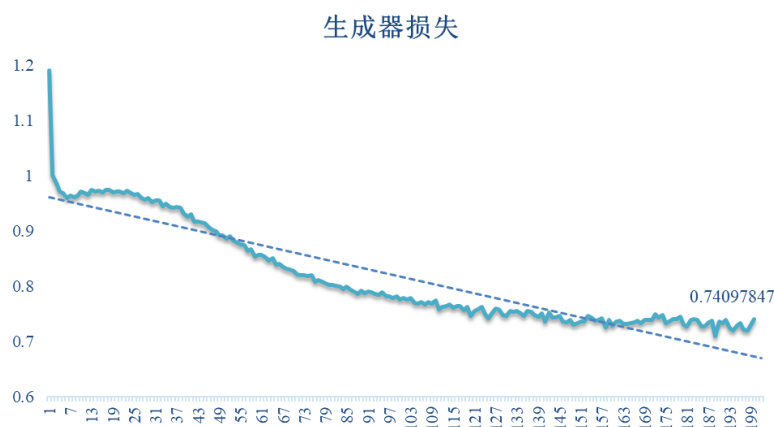


图 4.7 生成器损失值变化图

综上，整个生成对抗网络训练过程较为理想，没有出现一方把另一方完全击败的情况，生成器和判别器都在相互对抗中提升。生成对抗网络训练不稳定，容易出现一方的损失不降反升，另一方的损失迅速收敛的情况，在实验时，当损失函数只有基础的条件对抗损失和 L1 损失时，生成器学习率和判别器学习率同时为 0.0002 时，出现了生成器损失无法收敛一直攀升的情况。

4.2 结果分析

4.2.1 评估指标与对比模型

1) 评估指标

评估生成图像质量最直接的方法是进行 AMT 实验，[47]中的协议指出，志愿者接受一系列测验，在面对真实数据是需要标记为‘真’，面对合成数据是需要标记为‘假’，在每次测验中，每张图像出现 1 秒，志愿者需要在有限的时间内来标记真假。一组测验评估 50 张图像，前 10 张用于训练，即会告诉志愿者标记结果是否正确。召集 50 个志愿者进行评估，最后可以得到每个模型成功骗过志愿者的一个百分比。

FCN-scores 是用于评估图像到图像转换的方法，用于量化合成图像中的隐私保护和识别效用。在 FCN 中，像素精度（pixacc）和联合交互（IoU）是指示图像中物体分割是否正确的两个关键指标。pixacc 与 IoU 的计算公式如下所示：

$$\begin{aligned} \text{pixacc} &= \sum_i n_{ii} / \sum_i t_i \\ \text{IoU} &= \frac{1}{n_{cl}} \sum_i \frac{n_{ii}}{t_i + \sum_j n_{ji} - n_{ii}} \\ t_i &= \sum_j n_{ij} \end{aligned} \quad (4.1)$$

其中 n_{ij} 表示第 i 类像素被预测成属于第 j 类的数目， n_{cl} 为类别数， t_i 表示属于第 i 的所有像素的数目。

2) 对比模型

本文构建的道路建筑图像合成模型是在 pix2pix 的基础上添加了 SSIM 损失函数，为了验证添加新的损失函数对模型效果有提升作用，本文还进行了消融实验，

实验对比了只添加了 L1 损失、只添加了 SSIM 损失以及既添加了 L1 损失又添加了 SSIM 损失（本文构建的模型）这三种模型的结果。

4.2.2 图像适用性和隐私保护效力分析

使用 FCN-scores 评估合成图像，将合成模型第 200 个 epoch 的结果作为语义分割模型的输入，然后将分割结果于真实情况进行比较，对于预定义的敏感部位的像素精度和 IoU 应该低于实际情况，这意味着攻击者无法识别某些像素上的对象，因此无法从生成的图像中检测到真实的位置。

三个模型在 cityscapes 数据集上的像素精度和 IoU 如表 4.5 所示：

表 4.5 FCN-scores 在三个模型上的比较

模型	Loss L1	Loss SSIM	Loss L1+SSIM
全局像素精度	64.05%	43.82%	63.86%
敏感部位像素精度	63.89%	40.16%	42.23%
非敏感部位像素精度	64.11%	45.23%	60.33%
全局 IoU	21.36%	13.46%	21.98%
敏感部位 IoU	17.75%	7.45%	10.12%
非敏感部位 IoU	29.84%	14.21%	24.51%

在表 4.5 中，全局像素精度是指图像中每个像素被分类到正确类别的平均值，敏感部位像素精度是指定义的 20 类敏感信息中的每个像素被分类到正确类别的平均值，非敏感部位像素精度是指除敏感部位其他部位的像素被分类到正确类别的平均值。全局 IoU、敏感部位 IoU 和非敏感部位 IoU 定义与像素精度相似。全局和非敏感部位的像素精度与 IoU 可以用来评估图像识别效用，敏感部位的像素精度与 IoU 用来评估隐私保护效力。

从表 4.5 的数据，可以得到以下结论：

1).仅使用 L1 损失的模型（即 pix2pix）在全球、敏感部位和非敏感部位上的指标成绩相近，都要略优于其他两个模型，这说明 pix2pix 在图像到图像的转换上表现出色，但同时也说明它没有保护特定敏感部位的作用。

2).对于仅使用 SSIM 损失的模型在全局、敏感部位和非敏感部位上的指标成绩相差较大，同时也是三个模型中表现最差的。从表 4.5 可以明显看出使用 SSIM 损失的模型的敏感部位像素进度和敏感部位 IoU 分别为 40.16%、7.45%，与全局像素精度-IoU (43.82%-13.46%)、非敏感部位像素精度-IoU (45.23%-14.21%) 比较，敏感部位的 FCN-scores 表现最差，这也正符合隐私保护的预期。

3).对于本文构建的模型而言，同时使用了 L1 损失和 SSIM 损失，其在全局、敏感部位和非敏感部位的像素精度值分别为 63.86%、42.23%和 60.33%，在全局、敏感部位和非敏感部位的 IoU 值分别为 21.98%、10.12%和 24.51%。可以看出本文构建的模型的像素精度和 IoU 值介于 pix2pix 和仅使用 SSIM 损失的模型之间，但在敏感部位的 FCN-scores 要低与 pix2pix，在全局和非敏感部位的 FCN-scores 要高于仅使用 SSIM 损失的模型。这说明本文构建的模型较好的平衡了生成图像的识别效用与隐私保护效力。

综上，仅使用 L1 损失的 pix2pix，虽然在图像精度方面表现良好但是达不到保护特定敏感部位的效果，而仅使用 SSIM 损失的模型，在保护特定敏感部位的效果较好，这可能是因为 SSIM 衡量两张图像的相似程度更符合人眼的直观感受，从亮度、对比度和结构三个方面来考量。本文构建的模型结合了 L1 损失和 SSIM 损失，在图像的识别效用上和隐私保护效力上都呈现了良好的表现。

4.2.3 图像质量分析

本文将从两个方面来比较生成图像的质量，首先给出不同模型在 cityscapes 数据集上的生成图像，如图 4.8-4.11 所示（更多的合成效果图见附录 A）。



图 4.8 三个模型的生成结果与真实（ground truth）比较

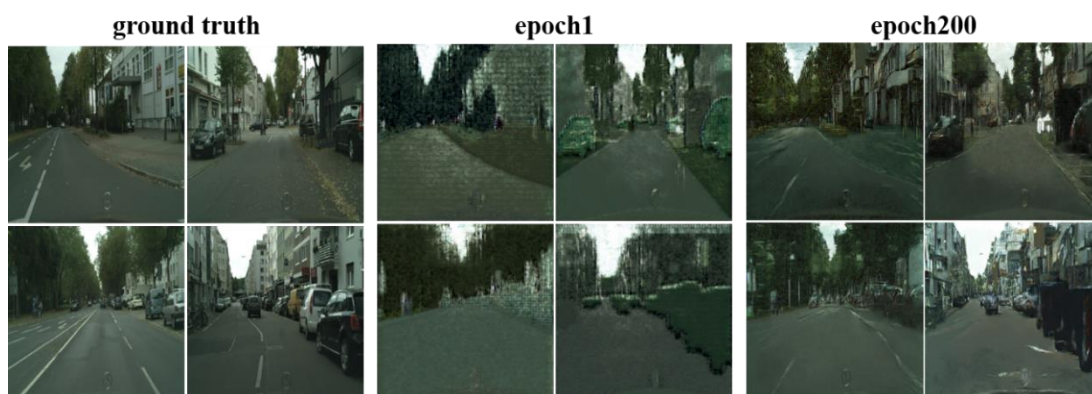


图 4.9 pix2pix (L1 损失) 在 epoch1 和 epoch200 上的结果与真实 (ground truth) 比较

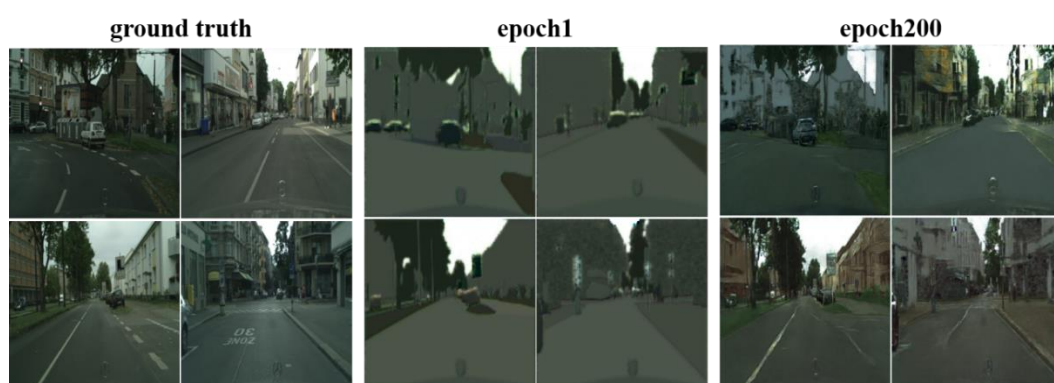


图 4.10 SSIM 损失在 epoch1 和 epoch200 上的结果与真实 (ground truth) 比较

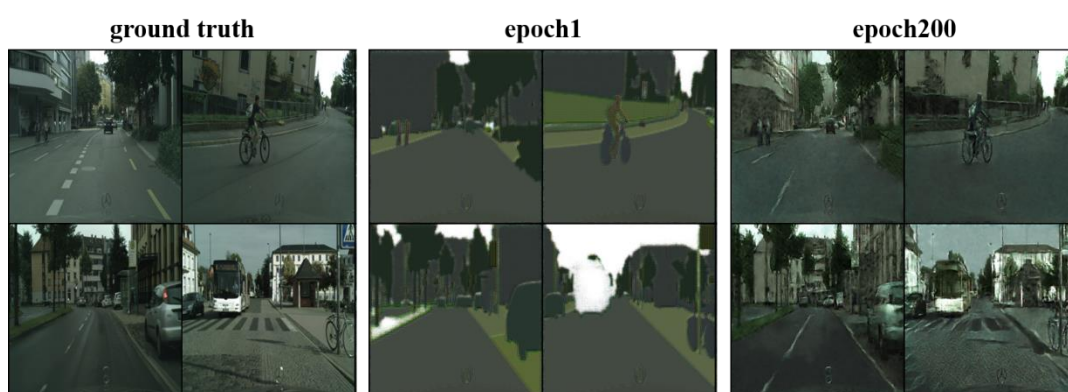


图 4.11 本文构建的模型在 epoch1 和 epoch200 上的结果与真实 (ground truth) 比较

图 4.8 展示了由 pix2pix 模型、使用 SSIM 损失函数的模型以及本文构建的模型生成的合成图像。与真实图像 (ground truth) 对比, 可以肉眼看出 pix2pix 在模拟 ground truth 中的道路建筑细节上表现更好, 基本上能还原建筑外墙的纹理。而使用了 SSIM 损失函数的模型能够较好地还原 ground truth 的物体轮廓信息, 但在

纹理还原上与 ground truth 相差较大，甚至在颜色都有明显不同，同时整个合成图像质量不太好，还有部分畸形的地方。直观上看，本文构建的模型生成的合成图像（图 4.8 最右）质量略优于前两个模型，能够较好地还原真实图像的物体结构，同时做到物体表面细节与真实图像不同。但是三个模型生成的图像在对真实图像的光影模拟上表现得都不佳。

图 4.9-图 4.11 分别展示了三个模型在训练 1 个 epoch 和 200 个 epoch 后生成的合成图像。首先对比训练 1 个 epoch 后生成的合成图像，可以看出，三个模型中只有 pix2pix 在第 1 个 epoch 训练时就学到了物体表面的纹理，其他两个模型在第 1 个 epoch 训练时主要学到的是物体轮廓，所以 pix2pix 在第一轮输出的图像是轮廓不清晰但是有部分纹理的合成图像，使用 SSIM 损失函数的模型与本文构建的模型输出的是物体轮廓明显，由不同灰度色块构成的合成图像。这也间接说明了添加了 SSIM 损失函数的模型在训练一开始就能够很好地捕捉到图像中物体地轮廓。

在训练 200 个 epoch 之后，三个模型地合成效果都有显著提升，均能够较好地模拟真实图像的轮廓结构，其中 pix2pix 还原程度最高，本文构建的模型其次，并且本文构建的模型能在保持图像中物体结构不变的同时，让其表面细节与真实图像存在差异。

除了对图像质量肉眼的评估，本文还自费寻找志愿者进行 AMT 实验。表 4.6 展示了三个模型在 cityscapes 数据集上的 AMT 实验结果。

表 4.6 三个模型的 AMT 实验分数

模型	分数
Loss L1 (pix2pix)	15.8% ± 2.3%
Loss SSIM	4.6% ± 0.5%
Loss L1+SSIM	10.01% ± 0.3%

从表 4.6 可以看出，本文构建的模型（Loss L1+SSIM）可以欺骗 10.01% 的参与者。表现最好的是 pix2pix 模型，能够骗过 15.8% 的参与者。总而言之，三个模型在生成图像质量上都有待提高。

4.3 系统部署

为了更好地展示本文构建的框架，本文基于 Flask 和 html 标签语义搭建了用于展示和部署道路建筑图像生成模型的网页。本文搭建的 Flask 项目目录结构如图 4.12 所示：

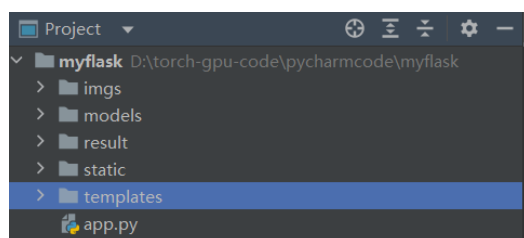


图 4.12 Flask 项目目录结构

其中 `models` 用于存放 FCN8s 网络结构文件、道路建筑图像合成模型网络结构文件以及加载两个模型的文件 `online_eval.py`，在 `online_eval.py` 中用 `torch.load()` 加载两个模型。`static` 用于存放静态资源文件，比如需要传递至前端的合成图像，从前端下载的真实图像。`templates` 用于存放模块，即 html 文件、css 文件等。`app.py` 是 flask 的应用程序文件，在应用程序中通过‘`@app.route`’装饰器定义路由和视图函数。本文编写的 `app.py` 简单地定义了两个路由，一个用来返回网页入口，一个用来下载来自前端的图像并传递合成图像的 url 至前端。使用 flask 完成前后端数据交互和图像合成的流程如图 4.13 所示：

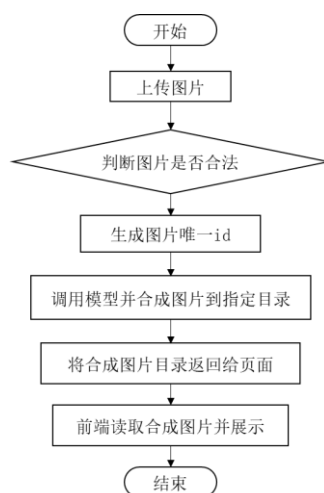


图 4.13 前后端数据交互流程图

网页包含六个模块：模型介绍、模型训练、模型对比、示例、在线合成。图 4.14-4.19 分别展示了每个模块的界面。该系统网页简单介绍了本文构建的模型的框架、基本思想、训练过程和实验结果，可以通过右上角的导航栏直接定位到任意一个模块。



图 4.14 模型介绍页面

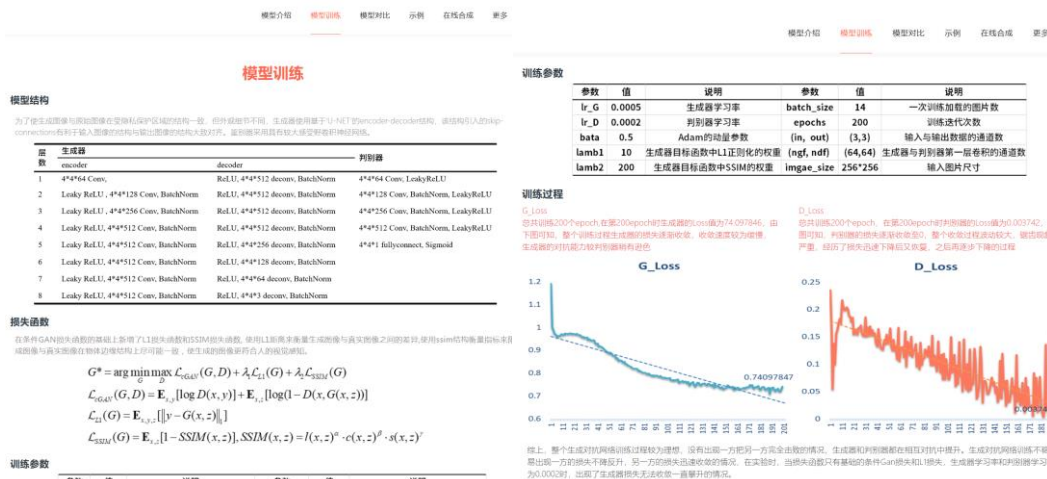


图 4.15 模型训练页面



图 4.16 模型对比页面



图 4.17 合成效果示例页面

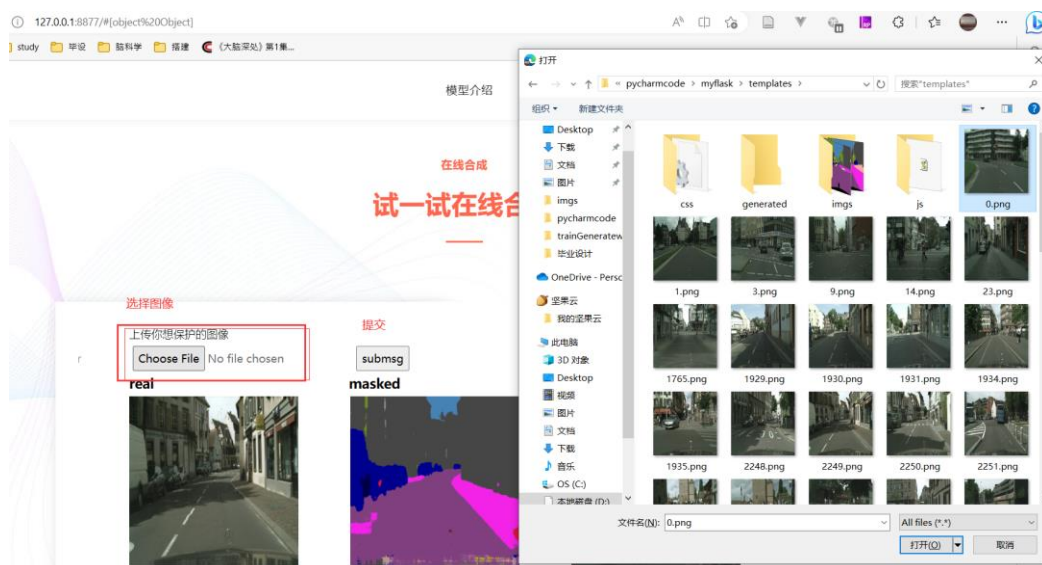


图 4.18 在线合成页面（1）



图 4.19 在线合成页面（2）

4.4 本章小结

本章介绍了本文实验的环境配置、数据集准备、FCN8s 模型训练配置、道路建筑图像合成模型训练配置以及训练结果的分析，最后还介绍了如何将本文训练好的模型部署到网页上，和对模型在网页上的可视化。

第5章 总结与展望

5.1 主要工作与总结

本文聚焦自动驾驶领域，开展了有关如何保护自动驾驶车载摄像头记录的道路建筑图像位置隐私的相关工作。首先分析了本文的研究背景与意义，并综述了国内外有关自动驾驶隐私保护、图像隐私保护和生成对抗网络的研究现状，从已有文献中受到启发，确定了本文工作的主要内容。然后系统地介绍了本文将用到的相关理论与技术。接着，给出了本文模型架构的设计思路和具体的实现细节，并使用 cityscapes 数据集对本文构建的模型进行了实验。最后，给出了模型部署过程，并给出了一个可视化的前端页面。

总而言之，本文使用新颖的图像隐私保护思路，和较为先进的方法技术构建了用于保护车载摄像头记录的道路建筑图像隐私的模型，同时实验结果表明，本文构建的模型能够较好地折衷合成图像的适用性和隐私保护效力。

下面将分为两部分详细说明本文的主要工作以及总结。

5.1.1 主要工作

本文完成的主要工作包括以下三个方面：

1) 创新性地引入 SSIM 结构损失函数作为约束条件，利用 FCN8s 语义分割模型提取真实道路建筑物图像敏感信息，构建用于合成道路建筑图像的基于生成对抗网络的图像合成模型，其中生成器基于 U-net 结构，判别器为卷积神经网络。

2) 使用 pytorch 深度学习框架搭建并训练本文设计的模型，针对构建的道路建筑图像合成模型进行了消融对比实验，从图像质量、FCN-scores、AMT 分数三个方面评估本文构建的合成模型与其他两个模型的图像适用性和隐私保护效力，结果表明本文构建的模型在全局、敏感部位和非敏感部位的像素精度值分别为 63.86%、42.23%和 60.33%，在全局、敏感部位和非敏感部位的 IoU 值分别为 21.98%、10.12%和 24.51%。与其他两个模型相比，能够较好地折衷合成的适用性和隐私保护效力；

3) 构建了一个可以保护车载摄像头记录的道路建筑图像位置隐私的图像处理框架，该框架由一个语义分割模型和一个图像合成模型组成。当真实道路建筑图像通过该框架时，首先会输入进语义分割模型，得到掩盖敏感信息的标签图，然后标签图和真实图像会同时输入图像合成模型，合成一张结构与原图相似但是敏感部位细节不同的图像。最后，使用 Flask 框架将两个模型部署至网页，并将模型结构在网页上可视化。

5.1.2 总结

在实现整个模型的过程中，首先对生成对抗网络的理论知识以及图像语义分割的理论知识进行了系统性地学习。在指导老师和学长学姐地帮助下，一点一点弄懂隐藏于神经网络下的数学公式和损失函数背后的数学原理，这让我把大一大二学习的高等数学、线性代数和概率论等基础数理知识巩固加深温习了一遍，感到受益匪浅。在拿到课题时，面对如何生成位置隐私受保护的图像，一时思绪空白无从下手，幸得有指导老师的帮助，引导我查阅参考文献，一点一点打开解题思路。最终在文献[32][42]的启发下，有了用语义分割+生成对抗网络组合模型的想法来解决本文提出的问题。之后经历了较长的知识储备技能学习阶段，在搭建完模型，开始训练时，遇到了自己的电脑算力不足，导致训练一个模型要四五天的时间，还是在学姐学长的介绍下使用在线实验平台进行实验，最终顺利完成实验。当然实验过程也并非一帆风顺，中间也遇到过由于网络参数设置不对、文件路径不对、图像格式转换不对等引起的种种错误，当然也在自己的努力下成功解决了各种问题。最后，由于自己对前后端知识的欠缺，在模型部署上耗费了大量时间，经过各种查阅资料和学习，才完成了简单的模型部署。

总而言之，整个框架和模型的实现都让我感受到了十足的成就感。通过毕业设计，让我深刻理解了“纸上得来终觉浅，得知此事要躬行”，大学前三年学习的理论知识，在最后得到了综合地实践，也是给毕业一份圆满的答卷。

5.2 后续研究工作展望

有关自动驾驶摄像头记录的道路建筑图像的隐私保护工作将会继续开展。本框架虽然能够在一定程度上解决问题，但是实现过程比较复杂，同时合成图像的精度有待提高。

在提取图像敏感信息部分需要采取更先进、智能快速的方法，同时对敏感部位的定义也有待考量。本文使用的 FCN8s 模型，当选定不同敏感部位时，需要重新训练模型，这带来了许多冗余操作。得益于近年来深度学习大模型带来的从量变到质变的突破，目前已有能够“分割万物”的模型，即 Meta AI 在今年 2023 年 4 月开源的 SegmentAnything 模型^[48]。该模型实现了分割技术的大众化，只要在图像中指定要分割的内容，就可以实现各种分割任务，不需要额外训练。若将该分割方法使用在本文构建的模型中，大概率上能够提升合成效率。

在图像合成部分，可以从两个方面进行改进，一方面是在原本模型的基础上调整网络结构和损失函数以获得更好的结果，另一方面可以直接使用更先进的模型，比如扩散模型^[49]，来合成符合要求的道路建筑图像。

此外，自己对深度学习这方面数理理论的理解还不够透彻，以及在前后端网页构建上的编程能力也有待提高，在后续的学习和工作过程中，我将针对自己薄弱的地方加强学习。虽然在之后的研究生学习过程中可能不在涉及自动驾驶隐私保护方面的工作，但仍然会探索有关深度学习的更多研究。

参考文献

- [1] Li Z, Yang W, Peng S, et al. A survey of convolutional neural networks: analysis, applications, and prospects [J]. IEEE Transactions on Neural Networks and Learning Systems, 2022: 6999-7019.
- [2] Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial networks[J]. Communications of the ACM, 2020, 63(11): 139-144..
- [3] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]// In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17). NY, USA: Curran Associates Inc, 2017:6000–6010.
- [4] IMARC Group, Machine vision market: global industry trends, share, size, growth, opportunity and forecast 2023-2028 [R] US: IMARC, 2023.
<https://www.researchandmarkets.com/reports/5732446/machine-vision-market-global-industry-trends>
- [5] 徐海龙, 陈志. 全球自动驾驶产业创新政策趋势及思考[J]. 全球科技经济瞭望, 2021,36(6):6.
- [6] 解正山. 数据驱动时代的数据隐私保护——从个人控制到数据控制者信义义务[J]. 法商研究,2020,37(2):14.
- [7] 卜冠华,周礼亮,李昊,张敏. 基于深度学习的 GPS 轨迹去匿名研究[J]. 计算机工程与科学,2022,44(02): 244-250.
- [8] Lee D, Hess D J. Public concerns and connected and automated vehicles: safety, privacy, and data security[J]. Humanities and Social Sciences Communications, 2022, 9(1): 1-13.
- [9] 田丰,吴振强,鲁来凤,等. 面向轨迹数据发布的个性化差分隐私保护机制[J]. 计算机学报,2021,44(4):15.
- [10] Krontiris I, Giannetsos T, Schoo P, et al. Buckle-up: autonomous vehicles could face privacy bumps in the road ahead[M]. Ruhr-Universität: Bochum, 2020:23-25.
- [11] 付宇,王红. 位置隐私保护的虚拟轨迹填充算法[J]. 计算机应用,2019,39（8）:8.
- [12] Wang H, Li Y, Gao C, et al. Anonymization and de-Anonymization of mobility trajectories: dissecting the gaps between theory and practice[J]. IEEE Transactions on Mobile Computing, 2021, vol.20 no.3: 796-815.

-
- [13] Orekondy T, Fritz M, Schiele B . Connecting pixels to privacy and utility: automatic redaction of private information in images[C]//Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 2018, pp. 8466-8475.
- [14] Dwork C, Mc Sherry F, Nissim K, et al. Calibrating noise to sensitivity in private data analysis[C]//Theory of Cryptography Conference. Springer, Berlin, Heidelberg, 2006: 265-284.
- [15] 吴云乘,陈红,赵素云等.一种基于时空相关性的差分隐私轨迹保护机制[J].计算机学报,2018,41(02):309-322.
- [16] Ahmed S, Chowdhury A R, Fawaz K, et al. Preech: a system for privacy-preserving speech transcription[C]//Proceedings of the 29th USENIX Conference on Security Symposium. Wilmington , DE , USA, 2020: 2703-2720.
- [17] Wu Y, Fan Y, Yong X, et al. Privacy-protective-gan for privacy preserving face de-identification[J]. 计算机科学技术学报:英文版,2019,34(1):14.
- [18] Pittaluga F, Koppal S J, Chakrabarti A. Learning privacy preserving encodings through adversarial training[C]// 2019 IEEE Winter Conference on Applications of Computer Vision (WACV). USA: Waikoloa, 2019:791-799,.
- [19] Chatzikokolakis K, Palamidessi C, Stronati M. Location privacy via geo-indistinguishability[J]. ACM Siglog News, 2015, 2(3):46-69.
- [20] Ray N K, Puthal D, Ghai D . Federated learning[J]. IEEE Consumer Electronics Magazine, 2021,(10-6):50-60.
- [21] Zhang H, Bosch J, Olsson H H . End-to-end federated learning for autonomous driving vehicles[C]// International Joint Conference on Neural Networks. IEEE. Shenzhen China, 2021:1-8.
- [22] 韩继田. 基于混沌和 DNA 编码的图像加密算法研究[D].兰州:兰州理工大学,2022.
- [23] 魏涵贵. 基于差分隐私的图像数据发布研究[D].广西:广西民族大学,2021.
- [24] 张啸剑, 付聪聪, 孟小峰. 结合矩阵分解与差分隐私的人脸图像发布[J]. 中国图象图形学报, 2020, 25(4):14.
- [25] 李琪,廖鑫,屈国庆等.基于 Arnold 变换的数字图像自适应隐写算法[J].通信学报,2016,37(06):192-198.

-
- [26] 付章杰,王帆,孙星明等.基于深度学习的图像隐写方法研究[J].计算机学报,2020,43(09):1656-1672.
 - [27] Volkhonskiy D, Nazarov I, Burnaev E. Steganographic generative adversarial networks[J]. International Conference on Machine Vision, 2017, 21(03):9-17.
 - [28] 葛瑞. 基于生成对抗网络的图像隐私保护方法研究[D].西安:西安电子科技大学,2020.
 - [29] Yu J, Zhang B, Kuang Z, et al. iPrivacy: image privacy protection by identifying sensitive objects via deep multi-task learning[J]. IEEE Transactions on Information Forensics & Security, 2017, 12(5):1005-1016.
 - [30] Chen J, Konrad J, Ishwar P. VGAN-based image representation learning for privacy-preserving facial expression recognition[C]// 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW). Salt Lake City, UT, USA:IEEE, 2018: 1651-165109.
 - [31] Xie L, Lin K, Wang S, et al. Differentially private generative adversarial network[OL]. 2018-02-19, <https://arxiv.org/abs/1802.06739>.
 - [32] 谢艺艺,张玉书,赵若宇,等. 基于 CycleGANs 的图像隐私保护[J]. 应用科学学报,2023,41(2): 228-239.
 - [33] Xiong Z, Cai Z, Han Q, et al. ADGAN: protect your location privacy in camera data of auto-driving vehicles[J]. IEEE Transactions on Industrial Informatics, 2020, PP(99):1-1.
 - [34] Long J, Shelhamer E, Darrell T. Fully convolutional networks for semantic segmentation[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. Boston, MA, USA, 2015: 3431-3440.
 - [35] Saharia C, Chan W, Chang H, et al. Palette: image-to-image diffusion models[C]// ACM SIGGRAPH 2022 Conference Proceedings. New York, NY, USA, 2022:1-10.
 - [36] Zhu J Y, Park T, Isola P, et al. Unpaired image-to-image translation using cycle-consistent adversarial networks[J]. IEEE International Conference on Computer Vision (ICCV), 2017:2242-2251.
 - [37] 田萱,王亮,丁琪. 基于深度学习的图像语义分割方法综述[J]. 软件学报, 2019, 30(2): 440-468.
 - [38] 徐姿,张慧灵. 非凸极小极大问题的优化算法与复杂度分析[J]. 运筹学学报, 2021,25(3): 74-86.

-
- [39] Mehdi Mirza, Simon Osindero. Conditional generative adversarial nets.[J]. Computer Science, 2014:2672-2680.
- [40] Arjovsky M, Chintala S, Bottou L. Wasserstein gan[C]// Proceedings of the 34th International Conference on Machine Learning. Sydney, NSW, Australia: JMLR.org, 2017:214-223.
- [41] Yu Y, Gong Z, Zhong P, et al. Unsupervised representation learning with deep convolutional neural network for remote sensing images[C]//Image and Graphics: 9th International Conference, ICIG 2017, Shanghai, China, September 13-15, 2017, Revised Selected Papers, Part II 9. Springer International Publishing, 2017: 97-108.
- [42] Isola P, Zhu J Y, Zhou T, et al. Image-to-image translation with conditional adversarial networks[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. Honolulu, HI, USA, 2017: 1125-1134..
- [43] 黄玉萍,梁炜萱,肖祖环. 基于 TensorFlow 和 PyTorch 的深度学习框架对比分析[J]. 现代信息科技, 2020, 4(4):4.
- [44] 余晓帆,朱丽青.基于 Flask 框架的社交网站数据爬取及分析[J].微型电脑应用, 2022, 38(3):9-12.
- [45] Li B, Zhang H, Wang Z, et al. Unsupervised monocular depth estimation with aggregating image features and wavelet SSIM (structural similarity) loss[J]. Intelligence & Robotics, 2021, 1(1):84-98.
- [46] Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift[C]//International conference on machine learning. pmlr, Lille, France, 2015: 448-456.
- [47] Zhang R, Isola P, Efros A A. Colorful image colorization[C]//Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part III 14. Springer International Publishing, 2016: 649-666.
- [48] Kirillov A, Mintun E, Ravi N, et al. Segment anything[OL]. 2023-04-05, <https://arxiv.org/abs/2304.02643>.
- [49] Ho J, Jain A, Abbeel P. Denoising diffusion probabilistic models[J]. Advances in Neural Information Processing Systems, 2020, 33: 6840-6851.

作者介绍

张晓珊,女,重庆邮电大学计算机科学与技术学院/人工智能学院,计算机科学与技术专业菁英班 2019 级本科生。

大学期间参与的科研训练计划以及发表的学术论文和专利:

1) 2019.12-2022.05 省部级大学科研训练计划项目“面向时空数据的联盟学习方法研究” 项目成员

2) 2021.06-2022.05 中国科学院大学重庆学院菁英班创新人才培养项目“地铁流量预测系统” 队长

3) Wang, H., Xu, Z., **Zhang, X.** et al. An optimal differentially private data release mechanism with constrained error. Front. Comput. Sci. 16, 161608 (2022). (SCI 期刊,导师一作)

4) 王豪, **张晓珊**, 陈贤, 夏英, 张旭. 一种关联元组数据的差分隐私发布方法及系统:, CN202011165692.7[P]. 2022.(已授权,导师一作)

大学期间参加的科创比赛以及获得的奖项:

1) 2021 年高教社杯全国大学生数学建模比赛 全国一等奖 (中国工业与应用数学学会、全国大学生数学建模竞赛委员会)

2) 2022 年第十三届“挑战杯”中国大学生创业计划竞赛重庆赛区 银奖 (团队排名第三)

3) 2022 年第八届中国国际“互联网+”大学生创新创业大赛重庆赛区 金奖 (团队排名第三)

致谢

毕业设计是我大学生涯画卷中最后一笔浓墨重彩,也是我入门学术研究的初探。在完成毕业设计的过程中,有太多人给予了我无私的帮助和支持,其中,我要特别感谢指导老师王豪老师对我的悉心指导,定期检查我的完成进度,耐心解答我的问题,王老师的严格要求和高质量标准是督促我顺利完成毕业设计的保证。

四载悠悠,学途漫漫,睹物思人,概莫能外,纵有不舍,皆是感恩。

感谢张婷婷老师和周见老师,给予我足够的鼓励与信任,帮助我快速适应大学学习生活。感谢我的家人、好朋友和室友们对我生活以及精神上的照顾与支持。

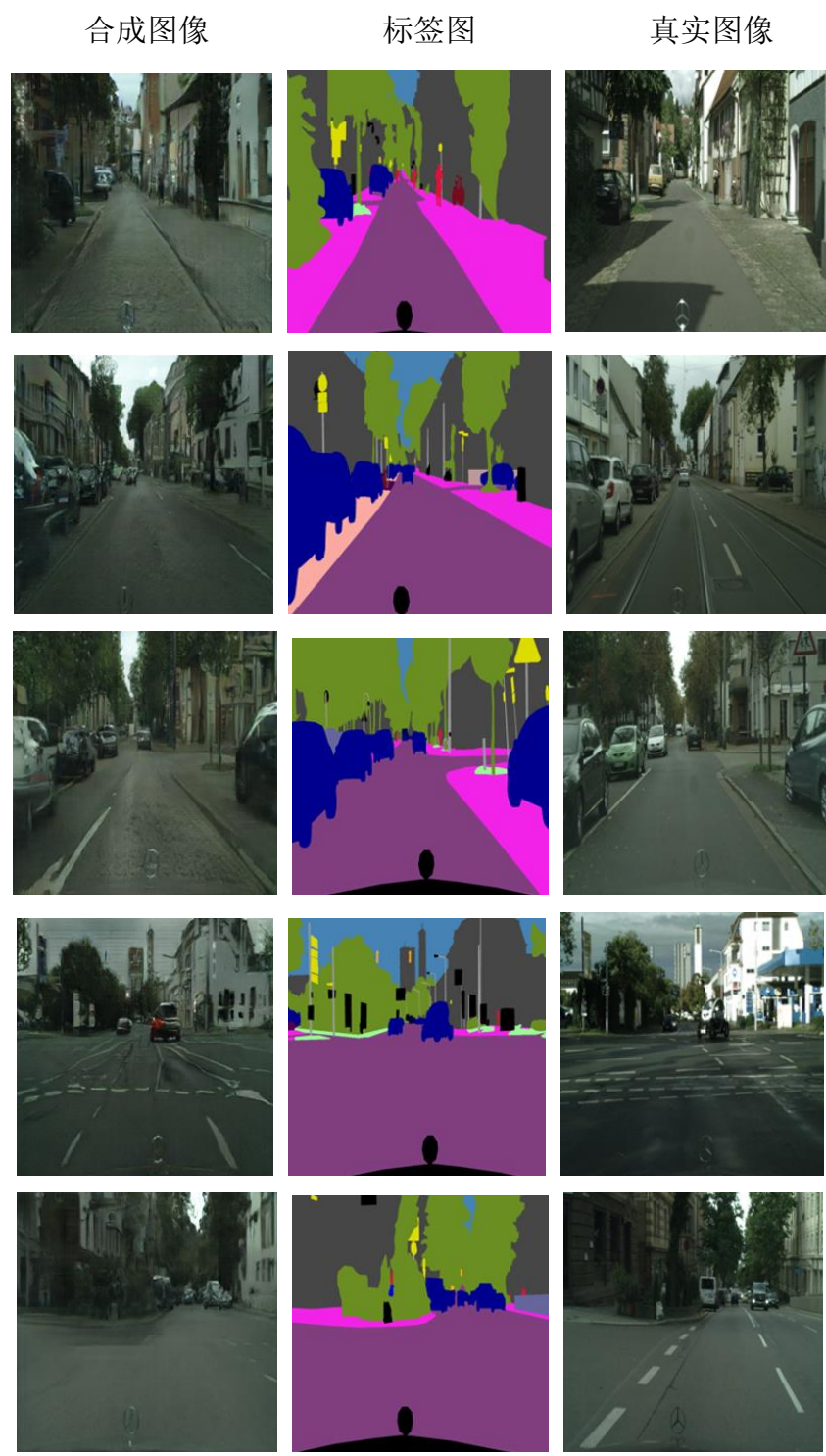
感谢王豪老师,王豪老师是我在菁英班的科研指导老师,从大一下学期开始,王豪老师就定期组织会议关心我们的学习情况,还会带领我们学习一些科研方法,如阅读文献、编程技能等,同时积极指导我们小组参加大学生科研训练计划、创新创业比赛等。不负老师的指导与栽培,在大三能够以第二专利人产出已授权专利一篇,并作为第三作者产出 SCI 论文一篇。王豪老师是我的科研启蒙老师,帮助我带领我打开科学研究这片广阔领域的大门。

感谢我的数模队友们——刘洋、李瑞堃,从大一开始,我们就一起学习各种模型知识、编程技术、写作技巧,一起参加各种大大小小的数模比赛,一起并肩作战,一起熬夜,只为在短短三四天建立模型解决问题递交一份完美的数模论文。我们一起在实验楼见过凌晨四五点微亮的天空,感受过 40°C 炽热的重邮校园,当疫情期间校园学生寥寥无几时,我们还在几平米的小实验室里建模型跑代码。当然努力付出也得到了对等的成果,我们获得了 2021 年高教社杯全国大学生数学建模比赛一等奖。这一切都将成为我大学四年最美好的回忆,感谢你们同我创造了这个属于我们的独一无二的大学生活。

感谢重庆邮电大学对我四年的栽培。是重邮的专业与高水准,让我学习到丰富的知识,为自己的专业打下坚实的基础。是重邮的多元与包容,让我有机会参加各种比赛、讲座、志愿活动,进行全方位的发展。是重邮的培养,众多老师的悉心指导教诲,家人同学的陪伴,助力我保研至西北工业大学,继续在人工智能、深度学习领域学习探索。至此,再次感谢陪我一路走过来的老师、同学、朋友和家人们。

最后,感谢答辩组的各位评委老师、专家教授在百忙之中抽空评阅我的论文,参加我的论文答辩。

附录 A 合成图像效果



附录 B 英文翻译

原文题目: Image-to-Image Translation with Conditional Adversarial Networks

作者: Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, Alexei A. Efros

来源: 《IEEE 计算机视觉和模式识别会议 (CVPR) 论文集》

Isola P, Zhu J Y, Zhou T, et al. Image-to-image translation with conditional adversarial networks[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2017: 1125-1134

注: 图表及参考文献已略去(见原文)

原文

Image-to-Image Translation with Conditional Adversarial Networks

ABSTRACT

We investigate conditional adversarial networks as a general-purpose solution to image-to-image translation problems. These networks not only learn the mapping from input image to output image, but also learn a loss function to train this mapping. This makes it possible to apply the same generic approach to problems that traditionally would require very different loss formulations. We demonstrate that this approach is effective at synthesizing photos from label maps, reconstructing objects from edge maps, and colorizing images, among other tasks. Indeed, since the release of the pix2pix software associated with this paper, a large number of internet users (many of them artists) have posted their own experiments with our system, further demonstrating its wide applicability and ease of adoption without the need for parameter tweaking. As a community, we no longer hand-engineer our mapping functions, and this work suggests we can achieve reasonable results without hand-engineering our loss functions either.

1. INTRODUCTION

Many problems in image processing, computer graphics, and computer vision can be posed as “translating” an input image into a corresponding output image. Just as a concept may be expressed in either English or French, a scene may be rendered as an RGB image, a gradient field, an edge map, a semantic label map, etc. In analogy to automatic language translation, we define automatic image-to-image translation as the task of translating one possible representation of a scene into another, given sufficient training data (see Figure 1). Traditionally, each of these tasks has been tackled with separate, special-purpose machinery (e.g., [16, 25, 20, 9, 11, 53, 33, 39, 18, 58, 62]), despite the fact that the setting is always the same: predict pixels from pixels. Our goal in this paper is to develop a common framework for all these problems.

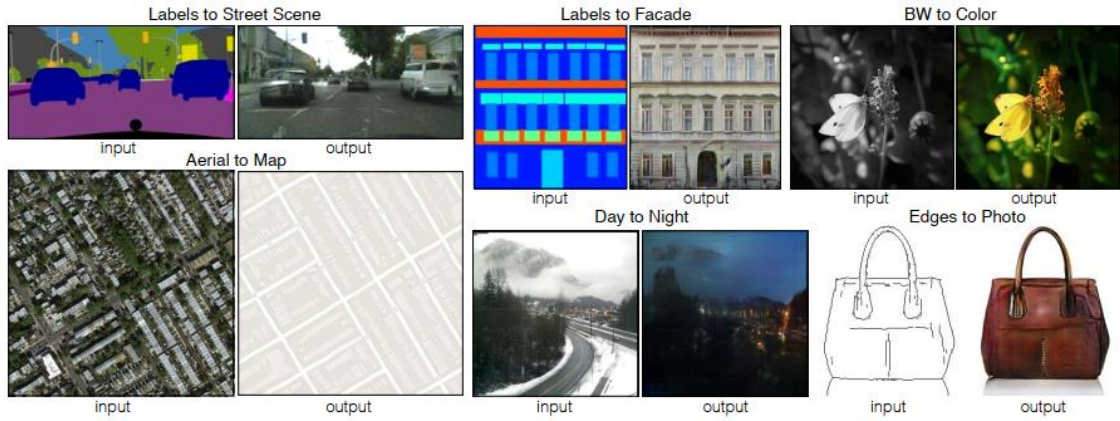


Figure 2.1: Many problems in image processing, graphics, and vision involve translating an input image into a corresponding output image. These problems are often treated with application-specific algorithms, even though the setting is always the same: map pixels to pixels. Conditional adversarial nets are a general-purpose solution that appears to work well on a wide variety of these problems. Here we show results of the method on several. In each case we use the same architecture and objective, and simply train on different data.

The community has already taken significant steps in this direction, with convolutional neural nets (CNNs) becoming the common workhorse behind a wide variety of image prediction problems. CNNs learn to minimize a loss function – an objective that scores the quality of results – and although the learning process is automatic, a lot of manual effort still goes into designing effective losses. In other words, we still have to tell the CNN what we wish it to minimize. But, just like King Midas, we must be careful what we wish for! If we take a naive approach and ask the CNN to minimize the Euclidean distance between predicted and ground truth pixels, it will tend to produce blurry results [43, 62]. This is because Euclidean distance is minimized by averaging all plausible outputs, which causes blurring. Coming up with loss functions that force the CNN to do

what we really want – e.g., output sharp, realistic images – is an open problem and generally requires expert knowledge.

It would be highly desirable if we could instead specify only a high-level goal, like “make the output indistinguishable from reality”, and then automatically learn a loss function appropriate for satisfying this goal. Fortunately, this is exactly what is done by the recently proposed Generative Adversarial Networks (GANs) [24, 13, 44, 52, 63]. GANs learn a loss that tries to classify if the output image is real or fake, while simultaneously training a generative model to minimize this loss. Blurry images will not be tolerated since they look obviously fake. Because GANs learn a loss that adapts to the data, they can be applied to a multitude of tasks that traditionally would require very different kinds of loss functions.

In this paper, we explore GANs in the conditional setting. Just as GANs learn a generative model of data, conditional GANs (cGANs) learn a conditional generative model [24]. This makes cGANs suitable for image-to-image translation tasks, where we condition on an input image and generate a corresponding output image.

GANs have been vigorously studied in the last two years and many of the techniques we explore in this paper have been previously proposed. Nonetheless, earlier papers have focused on specific applications, and it has remained unclear how effective image-conditional GANs can be as a general-purpose solution for image-to-image translation. Our primary contribution is to demonstrate that on a wide variety of problems, conditional GANs produce reasonable results. Our second contribution is to present a simple framework sufficient to achieve good results, and to analyze the effects of several important architectural choices. Code is available at <https://github.com/phillipi/pix2pix>.

2. RELATED WORK

2.1 Structured losses for image modeling

Image-to-image translation problems are often formulated as per-pixel classification or regression (e.g., [39, 58, 28, 35, 62]). These formulations treat the output space as “unstructured” in the sense that each output pixel is considered conditionally independent from all others given the input image. Conditional GANs instead learn a structured loss. Structured losses penalize the joint configuration of the output. A large body of literature has considered losses of this kind, with methods including conditional random fields [10], the SSIM metric [56], feature matching [15], nonparametric losses [37], the convolutional pseudo-prior [57], and losses based on matching covariance statistics [30].

2.2 Conditional GANs

We are not the first to apply GANs in the conditional setting. Prior and concurrent works have conditioned GANs on discrete labels [41, 23, 13], text [46], and, indeed, images. The image-conditional models have tackled image prediction from a normal map [55], future frame prediction [40], product photo generation [59], and image generation from sparse annotations [31, 48] (c.f. [47] for an autoregressive approach to the same problem). Several other papers have also used GANs for image-to-image mappings, but only applied the GAN unconditionally, relying on other terms (such as L2 regression) to force the output to be conditioned on the input. These papers have achieved impressive results on inpainting [43], future state prediction [64], image manipulation guided by user constraints [65], style transfer [38], and superresolution [36].

Our method also differs from the prior works in several architectural choices for the generator and discriminator. Unlike past work, for our generator we use a “U-Net”-based architecture [50], and for our discriminator we use a convolutional “PatchGAN” classifier, which only penalizes structure at the scale of image patches. A similar PatchGAN architecture was previously proposed in [38] to capture local style statistics. Here we show that this approach is effective on a wider range of problems, and we investigate the effect of changing the patch size.

3. METHOD

GANs are generative models that learn a mapping from random noise vector z to output image y , $G: z \rightarrow y$ [24]. In contrast, conditional GANs learn a mapping from observed image x and random noise vector z , to y , $G: \{x, z\} \rightarrow y$. The generator G is trained to produce outputs that cannot be distinguished from “real” images by an adversarially trained discriminator, D , which is trained to do as well as possible at detecting the generator’s “fakes”. This training procedure is diagrammed in Figure 2.

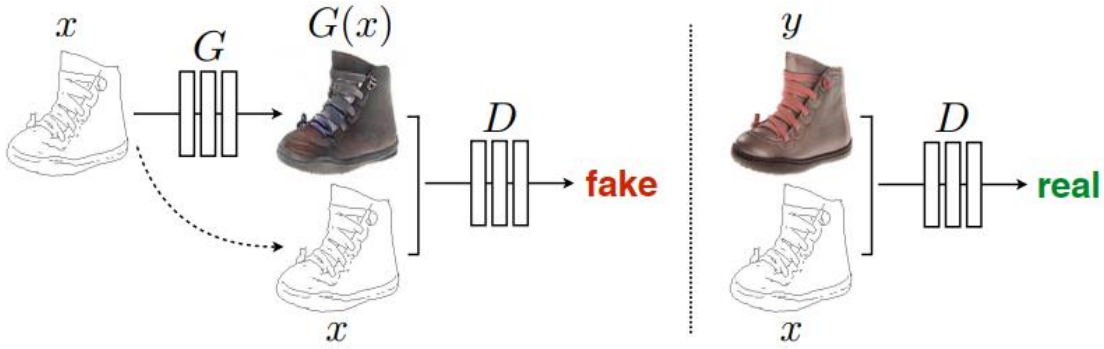


Figure 2: Training a conditional GAN to map edges!photo. The discriminator, D , learns to classify between fake (synthesized by the generator) and real fedge, photog tuples. The generator, G , learns to fool the discriminator. Unlike an unconditional GAN, both the generator and discriminator observe the input edge map.

3.1 Objective

The objective of a conditional GAN can be expressed as。

$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x,y}[\log D(x, y)] + \mathbb{E}_{x,z}[\log(1 - D(x, G(x, z)))] \quad (1)$$

where G tries to minimize this objective against an adversarial D that tries to maximize it, i.e. $G^* = \arg \min_G \max_D \mathcal{L}_{cGAN}(G, D)$.

To test the importance of conditioning the discriminator, we also compare to an unconditional variant in which the discriminator does not observe x :

$$\mathcal{L}_{GAN}(G, D) = \mathbb{E}_y[\log D(y)] + \mathbb{E}_{x,z}[\log(1 - D(G(x, z)))] \quad (2)$$

Previous approaches have found it beneficial to mix the GAN objective with a more traditional loss, such as L2 distance [43]. The discriminator’s job remains unchanged, but the generator is tasked to not only fool the discriminator but also to be near the ground truth output in an L2 sense. We also explore this option, using L1 distance rather than L2 as L1 encourages less blurring:

$$\mathcal{L}_{L1}(G) = \mathbb{E}_{x,y,z} [\|y - G(x, z)\|_1] \quad (3)$$

Our final objective is

$$G^* = \arg \min_G \max_D \mathcal{L}_{cGAN}(G, D) + \lambda \mathcal{L}_{L1}(G) \quad (4)$$

Without z , the net could still learn a mapping from x to y , but would produce deterministic outputs, and therefore fail to match any distribution other than a delta function. Past conditional GANs have acknowledged this and provided Gaussian noise z as an input to the generator, in addition to x (e.g., [55]). In initial experiments, we did not find this strategy effective the generator simply learned to ignore the noise – which is consistent with Mathieu et al. [40]. Instead, for our final models, we provide noise only in the form of dropout, applied on several layers of our generator at both training and test time. Despite the dropout noise, we observe only minor stochasticity in the output of our nets. Designing conditional GANs that produce highly stochastic output, and thereby capture the full entropy of the conditional distributions they model, is an important question left open by the present work.

3.2 Network architectures

We adapt our generator and discriminator architectures from those in [44]. Both generator and discriminator use modules of the form convolution-BatchNorm-ReLu [29]. Details of the architecture are provided in the supplemental materials online, with key features discussed below.

3.2.1 Generator with skips

A defining feature of image-to-image translation problems is that they map a high resolution input grid to a high resolution output grid. In addition, for the problems we consider, the input and output differ in surface appearance, but both are renderings of the

same underlying structure. Therefore, structure in the input is roughly aligned with structure in the output. We design the generator architecture around these considerations.

Many previous solutions [43, 55, 30, 64, 59] to problems in this area have used an encoder-decoder network [26]. In such a network, the input is passed through a series of layers that progressively downsample, until a bottleneck layer, at which point the process is reversed. Such a network requires that all information flow pass through all the layers, including the bottleneck. For many image translation problems, there is a great deal of low-level information shared between the input and output, and it would be desirable to shuttle this information directly across the net. For example, in the case of image colorization, the input and output share the location of prominent edges.

To give the generator a means to circumvent the bottleneck for information like this, we add skip connections, following the general shape of a “U-Net” [50]. Specifically, we add skip connections between each layer i and layer $n-i$, where n is the total number of layers. Each skip connection simply concatenates all channels at layer i with those at layer $n-i$.

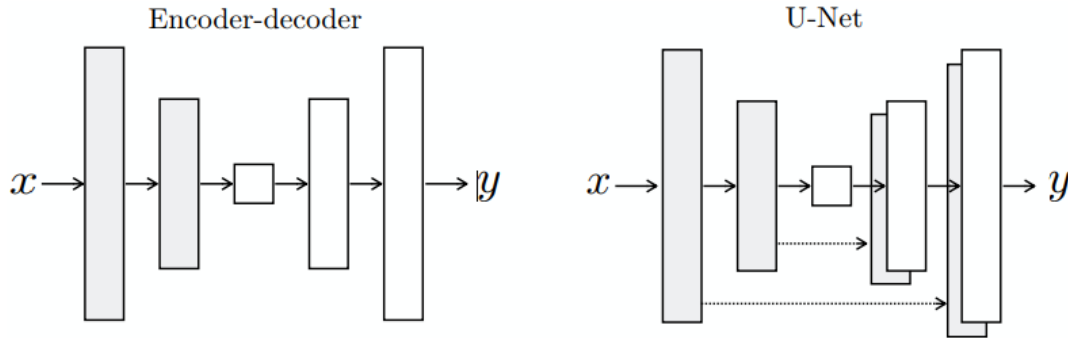


Figure 3.2: Two choices for the architecture of the generator. The “U-Net” [50] is an encoder-decoder with skip connections between mirrored layers in the encoder and decoder stacks

3.2.2 Markovian discriminator (PatchGAN)

It is well known that the L2 loss – and L1, see Figure 4 – produces blurry results on image generation problems [34]. Although these losses fail to encourage high frequency crispness, in many cases they nonetheless accurately capture the low frequencies. For problems where this is the case, we do not need an entirely new framework to enforce correctness at the low frequencies. L1 will already do.

This motivates restricting the GAN discriminator to only model high-frequency structure, relying on an L1 term to force low-frequency correctness (Eqn. 4). In order to

model high-frequencies, it is sufficient to restrict our attention to the structure in local image patches. Therefore, we design a discriminator architecture – which we term a PatchGAN – that only penalizes structure at the scale of patches. This discriminator tries to classify if each $N \times N$ patch in an image is real or fake. We run this discriminator convolutionally across the image, averaging all responses to provide the ultimate output of D .

In Section 4.4, we demonstrate that N can be much smaller than the full size of the image and still produce high quality results. This is advantageous because a smaller PatchGAN has fewer parameters, runs faster, and can be applied to arbitrarily large images.

Such a discriminator effectively models the image as a Markov random field, assuming independence between pixels separated by more than a patch diameter. This connection was previously explored in [38], and is also the common assumption in models of texture [17, 21] and style [16, 25, 22, 37]. Therefore, our PatchGAN can be understood as a form of texture/style loss.

3.3 Optimization and inference

To optimize our networks, we follow the standard approach from [24]: we alternate between one gradient descent step on D , then one step on G . As suggested in the original GAN paper, rather than training G to minimize $\log(1 - D(x, G(x, z)))$, we instead train to maximize $\log D(x, G(x, z))$ [24]. In addition, we divide the objective by 2 while optimizing D , which slows down the rate at which D learns relative to G . We use minibatch SGD and apply the Adam solver [32], with a learning rate of 0.0002, and momentum parameters $\beta_1 = 0.5, \beta_2 = 0.999$.

At inference time, we run the generator net in exactly the same manner as during the training phase. This differs from the usual protocol in that we apply dropout at test time, and we apply batch normalization [29] using the statistics of the test batch, rather than aggregated statistics of the training batch. This approach to batch normalization, when the batch size is set to 1, has been termed “instance normalization” and has been demonstrated to be effective at image generation tasks [54]. In our experiments, we use batch sizes between 1 and 10 depending on the experiment.

4. EXPERIMENTS

To explore the generality of conditional GANs, we test the method on a variety of tasks and datasets, including both graphics tasks, like photo generation, and vision tasks, like semantic segmentation:

- Semantic labels photo, trained on the Cityscapes dataset [12].
- Architectural labels photo, trained on CMP Facades [45].
- Map aerial photo, trained on data scraped from Google Maps.
- BW color photos, trained on [51].
- Edges photo, trained on data from [65] and [60]; binary edges generated using the HED edge detector [58] plus postprocessing.
- Sketch photo: tests edges photo models on human-drawn sketches from [19].
- Day night, trained on [33].
- Thermal color photos, trained on data from [27].
- Photo with missing pixels inpainted photo, trained on Paris StreetView from [14].

Details of training on each of these datasets are provided in the supplemental materials online. In all cases, the input and output are simply 1-3 channel images. Qualitative results are shown in Figures 8, 9, 11, 10, 13, 14, 15, 16, 17, 18, 19, 20. Several failure cases are highlighted in Figure 21. More comprehensive results are available at <https://phillipi.github.io/pix2pix/>.

Data requirements and speed We note that decent results can often be obtained even on small datasets. Our facade training set consists of just 400 images (see results in Figure 14), and the day to night training set consists of only 91 unique webcams (see results in Figure 15). On datasets of this size, training can be very fast: for example, the results shown in Figure 14 took less than two hours of training on a single Pascal Titan X GPU. At test time, all models run in well under a second on this GPU.

4.1 Evaluation metrics

Evaluating the quality of synthesized images is an open and difficult problem [52]. Traditional metrics such as per-pixel mean-squared error do not assess joint statistics of

the result, and therefore do not measure the very structure that structured losses aim to capture.

To more holistically evaluate the visual quality of our results, we employ two tactics. First, we run “real vs. fake” perceptual studies on Amazon Mechanical Turk (AMT). For graphics problems like colorization and photo generation, plausibility to a human observer is often the ultimate goal. Therefore, we test our map generation, aerial photo generation, and image colorization using this approach.

Second, we measure whether or not our synthesized cityscapes are realistic enough that off-the-shelf recognition system can recognize the objects in them. This metric is similar to the “inception score” from [52], the object detection evaluation in [55], and the “semantic interpretability” measures in [62] and [42].

4.1.1 AMT perceptual studies

For our AMT experiments, we followed the protocol from [62]: Turkers were presented with a series of trials that pitted a “real” image against a “fake” image generated by our algorithm. On each trial, each image appeared for 1 second, after which the images disappeared and Turkers were given unlimited time to respond as to which was fake. The first 10 images of each session were practice and Turkers were given feedback. No feedback was provided on the 40 trials of the main experiment. Each session tested just one algorithm at a time, and Turkers were not allowed to complete more than one session. 50 Turkers evaluated each algorithm. Unlike [62], we did not include vigilance trials. For our colorization experiments, the real and fake images were generated from the same grayscale input. For map aerial photo, the real and fake images were not generated from the same input, in order to make the task more difficult and avoid floor-level results. For map↔aerial photo, we trained on 256×256 resolution images, but exploited fully-convolutional translation (described above) to test on 512×512 images, which were then downsampled and presented to Turkers at 256×256 resolution. For colorization, we trained and tested on 256×256 resolution images and presented the results to Turkers at this same resolution.

4.1.2 FCN-score

While quantitative evaluation of generative models is known to be challenging, recent works [52, 55, 62, 42] have tried using pre-trained semantic classifiers to measure

the discriminability of the generated stimuli as a pseudo-metric. The intuition is that if the generated images are realistic, classifiers trained on real images will be able to classify the synthesized image correctly as well. To this end, we adopt the popular FCN-8s [39] architecture for semantic segmentation, and train it on the cityscapes dataset. We then score synthesized photos by the classification accuracy against the labels these photos were synthesized from.

4.2 Analysis of the objective function

Which components of the objective in Eqn. 4 are important? We run ablation studies to isolate the effect of the L1 term, the GAN term, and to compare using a discriminator conditioned on the input (cGAN, Eqn. 1) against using an unconditional discriminator (GAN, Eqn. 2).

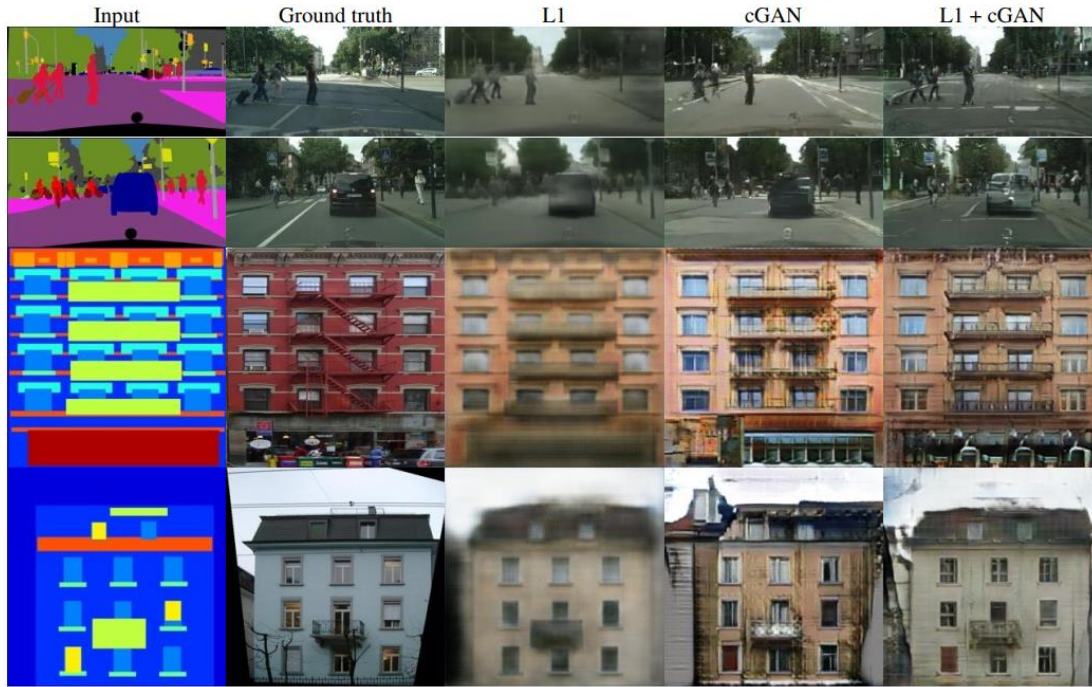


Figure 4.1: Different losses induce different quality of results. Each column shows results trained under a different loss. Please see

<https://phillipi.github.io/pix2pix/> for additional examples. Figure 4 shows the qualitative effects of these variations on two labels photo problems. L1 alone leads to reasonable but blurry results. The cGAN alone (setting $\lambda = 0$ in Eqn. 4) gives much sharper results but introduces visual artifacts on certain applications. Adding both terms together (with $\lambda = 100$) reduces these artifacts.

We quantify these observations using the FCN-score on the cityscapes labels photo task (Table 1): the GAN-based objectives achieve higher scores, indicating that the synthesized images include more recognizable structure. We also test the effect of removing conditioning from the discriminator (labeled as GAN). In this case, the loss does not penalize mismatch between the input and output; it only cares that the output look realistic. This variant results in poor performance; examining the results reveals that the generator collapsed into producing nearly the exact same output regardless of input photograph. Clearly, it is important, in this case, that the loss measure the quality of the match between input and output, and indeed cGAN performs much better than GAN. Note, however, that adding an L1 term also encourages that the output respect the input, since the L1 loss penalizes the distance between ground truth outputs, which correctly match the input, and synthesized outputs, which may not. Correspondingly, L1+GAN is also effective at creating realistic renderings that respect the input label maps. Combining all terms, L1+cGAN, performs similarly well.

Table 1: FCN-scores for different losses, evaluated on Cityscapes labels\$photos.

Loss	Per-pixel acc.	Per-class acc.	Class IOU
L1	0.42	0.15	0.11
GAN	0.22	0.05	0.01
cGAN	0.57	0.22	0.16
L1+GAN	0.64	0.20	0.15
L1+cGAN	0.66	0.23	0.17
Ground truth	0.80	0.26	0.21

Colorfulness A striking effect of conditional GANs is that they produce sharp images, hallucinating spatial structure even where it does not exist in the input label map. One might imagine cGANs have a similar effect on “sharpening” in the spectral dimension – i.e. making images more colorful. Just as L1 will incentivize a blur when it is uncertain where exactly to locate an edge, it will also incentivize an average, grayish color when it is uncertain which of several plausible color values a pixel should take on. Specially, L1 will be minimized by choosing the median of the conditional probability density function over possible colors. An adversarial loss, on the other hand, can in principle become aware that grayish outputs are unrealistic, and encourage matching the true color distribution [24]. In Figure 7, we investigate whether our cGANs actually achieve this effect on the Cityscapes dataset. The plots show the marginal distributions over output color values in Lab color space. The ground truth distributions are shown with a dotted line. It is apparent that L1 leads to a narrower distribution than the ground truth,

confirming the hypothesis that L1 encourages average, grayish colors. Using a cGAN, on the other hand, pushes the output distribution closer to the ground truth.

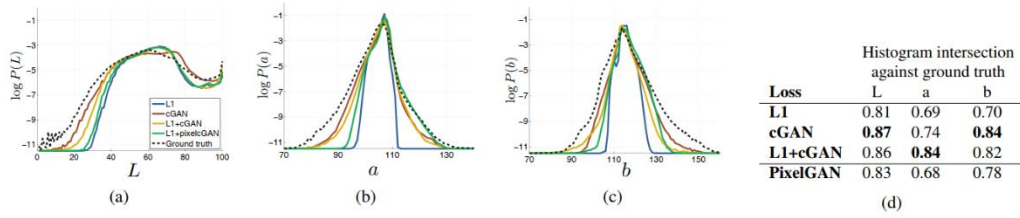


Figure 4.2: Color distribution matching property of the cGAN, tested on Cityscapes. (c.f. Figure 1 of the original GAN paper [24]). Note that the histogram intersection scores are dominated by differences in the high probability region, which are imperceptible in the plots, which show log probability and therefore emphasize differences in the low probability regions

4.3 Analysis of the generator architecture

A U-Net architecture allows low-level information to shortcut across the network. Does this lead to better results? Figure 5 and Table 2 compare the U-Net against an encoder-decoder on cityscape generation. The encoder-decoder is created simply by severing the skip connections in the U-Net. The encoder-decoder is unable to learn to generate realistic images in our experiments. The advantages of the U-Net appear not to be specific to conditional GANs: when both U-Net and encoder-decoder are trained with an L1 loss, the U-Net again achieves the superior results.



Figure 4.2: Adding skip connections to an encoder-decoder to create a “U-Net” results in much higher quality results.

Table 2: FCN-scores for different generator architectures (and objectives), evaluated on Cityscapes labels\$photos. (U-net (L1-cGAN) scores differ from those reported in other tables since batchsize was 10 for this experiment and 1 for other tables, and randomvariation between training runs.)

Loss	Per-pixel acc.	Per-class acc.	Class IOU
Encoder-decoder (L1)	0.35	0.12	0.08
Encoder-decoder (L1+cGAN)	0.29	0.09	0.05
U-net (L1)	0.48	0.18	0.13
U-net (L1+cGAN)	0.55	0.20	0.14

4.4 From PixelGANs to PatchGANs to ImageGANs

We test the effect of varying the patch size N of our discriminator receptive fields, from a 1×1 “PixelGAN” to a full 286×286 “ImageGAN”¹. Figure 6 shows qualitative results of this analysis and Table 3 quantifies the effects using the FCN-score. Note that elsewhere in this paper, unless specified, all experiments use 70×70 PatchGANs, and for this section all experiments use an L1+cGAN loss.

Table 3: FCN-scores for different receptive field sizes of the discriminator, evaluated on Cityscapes labels!photos. Note that input images are 256×256 pixels and larger receptive fields are padded with zeros

Discriminator receptive field	Per-pixel acc.	Per-class acc.	Class IOU
1×1	0.39	0.15	0.10
16×16	0.65	0.21	0.17
70×70	0.66	0.23	0.17
286×286	0.42	0.16	0.11

The PixelGAN has no effect on spatial sharpness but does increase the colorfulness of the results (quantified in Figure 7). For example, the bus in Figure 6 is painted gray when the net is trained with an L1 loss, but becomes red with the PixelGAN loss. Color histogram matching is a common problem in image processing [49], and PixelGANs may be a promising lightweight solution.



Figure 6: Patch size variations. Uncertainty in the output manifests itself differently for different loss functions. Uncertain regions become blurry and desaturated under L1. The 1×1 PixelGAN encourages greater color diversity but has no effect on spatial statistics. The 16×16 PatchGAN creates locally sharp results, but also leads to tiling artifacts beyond the scale it can observe. The 70×70 PatchGAN forces outputs that are sharp, even if incorrect, in both the spatial and spectral (colorfulness) dimensions. The full 286×286 ImageGAN produces results that are visually similar to the 70×70

PatchGAN, but somewhat lower quality according to our FCN-score metric (Table 3). Please see <https://phillipi.github.io/pix2pix/> for additional examples

Using a 16×16 PatchGAN is sufficient to promote sharp outputs, and achieves good FCN-scores, but also leads to tiling artifacts. The 70×70 PatchGAN alleviates these artifacts and achieves slightly better scores. Scaling beyond this, to the full 286×286 ImageGAN, does not appear to improve the visual quality of the results, and in fact gets a considerably lower FCN-score (Table 3). This may be because the ImageGAN has many more parameters and greater depth than the 70×70 PatchGAN, and may be harder to train.

Fully-convolutional translation An advantage of the PatchGAN is that a fixed-size patch discriminator can be applied to arbitrarily large images. We may also apply the generator convolutionally, on larger images than those on which it was trained. We test this on the map aerial photo task. After training a generator on 256×256 images, we test it on 512×512 images. The results in Figure 8 demonstrate the effectiveness of this approach.



Figure 8: Example results on Google Maps at 512×512 resolution (model was trained on images at 256×256 resolution, and run convolutionally on the larger images at test time). Contrast adjusted for clarity.

4.5 Perceptual validation

We validate the perceptual realism of our results on the tasks of map aerial photograph and grayscale color. Results of our AMT experiment for map photo are given in Table 4. The aerial photos generated by our method fooled participants on 18.9%

of trials, significantly above the L1 baseline, which produces blurry results and nearly never fooled participants. In contrast, in the photo map direction our method only fooled participants on 6.1% of trials, and this was not significantly different than the performance of the L1 baseline (based on bootstrap test). This may be because minor structural errors are more visible in maps, which have rigid geometry, than in aerial photographs, which are more chaotic.

Table 4: AMT “real vs fake” test on maps-aerial photos

Loss	Photo $\rightarrow$ Map	Map $\rightarrow$ Photo
	% Turkers labeled <i>real</i>	% Turkers labeled <i>real</i>
L1	2.8% $\pm$ 1.0%	0.8% $\pm$ 0.3%
L1+cGAN	6.1% $\pm$ 1.3%	18.9% $\pm$ 2.5%

We trained colorization on ImageNet [51], and tested on the test split introduced by [62, 35]. Our method, with L1+cGAN loss, fooled participants on 22.5% of trials (Table 5). We also tested the results of [62] and a variant of their method that used an L2 loss (see [62] for details). The conditional GAN scored similarly to the L2 variant of [62] (difference insignificant by bootstrap test), but fell short of [62]’s full method, which fooled participants on 27.8% of trials in our experiment. We note that their method was specifically engineered to do well on colorization.

Table 5: AMT “real vs fake” test on colorization.

Method	% Turkers labeled <i>real</i>
L2 regression from [62]	16.3% $\pm$ 2.4%
Zhang et al. 2016 [62]	27.8% $\pm$ 2.7%
Ours	22.5% $\pm$ 1.6%

4.6 Semantic segmentation

Conditional GANs appear to be effective on problems where the output is highly detailed or photographic, as is common in image processing and graphics tasks. What about vision problems, like semantic segmentation, where the output is instead less complex than the input?

Table 6: Performance of photo!labels on cityscapes.

Loss	Per-pixel acc.	Per-class acc.	Class IOU
L1	0.86	0.42	0.35
cGAN	0.74	0.28	0.22
L1+cGAN	0.83	0.36	0.29

To begin to test this, we train a cGAN (with/without L1 loss) on cityscape photo labels. Figure 10 shows qualitative results, and quantitative classification accuracies are reported in Table 6. Interestingly, cGANs, trained without the L1 loss, are able to solve this problem at a reasonable degree of accuracy. To our knowledge, this is the first demonstration of GANs successfully generating “labels”, which are nearly discrete, rather than “images”, with their continuous-valued variation². Although cGANs achieve some success, they are far from the best available method for solving this problem: simply using L1 regression gets better scores than using a cGAN, as shown in Table 6. We argue that for vision problems, the goal (i.e. predicting output close to the ground truth) may be less ambiguous than graphics tasks, and reconstruction losses like L1 are mostly sufficient.

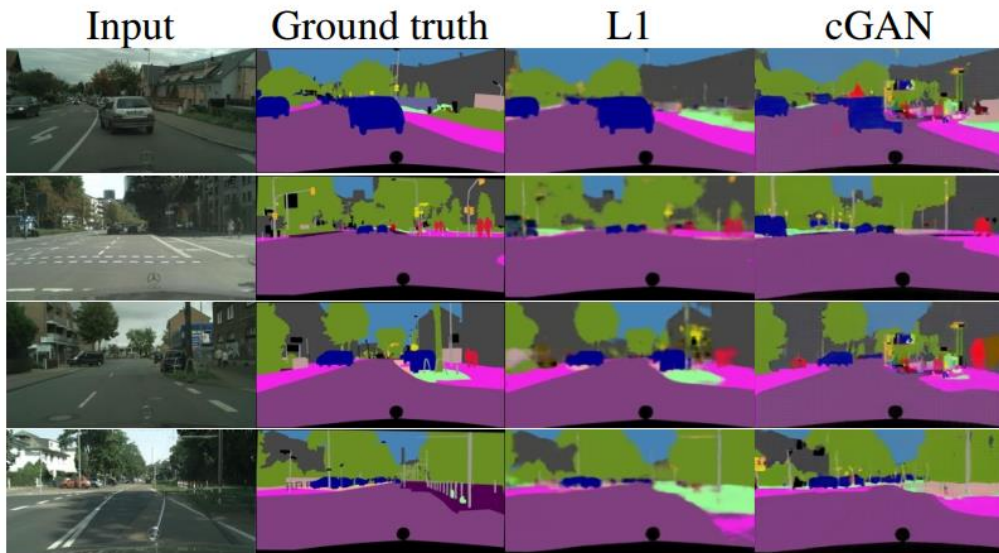


Figure 10: Applying a conditional GAN to semantic segmentation. The cGAN produces sharp images that look at glance like the ground truth, but in fact include many small, hallucinated objects.

4.7 Community-driven Research

Since the initial release of the paper and our pix2pix codebase, the Twitter community, including computer vision and graphics practitioners as well as visual artists, have successfully applied our framework to a variety of novel image-to-image translation tasks, far beyond the scope of the original paper. Figure 11 and Figure 12 show just a few examples from the #pix2pix hashtag, including Background removal, Palette generation, Sketch Portrait, Sketch Pokemon, “Do as I Do” pose transfer, Learning to see: Gloomy Sunday, as well as the bizarrely popular #edges2cats and #fotogenerator. Note that these

applications are creative projects, were not obtained in controlled, scientific conditions, and may rely on some modifications to the pix2pix code we released. Nonetheless, they demonstrate the promise of our approach as a generic commodity tool for image-to-image translation problems.

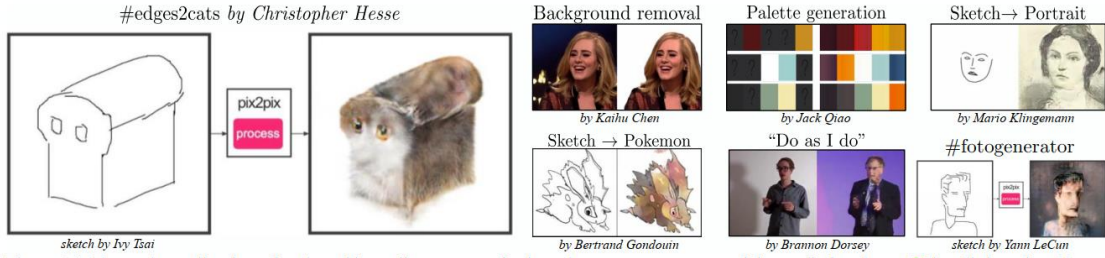


Figure 11: Example applications developed by online community based on our pix2pix codebase: #edges2cats [3] by Christopher Hesse, Background removal [6] by Kaihu Chen, Palette generation [5] by Jack Qiao, Sketch ! Portrait [7] by Mario Klingemann, Sketch!Pokemon [1] by Bertrand Gondouin, “Do As I Do” pose transfer [2] by Brannon Dorsey, and #fotogenerator by Bosman et al. [4].



Figure 12: Learning to see: Gloomy Sunday: An interactive artistic demo developed by Memo Akten[8] based on our pix2pixcodebase. Please click the image to play the video in a browser.

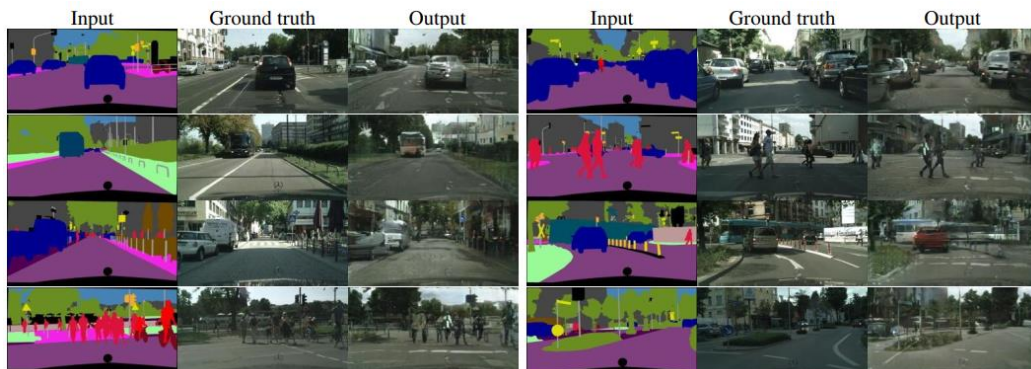


Figure 13: Example results of our method on Cityscapes labels!photo, compared to ground truth.

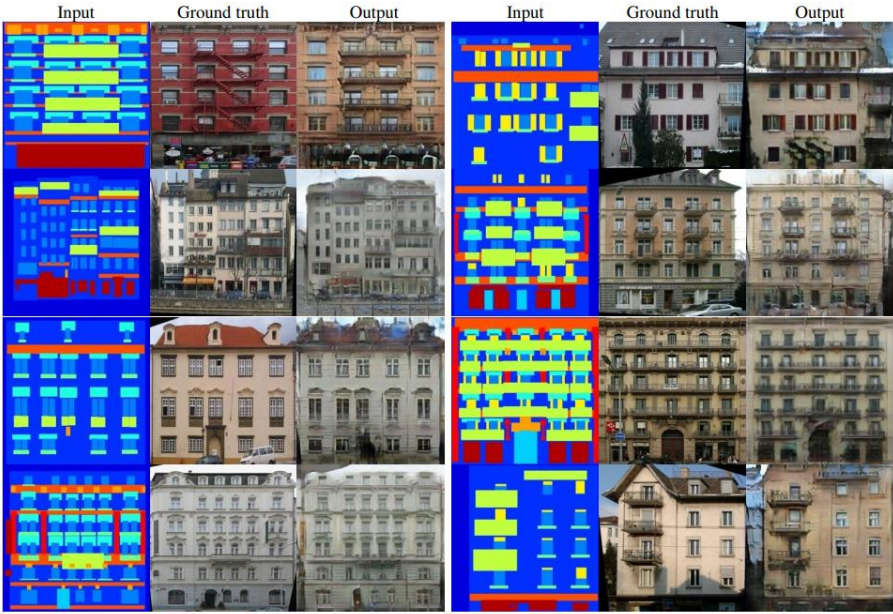


Figure 14: Example results of our method on facades labels!photo, compared to ground truth.

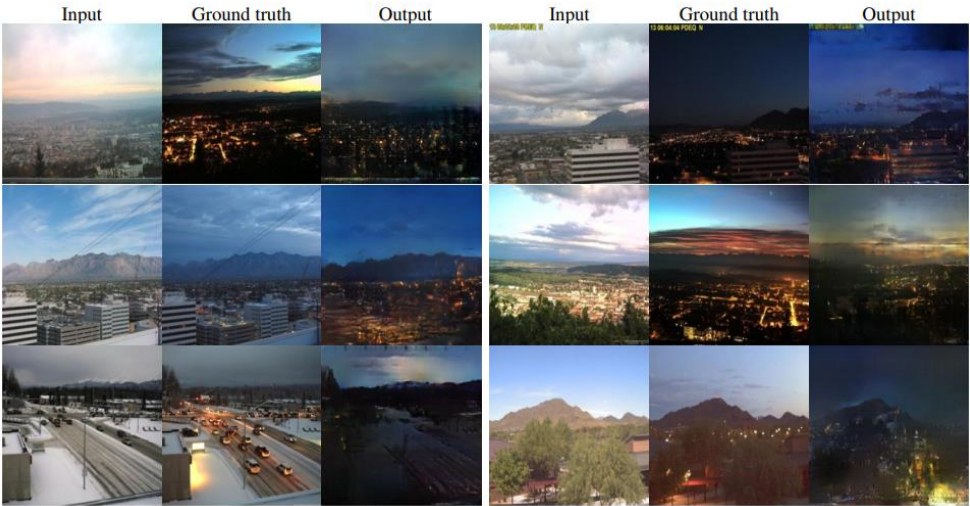


Figure 15: Example results of our method on day!night, compared to ground truth



Figure 16: Example results of our method on automatically detected edges!handbags, compared to ground truth.



Figure 17: Example results of our method on automatically detected edges!shoes, compared to ground truth.



Figure 18: Additional results of the edges!photo models applied to human-drawn sketches from [19]. Note that the models were trained on automatically detected edges, but generalize to human drawings

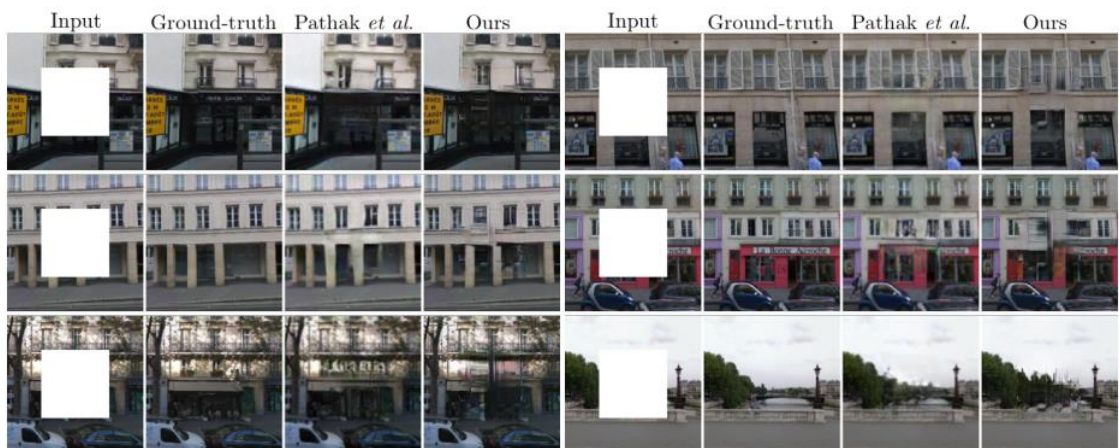


Figure 19: Example results on photo inpainting, compared to [43], on the Paris StreetView dataset [14]. This experiment demonstrates that the U-net architecture can be effective even when the predicted pixels are not geometrically aligned with the information in the input – the information used to fill in the central hole has to be found in the periphery of these photos.



Figure 20: Example results on translating thermal images to RGB photos, on the dataset from [27].

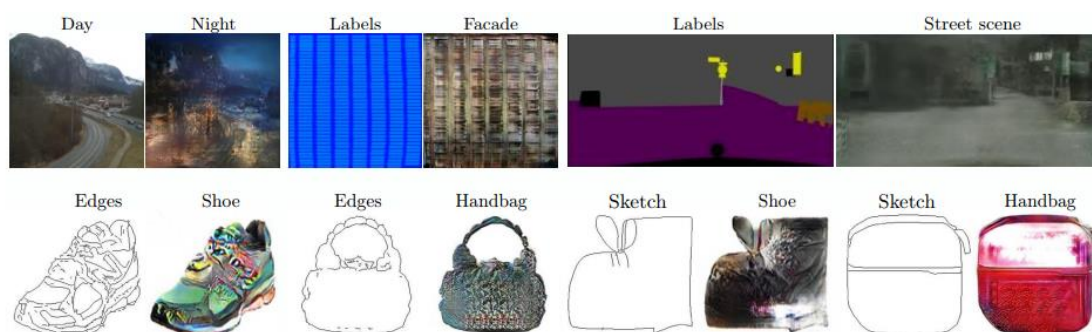


Figure 21: Example failure cases. Each pair of images shows input on the left and output on the right. These examples are selected as some of the worst results on our tasks. Common failures include artifacts in regions where the input image is sparse, and difficulty in handling unusual inputs. Please see <https://phillipi.github.io/pix2pix/> for more comprehensive results.

5. CONCLUSION

The results in this paper suggest that conditional adversarial networks are a promising approach for many image-to-image translation tasks, especially those involving highly structured graphical outputs. These networks learn a loss adapted to the task and data at hand, which makes them applicable in a wide variety of settings.

Acknowledgments: We thank Richard Zhang, Deepak Pathak, and Shubham Tulsiani for helpful discussions, Saining Xie for help with the HED edge detector, and the online community for exploring many applications and suggesting improvements. Thanks to Christopher Hesse, Memo Atten, Kaihu Chen, Jack Qiao, Mario Klingemann, Brannon Dorsey, Gerda Bosman, Ivy Tsai, and Yann LeCun for allowing the use of their creations in Figure 11 and Figure 12. This work was supported in part by NSF SMA-1514512, NGA NURI, IARPA via Air Force Research Laboratory, Intel Corp, Berkeley Deep Drive, and hardware donations by Nvidia. J.-Y.Z. is supported by the Facebook Graduate Fellowship. Disclaimer: The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of IARPA, AFRL or the U.S. Government.

译文

基于条件对抗网络的图像到图像翻译

摘要

本文探讨了基于条件对抗网络作为图像到图像翻译问题的通用解决方案。这些网络不仅学习从输入图像到输出图像的映射,还学习了一个损失函数来训练这个映射。这使得我们可以将同一种通用方法应用于传统上需要非常不同的损失函数的问题。我们展示了这种方法在从标签映射中合成照片、从边缘映射中重构物体和图像着色等任务上的有效性。事实上,自本文发布与 pix2pix 软件相关联后,许多互联网用户(其中许多是艺术家)已经发布了自己对我们系统的实验,进一步证明了它的广泛适用性和易于采用的特点,无需参数调整。作为一个社区,我们不再手工设计映射函数,而这项工作表明,我们可以在不手工设计损失函数的情况下获得合理的结果。

第 1 章 引言

图像处理、计算机图形学和计算机视觉中的许多问题可以被描述为将输入图像转换为相应的输出图像。就像一个概念可以用英语或法语来表达一样,一个场景可以被渲染成 RGB 图像、梯度场、边缘图或语义标签图等形式。类比于自动语言翻译,我们定义自动图像到图像的翻译为将一个场景的可能表示翻译为另一个,只要有足够的训练数据(见图 1)。传统上,每个任务都是用单独的、特殊目的的机器来处理的,尽管情境总是相同的:从像素预测像素。本文的目标是为所有这些问题开发一个通用框架。

社区已经采取了重要的步骤,卷积神经网络(CNNs)成为各种图像预测问题的常见工具。CNNs 学习最小化损失函数-评估结果质量的目标,虽然学习过程是自动的,但仍需要大量手动工作来设计有效的损失函数。换句话说,我们仍然需要告诉 CNN 我们希望它最小化什么。但是,就像国王米达斯一样,我们必须小心我们的愿望!如果我们采用天真的方法,要求 CNN 最小化预测像素和真实像素之间的欧氏距

离,它将倾向于产生模糊的结果。这是因为欧氏距离通过对所有合理的输出求平均值来最小化,导致模糊。制定能够强制 CNN 执行我们真正想要的任务的损失函数(例如输出清晰、逼真的图像)是一个开放问题,通常需要专业知识。

如果我们只能指定一个高层次的目标,比如“使输出与现实无法区分”,然后自动学习一个适合满足这个目标的损失函数,这将是理想的。幸运的是,这正是最近提出的 Generative Adversarial 网络所做的。GAN 学习一个损失,试图分类输出图像是否真实或虚假,同时训练一个生成模型以最大限度地减少这种损失。模糊的图像不会被容忍,因为它们显然看起来是假的。由于 GAN 学习了适应数据的损失,所以它们可以应用于传统上需要非常不同的损失函数的众多任务。在本文中,我们探讨了条件设置中的 GAN。正如 GAN 学习数据的生成模型一样,有条件的 GAN(cGAN)学习有条件生成的模型。这使得 cGAN 适用于图像对图像的翻译任务,我们在输入图像上调节并生成相应的输出图像。

在过去的两年中,GANs 已经受到了大力研究,我们在本文中探讨的许多技术之前已经被提出过。尽管如此,早期的论文主要集中在特定应用上,图像条件 GANs 作为图像到图像翻译的通用解决方案的有效性仍然不清楚。我们的主要贡献是证明在各种各样的问题上,条件 GANs 可以产生合理的结果。我们的第二个贡献是提出一个简单的框架,足以实现良好的结果,并分析几个重要架构选择的影响。代码可通过该连接获取: <https://github.com/phillipi/pix2pix>.

第 2 章 相关工作

2.1 图像建模中的结构损失

图像到图像的翻译问题通常被表述为每个像素的分类或回归问题。这种表述将输出空间视为“非结构化”,也就是说,考虑到输入图像,每个输出像素被认为是条件独立的。相比之下,条件生成对抗网络(conditional GANs)学习的是结构化损失。结构化损失惩罚输出的联合配置。已有大量文献考虑了这种类型的损失,包括条件随机场、结构相似度度量(SSIM metric)、特征匹配、非参数损失、卷积伪先验(convolutional pseudo-prior)和基于协方差统计量的损失等方法。条件生成对抗网络与众不同之处在于,它学习的损失是可以学习的,并且在理论上可以惩罚任何可能导致输出和目标之间差异的结构。

2.2 条件 GANs

条件生成对抗网络(Conditional GANs)在有条件的情况下应用并不是第一次。之前的工作已经将 GANs 应用于离散标签、文本以及图像的条件设定。图像条件模型已经解决了从法线图预测图像、未来帧预测、产品照片生成以及从稀疏注释中生成图像等问题。其他几篇论文也使用了 GANs 进行图像到图像的映射,但仅无条件地应用了 GANs,依靠其他项(如 L2 回归)来强制输出受到输入的限制。这些论文在修复缺失部分的图像、预测未来状态、受用户约束进行图像操作、风格转移和超分辨率等方面取得了令人印象深刻的成果。每种方法都是针对特定应用进行定制的。我们的框架不同之处在于它没有应用特定于某种应用的内容,这使得我们的设置比大多数其他方法要简单得多。

我们的方法还在生成器和鉴别器的几个架构选择上与之前的工作不同。与过去的工作不同,我们的生成器采用基于“U-Net”的架构,而我们的鉴别器则采用卷积“PatchGAN”分类器,它只惩罚图像块大小的结构。之前的文献中提出了类似的 PatchGAN 架构,以捕捉本地风格统计信息。在这里,我们展示了这种方法在更广泛的问题上的有效性,并研究了改变图像块大小的影响。

第 3 章 方法

GANs 是生成模型,它们学习从随机噪声向量 z 到输出图像 y 的映射,即 $G: z \rightarrow y$ 。相比之下,条件 GAN 学习从观察到的图像 x 和随机噪声向量 z 到 y 的映射,即 $G: \{x, z\} \rightarrow y$ 。生成器 G 被训练以产生输出,这些输出无法被一个经过对抗训练的鉴别器 D 区分出来。鉴别器 D 被训练以尽可能地检测生成器的“伪造品”。这个训练过程在图 2 中被示意出来。

3.1 目标函数

一个条件 GAN 的目标可以表示为:

$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x,y}[\log D(x, y)] + \mathbb{E}_{x,z}[\log(1 - D(x, G(x, z)))] \quad (1)$$

其中, G 试图最小化这个目标函数,而对抗性的 D 试图最大化它,即 $G^* = \arg \min_G \max_D \mathcal{L}_{cGAN}(G, D)$

为了测试条件判别器的重要性,我们还将其与一个无条件变体进行比较,其中判别器不观察输入 x :

$$\mathcal{L}_{GAN}(G, D) = \mathbb{E}_y[\log D(y)] + \mathbb{E}_{x,z}[\log(1 - D(G(x, z)))] \quad (2)$$

先前的方法发现将 GAN 目标函数与传统的损失(如 L2 距离)混合在一起是有益的。判别器的工作保持不变,但生成器的任务不仅是欺骗判别器,还要在 L2 意义下接近真实输出。我们也探讨了 this 选项,使用 L1 距离代替 L2,因为 L1 会更少模糊:

$$\mathcal{L}_{L1}(G) = \mathbb{E}_{x,y,z}[\|y - G(x, z)\|_1] \quad (3)$$

我们最终的目标函数是:

$$G^* = \arg \min_G \max_D \mathcal{L}_{cGAN}(G, D) + \lambda \mathcal{L}_{L1}(G) \quad (4)$$

没有 z , 网络仍然可以学习从 x 到 y 的映射,但会产生确定性输出,因此无法匹配除 Delta 函数以外的任何分布。过去的条件 GAN 已经承认了这一点,并在生成器中提供了高斯噪声 z 作为输入,除了 x 之外。在初始实验中,我们发现这种策略并不有

效-生成器只是学会忽略噪声-这与 Mathieu 等人的发现一致。相反,对于我们的最终模型,我们只在生成器的几个层上应用 dropout,用于训练和测试时间。尽管使用了 dropout 噪声,我们观察到我们的网络输出仅有轻微的随机性。设计产生高度随机输出的条件 GAN,从而捕获它们建模的条件分布的全部熵,是当前工作中尚未解决的重要问题。

3.2 网络结构

我们的生成器和判别器架构是从中调整过来的。生成器和判别器都使用了卷积-批规范化-ReLu 模块。有关架构的详细信息可以在在线补充材料中找到,下面讨论关键特征:

3.2.1 使用跳级结构的生成器

图像到图像翻译问题的一个关键特征是将高分辨率的输入网格映射到高分辨率的输出网格。此外,在我们考虑的问题中,输入和输出在表面外观上存在差异,但两者都是同一基础结构的渲染结果。因此,输入中的结构大致与输出中的结构对齐。我们围绕这些考虑设计了生成器架构。

许多之前的解决方案使用编码器-解码器网络来解决此领域的问题。在这样的网络中,输入通过一系列逐渐降采样的层传递,直到瓶颈层,然后将其过程反转。这样的网络要求所有信息流通过所有层,包括瓶颈。对于许多图像翻译问题,输入和输出之间共享大量低级别的信息,直接穿越网络传输这些信息将是理想的。例如,在图像着色的情况下,输入和输出共享显著边缘的位置。

为了让生成器具备绕过瓶颈的手段来传递这种信息,我们添加了跳跃连接(skip connections),遵循“U-Net”的一般形状。具体而言,我们在第 i 层和第 $n-i$ 层之间添加跳跃连接,其中 n 是总层数。每个跳跃连接简单地将第 i 层的所有通道与第 $n-i$ 层的通道连接起来。

3.2.2 马尔科夫链的判别器

众所周知,L2 损失 (L1 也一样,参见图 4) 在图像生成问题中会产生模糊的结果。虽然这些损失不能促进高频率的锐度,但在许多情况下它们仍然能够准确地捕

捉低频。对于这种情况,我们并不需要完全新的框架来强制执行低频的正确性,L1 就足够了。

这种情况促使我们将 GAN 鉴别器限制为仅模拟高频结构,依赖于 L1 项来强制实现低频正确性(方程式 4)。为了模拟高频率,我们只需将注意力限制在局部图像块中的结构上即可。因此,我们设计了一个鉴别器架构,称为 PatchGAN,它仅惩罚大小为块的结构。这个鉴别器试图分类图像中每个 $N \times N$ 块是真实的还是假的。我们在图像上卷积地运行此鉴别器,平均所有响应以提供 D 的最终输出。

在第 4.4 节中,我们证明 N 可以比图像的完整尺寸小得多,但仍能产生高质量的结果。这是有利的,因为较小的 PatchGAN 具有较少的参数,运行速度更快,并且可以应用于任意大小的图像。

这样的判别器有效地将图像建模为马尔可夫随机场,假设相隔一个补丁直径的像素之间是独立的。这种联系先前在中被探讨过,并且也是纹理和风格模型的常见假设。因此,我们的 PatchGAN 可以被理解为一种纹理/风格损失。

3.3 优化和推理

为了优化我们的网络,我们遵循[24]中的标准方法:在 D 上进行一次梯度下降步骤,然后在 G 上进行一次梯度下降步骤。正如原始的 GAN 论文所建议的那样,我们训练 G 以最大化 $\log D(x, G(x, z))$,而不是训练 G 以最小化 $\log(1 - D(x, G(x, z)))$ [24]。此外,我们在优化 D 时将目标函数除以 2,这可以减缓 D 相对于 G 学习的速度。我们使用小批量随机梯度下降(minibatch SGD),并应用 Adam 求解器[32],学习率为 0.0002,动量参数 $\beta_1 = 0.5, \beta_2 = 0.999$ 。

在推理阶段,我们以与训练阶段完全相同的方式运行生成器网络。这与通常的协议不同,因为我们在测试时应用了 dropout,并且我们使用测试批次的统计信息,而不是训练批次的聚合统计信息来应用批归一。当批大小设置为 1 时,这种批归一化方法被称为“实例归一化”,已经被证明在图像生成任务中非常有效。在我们的实验中,我们根据实验的不同使用批次大小在 1 到 10 之间变化。

第 4 章 实验

为了探索条件 GAN 的普适性,我们在各种任务和数据集上测试了该方法,包括图形任务,如照片生成,以及视觉任务,如语义分割:

- 城市景观语义标签↔照片,使用 Cityscapes 数据集进行训练。
- 建筑标签↔照片,使用 CMP Facades 进行训练。
- 地图↔航拍照片,使用从 Google Maps 抓取的数据进行训练。
- 黑白照片↔彩色照片,使用文献的数据集进行训练。
- 边缘→照片,使用来自文献的数据进行训练;使用 HED 边缘检测器生成二进制边缘并进行后处理。

- 素描→照片: 使用来自文献的手绘素描测试边缘→照片模型。
- 日间照片→夜间照片,使用来自文献的数据进行训练。
- 热成像→彩色照片,使用来自文献的数据进行训练。
- 有缺失像素的照片→修复后的照片,使用巴黎街景数据进行训练。

有关在这些数据集上的训练细节可在在线补充材料中找到。在所有情况下,输入和输出均为 1-3 通道图像。定性结果显示在图 8、9、11、10、13、14、15、16、17、18、19、20 中。一些失败案例在图 21 中突出显示。更全面的结果可在 <https://phillipi.github.io/pix2pix/> 上获得。

数据需求和速度 我们注意到,即使在小数据集上,通常也可以获得不错的结果。我们的 Facade 训练集仅包含 400 张图像(见图 14 中的结果),而白天到夜晚的训练集仅包含 91 个唯一的网络摄像头(见图 15 中的结果)。在这样的数据集上,训练速度非常快:例如,图 14 中显示的结果仅需在单个 Pascal Titan X GPU 上进行不到两个小时的训练。在测试时间,所有模型在此 GPU 上运行时间都少于一秒。

4.1 评价指标

评估合成图像的质量是一个开放且困难的问题。传统的指标,如像素均方误差,不能评估结果的联合统计量,因此无法衡量结构化损失旨在捕捉的结构。为了更全面地评估我们结果的视觉质量,我们采用了两种策略。首先,我们在 Amazon

Mechanical Turk (AMT) 上进行“真实与虚假”感知研究。对于像着色和照片生成这样的图形问题,对人类观察者的可信度通常是最终目标。因此,我们使用这种方法测试我们的地图生成、航空照片生成和图像着色。

4.1.1 AMT 实验

在我们的 AMT 实验中,我们遵循了[62]的协议: Turkers 会面对一系列试验,比较我们算法生成的“真实”图像和“虚假”图像。在每个试验中,每个图像会出现 1 秒钟,然后图像消失,Turkers 会无限制地回答哪个是虚假的。每个会话的前 10 张图像是练习,并且 Turkers 会得到反馈。在主实验的 40 次试验中,没有提供反馈。每个会话一次只测试一个算法,Turkers 不允许完成多个会话。大约有 50 个 Turkers 评估了每个算法。与[62]不同的是,我们没有包括警觉性试验。对于我们的着色实验,真实和虚假图像是从同一灰度输入生成的。对于地图↔空中照片,真实和虚假图像不是从同一输入生成的,以使任务更加困难并避免底部结果。对于地图↔空中照片,我们在 256×256 分辨率的图像上进行训练,但利用全卷积翻译(如上所述)在 512×512 图像上进行测试,然后将其降采样并以 256×256 分辨率呈现给 Turkers。对于着色,我们在 256×256 分辨率的图像上进行训练和测试,并以相同的分辨率呈现结果给 Turkers。

4.1.2 FCN-score

尽管生成模型的定量评估被认为是具有挑战性的,但近期的研究[52,55,62,42]尝试使用预训练的语义分类器作为伪度量来测量生成刺激的可区分性。其基本思想是,如果生成的图像足够真实,那么在真实图像上训练的分类器也应该能够正确分类合成的图像。为此,我们采用了常用的 FCN-8s[39]架构进行语义分割,并在 Cityscapes 数据集上对其进行训练。然后,我们通过对合成的照片进行分类精度评分来衡量其真实度,该评分是根据合成照片所生成的标签进行的。

4.2 目标函数的分析

哪些是方程式 4 中目标函数的重要组成部分? 我们进行了消融研究,以隔离 L1 项、GAN 项的影响,并比较使用一个基于输入的鉴别器(条件 GAN,方程式 1)和使用一个无条件的鉴别器(GAN,方程式 2)的效果。

图 4 展示了这些变化对两个标签→照片问题的定性影响。仅使用 L1 会导致合理但模糊的结果。单独使用 cGAN（在公式 4 中设置 $\lambda = 0$ ）能够得到更清晰的结果,但在某些应用中会引入视觉伪影。将两个项结合在一起（ $\lambda = 100$ ）,能够减少这些伪影。

我们使用城市街景标签到照片任务上的 FCN 分数来量化这些观察结果(表 1): 基于 GAN 的目标函数实现了更高的分数,表明合成的图像包含更多可识别的结构。我们还测试了从鉴别器中删除条件的影响(标记为 GAN)。在这种情况下,损失不会惩罚输入和输出之间的不匹配,它只关心输出看起来是否真实。这种变体的性能很差;检查结果发现生成器崩溃成产生几乎完全相同的输出,而不管输入照片是什么。显然,在这种情况下,损失度量输入和输出之间的匹配质量非常重要,因此 cGAN 比 GAN 要好得多。但是请注意,添加 L1 项也会鼓励输出遵守输入,因为 L1 损失惩罚地面实况输出和合成输出之间的距离,地面实况输出与输入匹配,而合成输出可能不匹配。相应地,L1+GAN 也能够创建尊重输入标签映射的逼真渲染。结合所有项,L1+cGAN 的性能也非常好。

条件生成对抗网络的一个引人注目的效应是它们产生了锐利的图像,即使在输入标签映射中不存在空间结构,也能幻化出来。人们可能会想象 cGAN 对光谱维度的“锐化”也有类似的效果,即使图像中不存在颜色,也能使图像更加丰富多彩。就像当不确定边缘的确切位置时,L1 将激励模糊一样,当不确定像素应该采用几种可能的颜色值中的哪一个时,L1 也将激励平均的灰色。特别地,L1 将通过在可能颜色上选择条件概率密度函数的中值来最小化。另一方面,对抗损失在原则上可以意识到灰色输出是不现实的,并鼓励匹配真实的颜色分布。在图 7 中,我们调查了我们的 cGAN 在 Cityscapes 数据集上是否实际实现了这种效果。这些图表显示了在 Lab 颜色空间中输出颜色值的边缘分布。虚线表示地面真实分布。显然,L1 会导致比地面真实分布更窄的分布,证实了 L1 鼓励平均灰色的假设。另一方面,使用 cGAN 将输出分布推向地面真实分布。

4.3 生成器结构的分析

U-Net 结构允许低层次信息在网络中进行快捷连接,这是否会导致更好的结果呢? 图 5 和表 2 比较了 U-Net 和编码器-解码器在城市景观生成上的表现。编码器

-解码器仅通过切断 U-Net 中的跳跃连接来创建。在我们的实验中,编码器-解码器无法学习生成逼真的图像。U-Net 的优点似乎不仅仅适用于条件 GAN: 当 U-Net 和编码器-解码器都使用 L1 损失进行训练时,U-Net 再次取得了优秀的结果。

4.4 从 PixelGANs 到 PatchGANs 到 ImageGANs

我们测试了鉴别器感受野的不同裁剪尺寸 N 的影响,从一个 1×1 的“PixelGAN”到一个完整的 286×286 的“ImageGAN”¹。图 6 展示了这项分析的定性结果,表 3 则用 FCN-score 量化了影响。请注意,在本文的其他部分,除非另有说明,所有实验均使用 70×70 的 PatchGANs。而在本节中,所有实验均使用 L1+cGAN 损失函数。

PixelGAN 对空间锐度没有影响,但会增加结果的色彩鲜艳度(在图 7 中进行了量化)。例如,当使用 L1 损失进行训练时,图 6 中的公交车被涂成了灰色,但是使用 PixelGAN 损失后,公交车变成了红色。颜色直方图匹配是图像处理中常见的问题[49],而 PixelGAN 可能是一个有前途的轻量级解决方案。

使用 16×16 的 PatchGAN 足以促进清晰的输出,并且获得了良好的 FCN-score,但也会导致平铺伪影。 70×70 的 PatchGAN 缓解了这些伪影,并实现了略微更好的分数。超出此规模,到完整的 286×286 的 ImageGAN,并没有改善结果的视觉质量,事实上,FCN-score 显著降低(见表 3)。这可能是因为 ImageGAN 的参数更多,深度更大,比 70×70 的 PatchGAN 更难训练。

全卷积翻译 PatchGAN 的一个优点是可将固定尺寸的 patch 鉴别器应用于任意大小的图像。我们也可以对比发生器进行卷积操作,用于比其训练图像更大的图像上。我们在地图航拍照任务上进行了测试。在 256×256 图像上训练一个生成器后,我们在 512×512 图像上进行了测试。图 8 的结果展示了这种方法的有效性。

4.5 图像感知评估

我们在地图↔航拍照和灰度↔彩色的任务上验证了我们结果的感知逼真度。我们在地图↔照片任务的 AMT 实验结果如表 4 所示。我们的方法生成的航拍照片在 18.9%的试验中欺骗了参与者,显著高于 L1 基线,后者产生模糊的结果,并且几乎从未欺骗参与者。相比之下,在照片↔地图的方向上,我们的方法只在 6.1%的试验中欺骗了参与者,这与 L1 基线的表现没有显著差异(基于 bootstrap 测试)。这可能

是因为在地图中,微小的结构错误更加明显,因为地图具有刚性几何结构,而航拍照则更加混乱。

我们在 ImageNet [51]上训练了着色,然后在[62, 35]引入的测试集上进行了测试。我们的方法,采用 L1 + cGAN 损失,让参与者在 22.5%的测试中被愚弄了(见表 5)。我们还测试了[62]的结果和其使用 L2 损失的变体(详见[62])。条件 GAN 的得分与[62]的 L2 变体类似(差异不显著),但不及[62]的全方法,在我们的实验中,该方法在 27.8%的测试中让参与者被愚弄。我们注意到,他们的方法是专门设计用于着色的。

4.6 语义分割

条件生成对抗网络(Conditional GANs)似乎对输出高度详细或照片般的问题有效,这在图像处理和图形任务中很常见。那么对于像语义分割这样输出比输入更简单的视觉问题呢?

为了开始测试这一点,我们在城市景观照片标签上训练了一个带/不带 L1 损失的条件 GAN。图 10 显示了定性结果,表 6 报告了定量分类准确率。有趣的是,训练时没有 L1 损失的 cGAN 能够以合理的准确度解决这个问题。据我们所知,这是第一个成功生成“标签”的 GAN 演示,这些标签几乎是离散的,而不是连续变化的“图像”。尽管 cGAN 取得了一定的成功,但它们远不是解决这个问题最好的方法:如表 6 所示,仅使用 L1 回归得到的分数比使用 cGAN 更好。我们认为,对于视觉问题,目标(即预测与真实值接近的输出)可能比图形任务的目标不那么模糊,重建损失如 L1 通常足够。

4.7 社区驱动的研究

自从我们的论文和 pix2pix 代码库首次发布以来,包括计算机视觉和图形学从业者以及视觉艺术家在内的 Twitter 社区已经成功地将我们的框架应用于各种新颖的图像到图像翻译任务,远远超出了原始论文的范围。图 11 和图 12 仅展示了来自 #pix2pix 标签的一些示例,包括背景去除、调色板生成、素描↔肖像、素描↔Pokemon、“Do as I Do”姿势转移、学习如何看待:忧郁的星期天,以及奇怪的流行标签 #edges2cats 和 #fotogenerator。请注意,这些应用程序是创意项目,不是在受

控的科学条件下获得的,并且可能依赖于我们发布的 `pix2pix` 代码的一些修改。尽管如此,它们展示了我们的方法作为图像到图像翻译问题的通用商品工具的潜力。

第 5 章 结论

本文的结果表明,有条件的对抗网络是许多图像到图像转换任务的一种有前途的方法,特别是涉及高度结构化图形输出的任务。这些网络学习适用于手头任务和数据的损失函数,使它们适用于各种情况。

我们感谢 Richard Zhang、Deepak Pathak 和 Shubham Tulsiani 的有益讨论,感谢 Saining Xie 对 HED 边缘检测器的帮助,感谢网络社区对许多应用程序的探索和改进建议。感谢 Christopher Hesse、Memo Akten、Kaihu Chen、Jack Qiao、Mario Klingemann、Brannon Dorsey、Gerda Bosman、Ivy Tsai 和 Yann LeCun 允许在图 11 和图 12 中使用他们的作品。本研究部分得到了美国国家科学基金会 SMA-1514512、美国国家地理空间情报局 NURI、美国情报高级研究计划局、英特尔公司、伯克利深度驾驶和 Nvidia 捐赠的硬件设备的支持。J.-Y.Z. 获得 Facebook 研究生奖学金的支持。声明: 本文所述观点和结论仅代表作者个人观点,不应被解释为 IARPA、AFRL 或美国政府的官方政策或认可,无论是明示的还是暗示的。